

PADSI

Container like file and network resources segmentation for the Linux desktop

For version 0.9.30

Contents

1	Introduction	4
1.1	Context	4
1.2	Features & requirements	4
1.2.1	Missing features	5
1.3	UX	5
1.4	High level concepts	7
1.4.1	Zone	7
1.4.2	Network resource	8
1.4.3	Virtual machine	8
1.5	Virtual machines's associated concepts	8
1.5.1	VM version	8
1.5.2	Per user customization	8
1.5.3	VM's nickname	9
1.5.4	VM viewer	9
1.6	Threat model	10
2	Architecture	11
2.1	PADSI services	11
2.1.1	Systemd service	11
2.1.2	User service	11
2.1.3	Zone service	12
2.1.4	Admin. service	12
2.2	Files isolation	12
2.2.1	Files isolation applied to zones	13
2.2.2	Files isolation applied to virtual machines	14
2.3	Network isolation	15
2.3.1	Overview	15
2.3.2	Host network restriction	16
2.3.3	Infrastructure dedicated to each zone	17
2.3.4	DNS & Firewall services	18
2.4	Routing traffic to the outside world	20
2.5	Routing traffic though a traffic shaper	21
2.5.1	WireGuard VPN	21
2.5.2	Other VPN types	22
2.6	Hotplug events	23
2.7	Mounting filesystems (using FUSE)	24
2.7.1	Limitations	25
2.8	Desktop integration	25

2.8.1	Desktop file entries	25
2.8.2	Outlined windows	27
2.8.3	Outlined icons	27
2.8.4	Clipboard sharing restrictions	28
2.8.5	X11 support	28
2.8.6	Desktop features	28
2.8.7	D-Bus router	28
2.9	Security devices integration	29
2.9.1	Smartcards	29
2.9.2	FIDO2 authenticators	30
2.9.3	GPG card	30
2.10	Infrastructure for the Web access	30
2.10.1	Behavior of the Web proxy	30
2.10.2	Web proxy's architecture	31
2.10.3	Web redirection's architecture	32
2.11	Flatpak applications	35
2.12	Virtual machines	35
2.12.1	Forcing the usage of virtual machines	35
2.12.2	About VM versions	35
2.12.3	Privileges management	36
2.12.4	QEMU snapshots	36
2.12.5	VM version installation / import	36
2.12.6	VM version run	37
2.12.7	VM version update	38
2.12.8	VM version save and load	39
2.12.9	VM versions merge and cleanups	40
2.12.10	Relation to the zone' network infrastructure	41
2.12.11	Relation to zones	43
2.12.12	VM viewer	43
2.12.13	SSH connections	43
2.12.14	USB redirection	43
2.12.15	PADSI VM agent	45
2.12.16	Miscellaneous	46
2.13	"Bubbles" implementation	47
2.14	Command line tools	47
2.14.1	User interactions	47
2.14.2	For diagnostics	48
2.14.3	For administrators	48
2.15	Logging	48
2.15.1	Netfilter logging	49
2.16	System administration	49
2.16.1	Remote access	49
2.16.2	Local administration	51
2.16.3	padsi-do tool	52
2.17	Misc.	52
2.17.1	Controlled access to zone's associated files	52
2.17.2	Other	52
2.17.3	Side benefits	52
3	Using PADSI	52
3.1	padsi-cli	53
3.1.1	Getting information from the system	53
3.1.2	Running programs	53
3.1.3	Managing virtual machines	53
3.2	VM viewer	54
3.3	User defined desktop entry (launcher) files	54
3.4	padsi-do	55

3.5	<code>padsi-sys</code>	55
4	Installation	56
4.1	Requirements	56
4.2	Installation	56
4.3	VM load quick start	57
4.4	VM installation quick start	57
5	Configuration and administration	59
5.1	Configuration	59
5.1.1	Global configuration	59
5.1.2	Zones configuration	60
5.1.3	Traffic shaper	62
5.1.4	Network rules	63
5.1.5	Network resources sets	63
5.1.6	Virtual machines configuration	64
5.1.7	Administration namespaces' configuration	65
5.2	Patching / new versions of PADSI	65
5.3	System administration	65
5.3.1	Remote access	65
5.3.2	Local administration	66
5.3.3	Unattended upgrades (Debian specific)	67
6	Virtual machines administration	67
6.1	Creating a VM disk image	67
6.1.1	VM import from an externally created VM	67
6.1.2	VM installation using PADSI's tools	68
6.2	Updating a VM version	69
6.3	Updating the <code>padsi-agent</code> program	69
6.3.1	In a Linux VM	69
6.3.2	In a Windows VM	69
7	Development	69
7.1	Technologies used	69
7.2	Source code layout	70
7.3	Locations of files used by a running instance	70
7.3.1	Runtime files	70
7.3.2	Persistent files	71
7.4	High level objects	71
7.5	Run components	71
7.6	Command line tools	71
7.6.1	<code>padsi-cli</code>	71
7.7	GNOME shell extension	72
7.8	D-Bus router	73
7.9	Development environment	76
7.10	Virtual Machines	77
7.10.1	Scripts	77
7.11	Misc.	77
7.11.1	Processes tree	77
7.11.2	PADSI system and user services lifecycle	78
A	Windows VM installation	79
B	Screen sharing	79

C	Wayland proxy	80
C.1	Connection to a Wayland compositor’s cinematic	80
C.2	Copy/paste cinematic	81
C.3	Implementation details	82
C.3.1	X11 filtering	82

1 Introduction

1.1 Context

PC endpoints are a major compromise vector as they are usually quite poorly secured and have a network access to most of the resources available internally. This situation leads to daily catastrophic enterprises impacts as threat actors easily build upon the compromise of a single PC endpoint.

The PADSİ project aims to drastically reduce the risks posed by PC endpoints by isolating network and files accesses within the boundaries of business use cases. PADSİ also aims at reducing as much as possible the number of elements in the infrastructure in order to limit the attack surface, and focuses on simplicity and observability.

Use case examples include “office work” (using e-mail, working on documents, presentations, and using business specific applications, ...) and “Internet surfing” (information research, public information consultation, ...). Specifically:

- When in the “office work” use case, the user needs an access to enterprise resources (being on premise or located in external cloud environments) and associated information stored locally on files, but has no need for accesses to unrelated services exposed on the Internet;
- When in the “Internet surfing” use case, the use needs to access publicly available resources over the Internet, but not access to work related resources, and neither to work related files.

This very simple isolation concept strongly mitigates the following most prevalent attack vectors:

- credentials phishing, where the user typically receives an email, clicks a link and is redirected to an attacker controlled server where he is asked to enter his credentials: because this attacker controlled server is not in the “office work” use case, the associated web page cannot be displayed and the user can’t provide his credentials. A side benefit is the user can easily identify he was victim of a phishing attempt.
- ransomware: where a threat actor has compromised the system and is able to execute an encryptor. If the compromised program is while in the “office work” use case, the threat actor has no way of communicating with the compromised PC endpoint and thus can’t exfiltrate information nor define encryption keys to encrypt that information¹; as a side benefit, the administrator can easily identify from the logs that the system has been compromised. If however, the compromised program is while in the “Internet surfing” use case, the threat actor can deploy his ransomware, but it can only exfiltrate and encrypt non work related files, which makes the attack useless for him and harmless for the user.

PADSİ thus proposes fine grained isolation (implemented as “containers like” zones) to define as many use cases as necessary, each use case isolation is defined a **zone**. It is particularly aimed as system administrators where the impacts of a PC endpoint compromise are the highest.

Also, because the PADSİ system aims at covering many use cases, system administrators can provide virtual machines to be used by some users in some the context of some use cases. These virtual machines have at least the same restrictions in terms of network and files access as is defined for the use case and can have even more restrictions if deemed necessary.

Finally, as a side benefit, because the access to non professional services can be strictly restricted, PADSİ can act a very simple yet effective data loss prevention system.

1.2 Features & requirements

PADSİ’s features are:

- isolation of user’s work (within “zones”) based on filesystem and network filtering;

¹The threat actor can of course still work “blindly” and destroy information, but this is then not a ransomware but a wiper and is another type of threat.

- support for multiple users logged in at the same time;
- network filtering defined as rules (allow or deny) based on either IP addresses or domains which can be further specialized with transport protocols (TCP or UDP) and port numbers;
- sharing of directories among zones to allow inter-zone files transfer (in a read-write or read-only fashion);
- usage of virtual machines (Windows and Linux) for specific jobs;
- enhanced security via virtual machines isolation instead of sandbox isolation (refer to section [2.12.1](#));
- polished UX to help the user understand his environment;
- easy installation, configuration and administration, and easy integration in existing infrastructures;

As for the requirements, they are:

- a recent Linux OS in which modern kernel namespaces isolation mechanisms are available;
- a CPU with virtualization extensions to use virtual machines; enough RAM also needs to be anticipated in that case.

1.2.1 Missing features

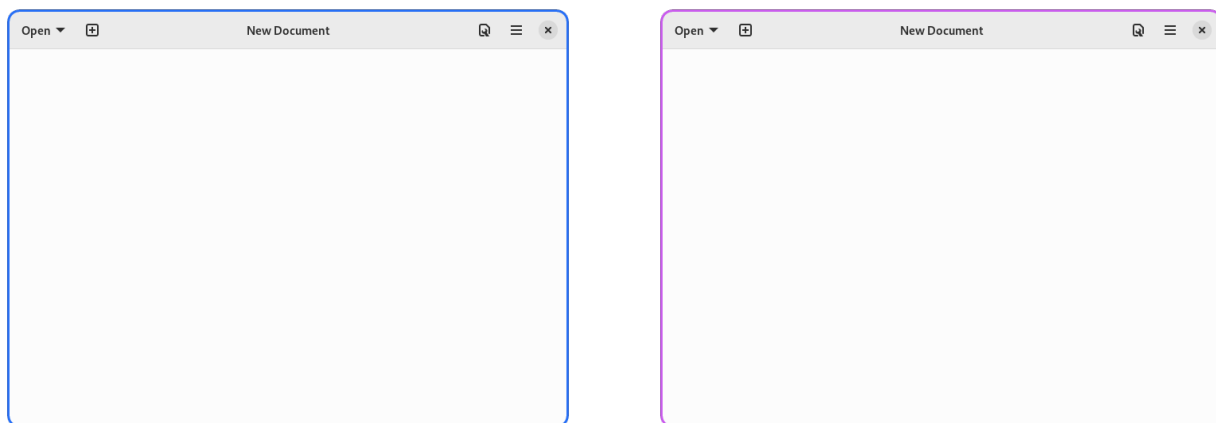
Some missing features which will eventually be implemented include:

- seamless integration of VM's applications windows in the desktop to avoid having to use a VM's own desktop;

1.3 UX

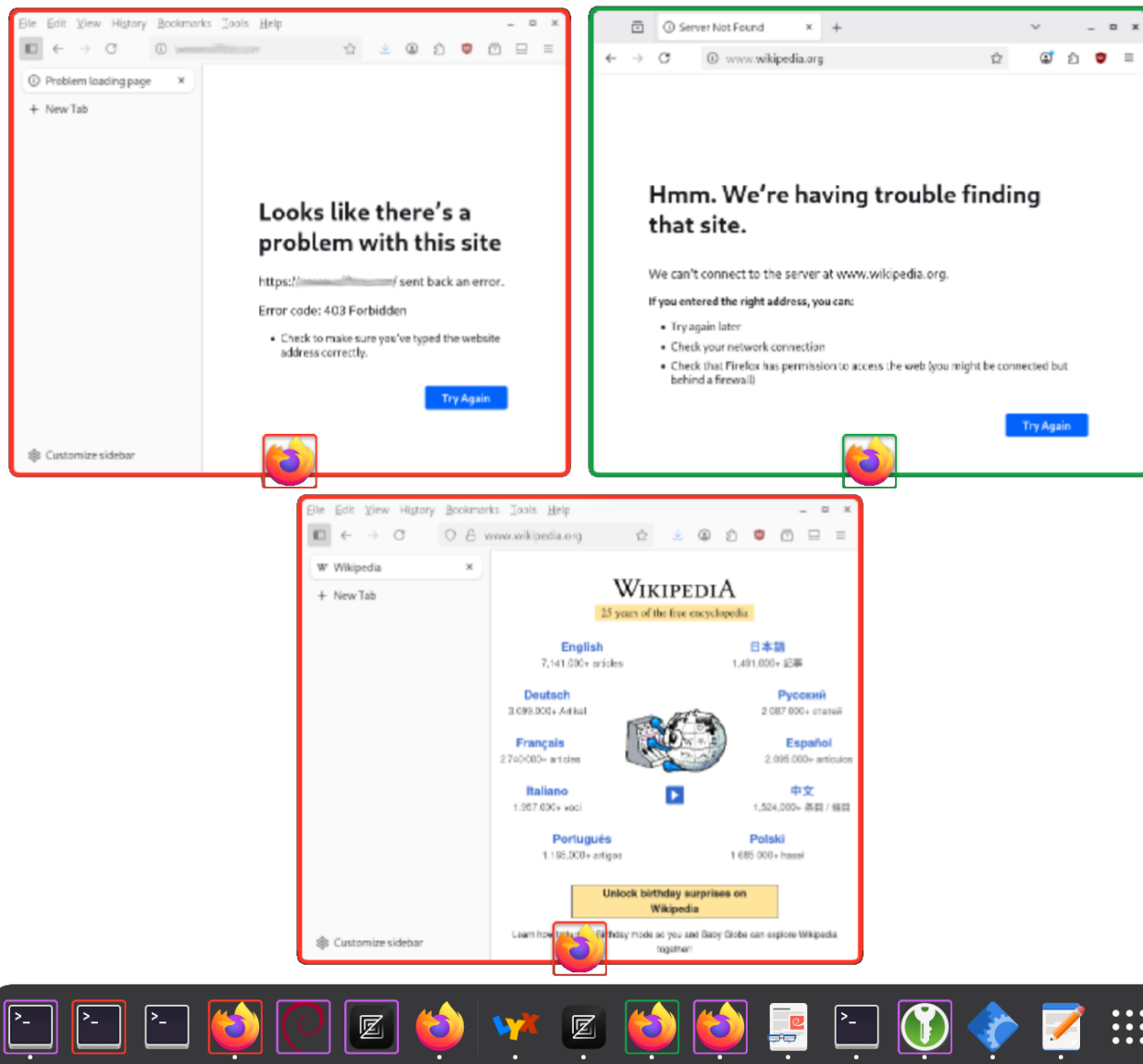
PADSI's design choices have been guided by security first and UX second (with however security features being as little intrusive as possible for the user), and PADSI's configuration and management is made as easy as possible for administrators.

To let the end user in which use case an application is used and thus in which zone that application is executed, the desktop environment adds an outline to each of the displayed window with the zone's color. For example the following windows correspond to two instances of the same GNOME text editor program executed each in its own zone as shown by the blue and purple outlines of each window²:



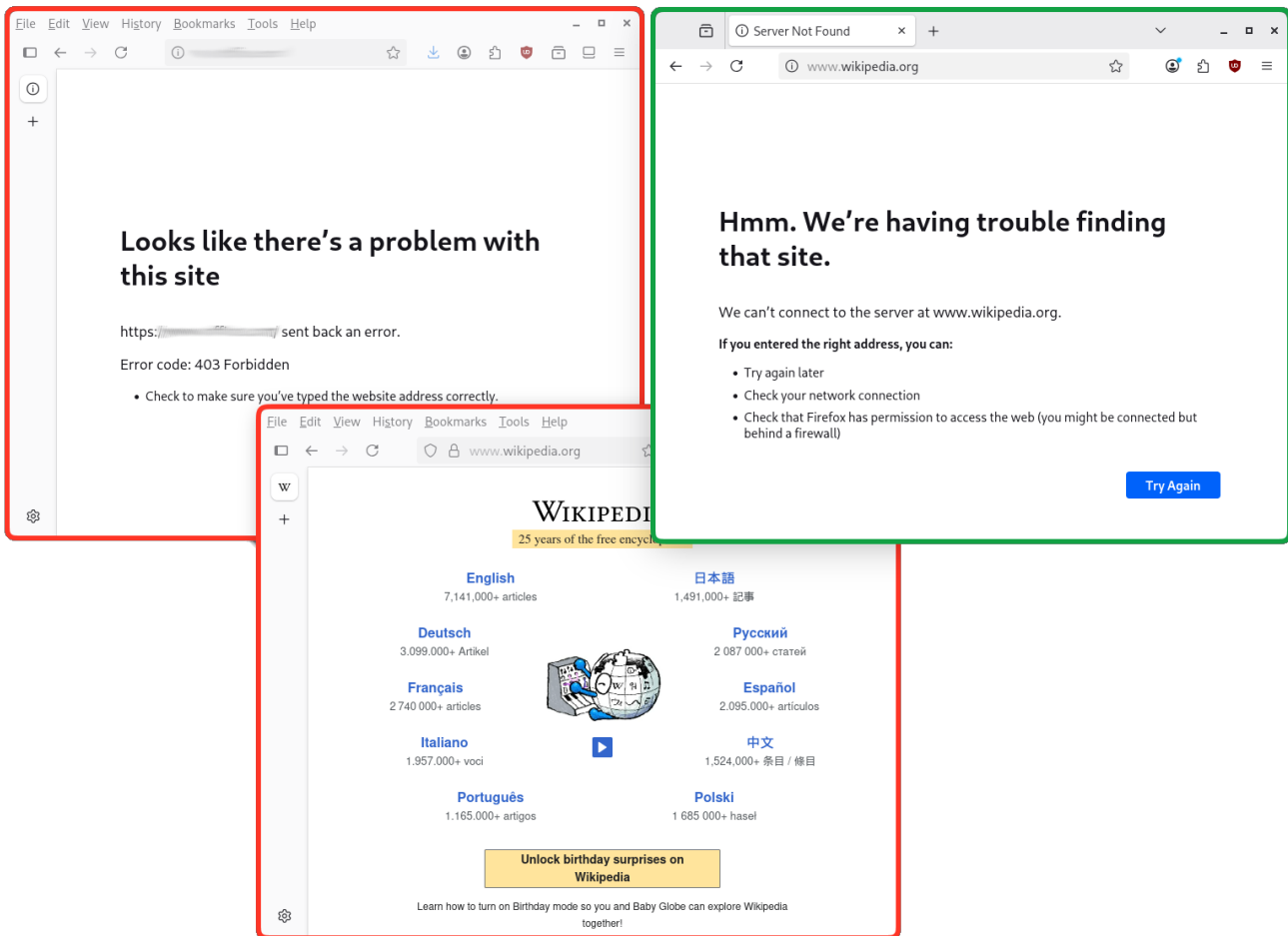
Similarly, the application's icon (used before the application is launched and when it's running) also has a similar outline in the same color, as shown in the following screenshot (where more applications than actually displayed are in different workspaces):

²The color of each zone is chosen by the system administrator and set among each zone's properties.



To illustrate how network access filtering might appear, the screenshot below shows two Firefox web browser running in a zone where Internet access is allowed but prevents access to “business applications” (outlined in orange), and in a zone where broad Internet access is blocked and only some specific business applications can be used (outlined in green). Note the following:

- browser settings differ per zone (one shows the vertical tabs in its sidebar, whereas tabs are horizontal in the other), as each zone has its own user settings;
- being a public Web site (nos business here), access to the Wikipedia is blocked in the “green” zone whereas it displays nicely in the “orange” zone;
- similarly, access to a business application is blocked in the “orange” (and would be allowed in the “green” one if it was shown here);
- finally, the error messages where access is blocked differ in this case as the failure to connect reason for the browser’s perspective varies (a Web proxy blocked the connection in the “orange” zone and the DNS failed to resolve “www.wikipedia.org” in the “green” zone). Of course the situation actually depends on how PADSII is configured.



Finally note that end users are expected to:

- have knowledge of the use cases which have been implemented and what can be done in each use case;
- have been informed about the constraints imposed for each use case in terms of network and files access.

1.4 High level concepts

Before reading the rest of the document or using PADS, the following concepts must be understood.

1.4.1 Zone

A “zone” is a logical description of a set of file and network resources in which a user’s programs are executed in order to perform some work around a use case. For example, a “work zone” might contain all the work related documents (office documents, images, credentials, passwords, safes and the like) and provides access to network file shares, e-mail, and other business related Web services. This zone would be different than the “Internet zone” where the user performs public Web searches, reads blogs or access social media and would only contain the user’s bookmarks and a few downloaded files for local consumption.

From a technological point of view, PADS makes a heavy use of applications network and files isolation to create small isolated “container like” areas in which programs achieving the same goal are. Examples of such isolated areas are typically the user’s programs in a zone, a virtual machine and its associated visualization program, or the network infrastructure used by the programs in a zone. PADS uses the generic term “**bubble**” to define such areas (refer to section 2.13 for details).

Also, even though the user sees each zone as a unique entity, the actual implementation will often lead to several “bubbles” being started to create that illusion: some “bubbles” will provide network services, some will run virtual machines, one or more will run the actual user’s programs, while some may run Flatpak applications. A typical zone will run between 2 and 4 “bubbles”. “Bubbles” are implemented via the Linux kernel’s namespaces.

Execution of programs outside of a zone Except from a UX point of view, allowing the user to execute programs outside of any zone is not a security risk.

The user can easily execute programs outside of any zone either by deliberate configuration (e.g. allow the Gnome Terminal to be executed in the “init zone” like the calculator would be), by user configuration (map a keyboard shortcut to execute that same program), or directly via the Gnome Shell via the Alt+F2 keyboard shortcut.

1.4.2 Network resource

A network resource is fundamentally a set of IP addresses or domain names and optionally network protocols and port numbers which can be allowed or denied. For example, one could allow to SSH into a specific server using its IP address with a rule “allow TCP port 22 to 10.1.2.3”, or deny access to the Youtube video platform with a rule like “deny youtube.com.”.

1.4.3 Virtual machine

Some of the tools users need may not be available directly on the system because maybe they are not packaged, maybe the administrator does not want the software to be installed on the system, maybe the software is obsolete, maybe it requires Windows, or maybe the tool requires administrative privileges. For all these reasons (and probably many others), PADSIs lets administrators create virtual machines (**VM**) and users “run” them within zones, thus with the same (or even greater) isolation from the system as any other program running in the same zone. Because of that isolation, system administrators can decide (based on risks assessments) to give end user complete control (i.e. administrative privileges) over the OS running in the VM.

Virtual machines can run any operating system supported by the [QEMU](#) emulator, but proper files and desktop integration is performed only for Windows and most Linux distributions.

However, contrary to the usage of traditional virtual machines, PADSIs is based on the assumption that virtual machines are used as temporary tools instantiated on demand and destroyed immediately after (of course an user can keep using a virtual machine as long as the administrator will allow for).

Programs running in virtual machines can be given access to the files and directories present in the zone in which they are executed, so the user can use the same resources when executing programs in the virtual machine or in the associated zone.

In this context, virtual machines are throwable objects, and from a security standpoint, this ensures that any compromised virtual machine does not offer a foothold in the infrastructure to the attacker.

1.5 Virtual machines’s associated concepts

1.5.1 VM version

As stated above, virtual machines in PADSIs are meant to be short lived (depending of course on the user’s use cases), which is why PADSIs distinguishes a VM’s definition as presented in section 5.1.6 from all the different versions in terms of actual VM disk image files which can exist at any given time. Specifically, a “**VM version**” is a set of the the following files³ sharing the same VM definition in term of configuration:

- a VM disk image file (with the `.img` extension);
- a VM’s UEFI’s NVRAM [OVMF](#) file (with the `.vars` extension); and
- a VM’s information file (with the `.infos` extension).

Thus when a VM is “**running**”, PADSIs actually uses the files associated to a specific VM version. This is done in a way such that the disk space used by the VM versions is kept to a minimum and that users and administrators can do their jobs at the same time (thus without requiring any sort of coordination).

1.5.2 Per user customization

In order to limit the administrators’ work to the minimum (installing a “VM master”) while at the same time having per user VM, before a user can actually run a VM, PADSIs imposes a **user customization** phase of the VM, which, at least:

³Note that these files are treated by PADSIs as a whole and evolve simultaneously when VMs are used.

- creates the user in the VM (with the same name and possibly UID/GID as in the host); and
- sets up auto login for that user.

The administrator can specialize further the per user VM to install some specific software, to activate licenses, or for any other need.

1.5.3 VM's nickname

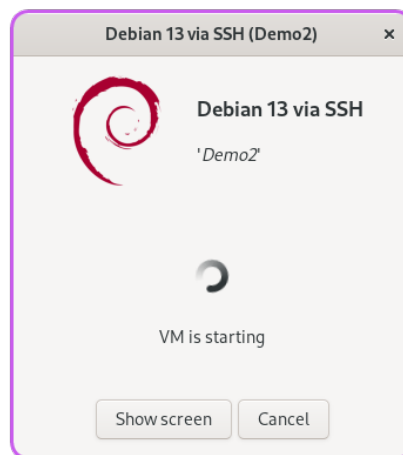
As the end user can start several VMs simultaneously, he can give each instance a “**nickname**” which can be anything as long as it starts with a letter and only contains letters, numbers and the “-” and “_” characters. If no nickname is provided, then a randomly generated one will be assigned.

A VM's nickname is also used to keep track of VMs between successive runs in case the user needs to stop a VM and restart it at a later time, and a VM can be joined via the network in a zone via its <nickname>.vm name (refer to section 2.3.4 for more details).

1.5.4 VM viewer

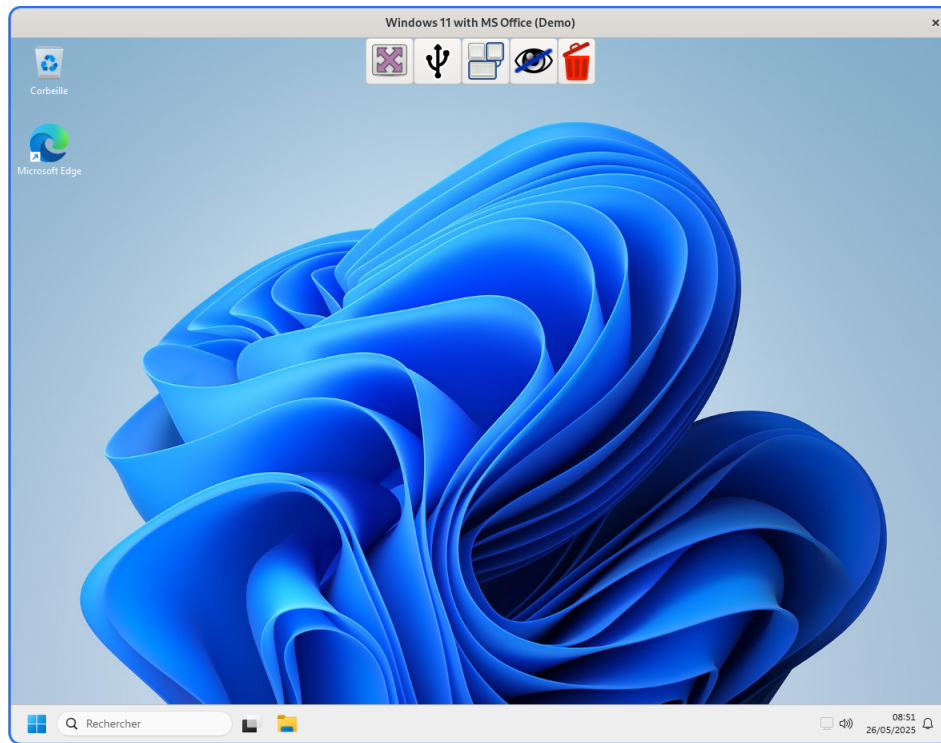
The VM viewer is the program which displays the VM's UI interface to the user while a VM is running. It is written using the GTK+ toolkit and the associated SPICE specific widgets. It has two display modes:

- a “**summary**” mode displayed when the VM starts, ex. below when a Debian 12 VM nicknamed “Demo-VM” is starting:



- a “**console**” mode automatically displayed when the VM the end user's session is actually opened⁴ in the VM which can thus makes it actually “usable”, ex. below a Windows 11:

⁴This only works if the PADSI's VM agent is installed in the VM.



Notes:

- the buttons bar at the top allows the user to interact with the VM (set it to fullscreen, attach/detach USB devices, etc.); they automatically hide and are shown again when the mouse's cursor hovers over the top of the window;
- if the PADSIs VM agent is installed in the VM, then the host name is set to the VM's nickname.

1.6 Threat model

PADSIs is meant to address the following threats:

- system compromise via an application opening a malicious specially crafted document (Web browser, PDF reader, ...) which leads to the local execution of a malicious program or piece of code by the threat actor;
- system compromise via clicking on a malicious link from a Web browser which leads to social engineering attacks where the user is lead to provide credentials or other sensitive information.

Specifically, PADSIs aims at reducing the impacts of a compromise of the system:

- on the system itself by protecting access to files and other sensitive information (like credentials exfiltration via phishing and files exfiltration and loss via ransomware);
- on the other elements of the infrastructure to which the system is connected to (via network access limitations and monitoring).

Use cases PADSIs is adapted to the following use cases:

- enterprise usage without any strong adherence to Windows, to protect access to company data and allow secure activities;
- development activities to (partially) address the risk of supply chain attacks;
- personal usage to protect one's personal information.

2 Architecture

This sections provides an architectural view of how PADSI is build and operates, and is meant to be used by people in need to understand how PADSI works under the hood (administrators, security auditors, developers, etc).

2.1 PADSI services

2.1.1 Systemd service

PADSI is run via the **PADSI systemd service**, as root which actually runs the **padsi-system-service** program.

To handle updates to the PADSI software itself after a new version of a package has been installed, the `/run/padsi-needrestart` file is created. When no user is connected, then PADSI's systemd service is restarted (after a check to make sure the deployed configuration, which may have been changed, is still valid). When the PADSI's systemd service starts, it removes any `/run/padsi-needrestart` file which may still be present.

In order to customize when the PADSI service is allowed to restart, the configuration can specify a program to execute before the restart occurs. If this program's exit status is not zero, then the restart will not be performed.

Notes:

- this optional process is executed as the “padsi” user and not root.
- the restart is triggered by sending the TERM signal to itself, which cause it to stop nicely and systemd is the one which actually restarts the service.

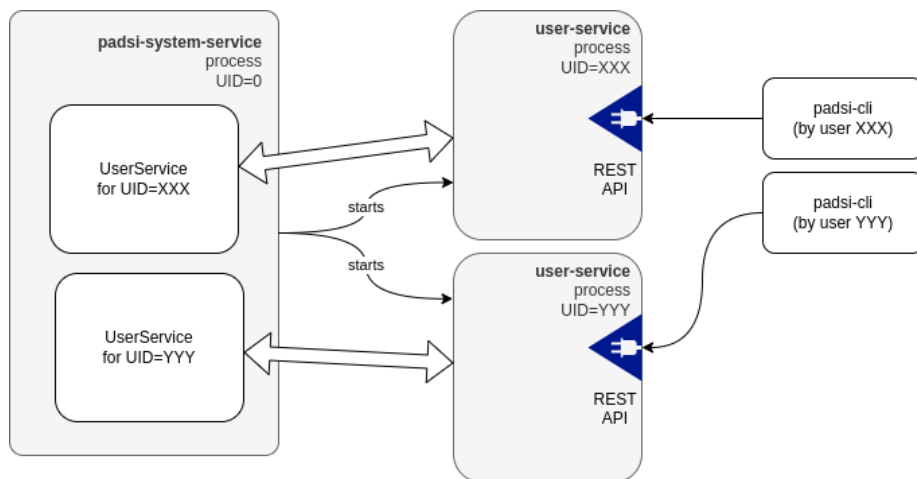
2.1.2 User service

When a user opens a desktop session, the system service spawns a **PADSI user service** (the **user-service** process) as the user which will be responsible to set up any zone or other artifact as required by the user and depending on the configuration. This user service is then killed when the user logs out.

The user service communicates with the system service via dedicated pipes to ask it to execute privileged actions such as mounting drives or modifying the network configuration.

The user service in turn sets up a small REST server which listens on a Unix socket protected so that only the user can access it (a user can't access another user's dedicated similar socket). All the actions a user might want to do (start processes, VMs, ...) are performed via HTTP⁵ calls to the user service's REST service via the **padsi-cli** command line tool (or any other tool which can use HTTP). Note that the user service's REST API is only accessible from the “init” namespaces, but not from any program running in a zone.

The following diagram illustrates this point of architecture where two users (one with UID XXX and the other with UID YYY) have simultaneously opened a graphical session:



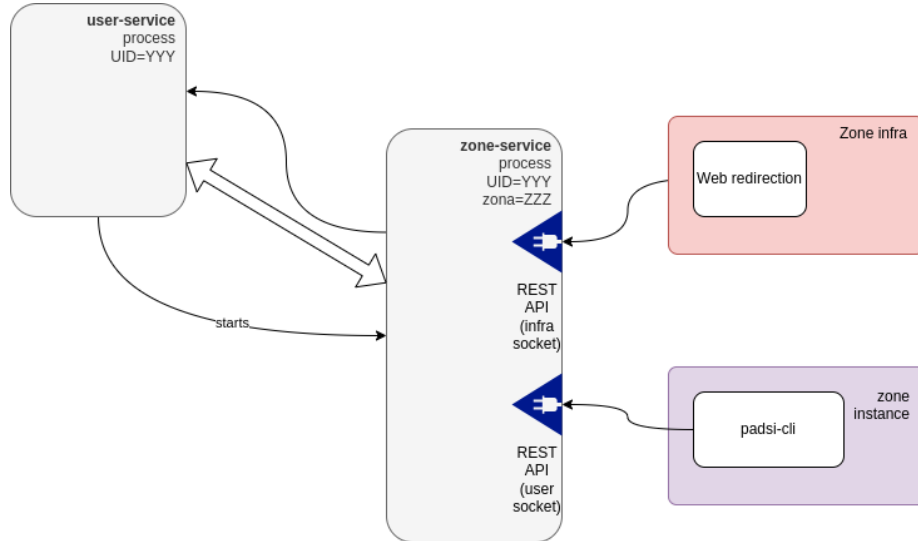
Note: the detection of a user opening a graphical session or closing it is based on the presence of a GNOME Shell process for each user. Using the presence of a per user `/run/user/<UID>` directory is not a good enough indicator.

⁵No HTTPS, access control is ensured by the system's filesystem permissions and there is little way to intercept communications taking place through Unix sockets.

2.1.3 Zone service

The **PADSI zone service** (the **zone-service** process), of which an instance is spawned by the PADSI user service for each zone which is started, is a small service available for processes running in a zone (and its associated infrastructure) which allows to use a trimmed down version of the **padscli** tool from within the zone (for example to start virtual machines) or to perform web redirection (refer to section 2.10.3).

The following diagram shows the associated architecture:



2.1.4 Admin. service

For administration purposes (refer to section 2.16, PADSI also runs an **admin. service** (the **admin-service** process) which:

- creates the administration namespaces according to the configuration;
- allows system administrators to perform virtual machine's files manipulations (import, load, save, ...)

This service, which runs as the "padsi" system user, also communicates with the system service in the same way as the user service does, for the same purposes. It also offers some REST API which can only be used by the root user or the "padsi" user.

2.2 Files isolation

Zones are defined by which files and directories processes running with them can access. Such isolation is provided by the Linux's kernel **mount namespace**.

Each zone thus has its own per user home directory, the **zone's user home directory**, in which all the user's files in that zone are located (the user has a different home directory in each zone even though the \$HOME environment variable remains the same), where all his documents are stored.

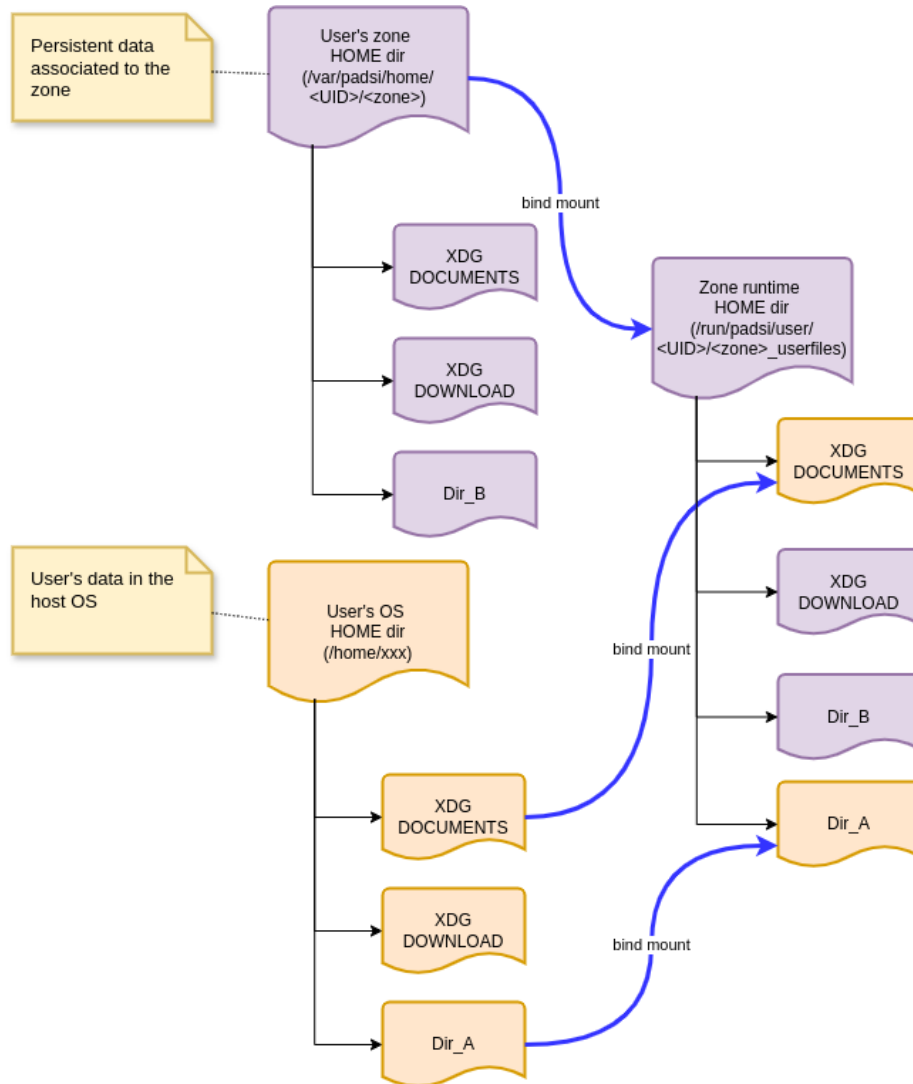
Moreover, in order to be able to share files between zones, each zone's settings can also specify bind mounts which make files or directories either from other zones or from the actual home directory of the user in the Linux system appear in the zone. For example, if a user has to download some resources from the Internet before using them from a different work zone, the work zone could bind the Internet's ~/Download directory to the ~/Internet-downloaded-stuff directory (possibly read-only if necessary).

So before the zone's "bubble" is set up, what will be the user home directory in the "bubble" is assembled as a temporary directory named **zone runtime home directory** from the zone's settings.

For example the following diagram shows:

- the actual home directory of the user in the system (bottom left) with the traditional XDG directories (like Documents/, and Downloads/ shown here), in an orange color;
- a **zone's user home directory** (top left) also with the traditional XDG directories, in a purple color,

- on the right is the **zone runtime home directory**, built as follows:
 - the “root” is a bind mount of the zone’s user home directory;
 - the Documents/ and Dir_A/ directories are binds from the the same directories in the actual home directory of the user;
 - the Download/ and Dir_B/ directories, not having been “overwritten” by a bind are from the zone’s user home directory.

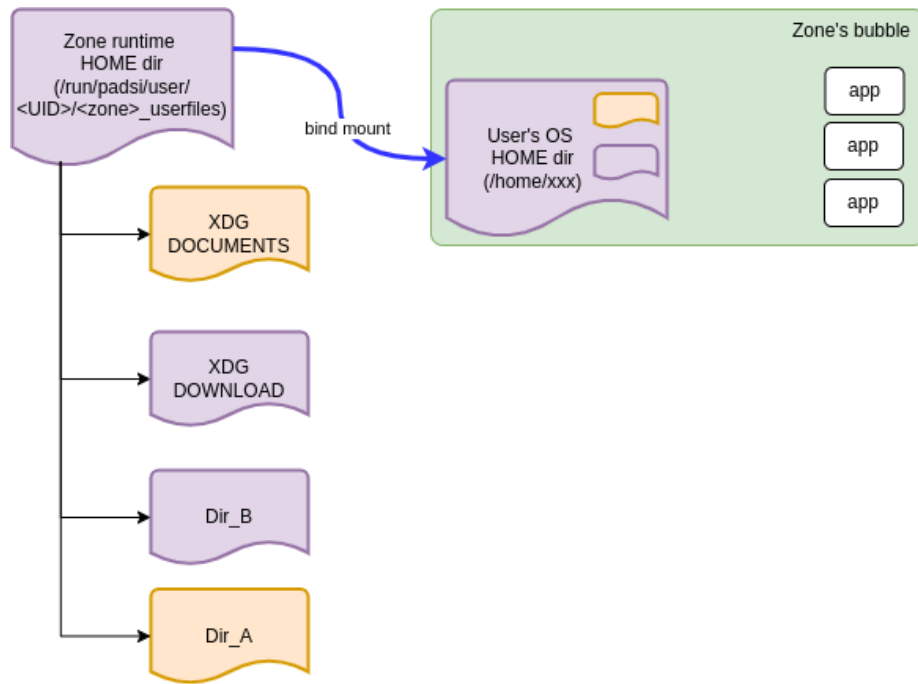


Notes about PADSI's configuration and the system configuration:

- using the actual home directory of the user is simply by design a convenience choice, any other directory could have been chosen, but it seemed right to do so, considering that, depending on PADSI's configuration and on the system configuration (not covered in this document), the user may be able to execute programs outside of any zone and still access some files from his home directory in some zones;
- in order to enforce zone's files strong isolation, programs run by the user in the “init” namespaces (i.e. outside of any zone) don't have the permissions to access any zone's user home directory for himself.

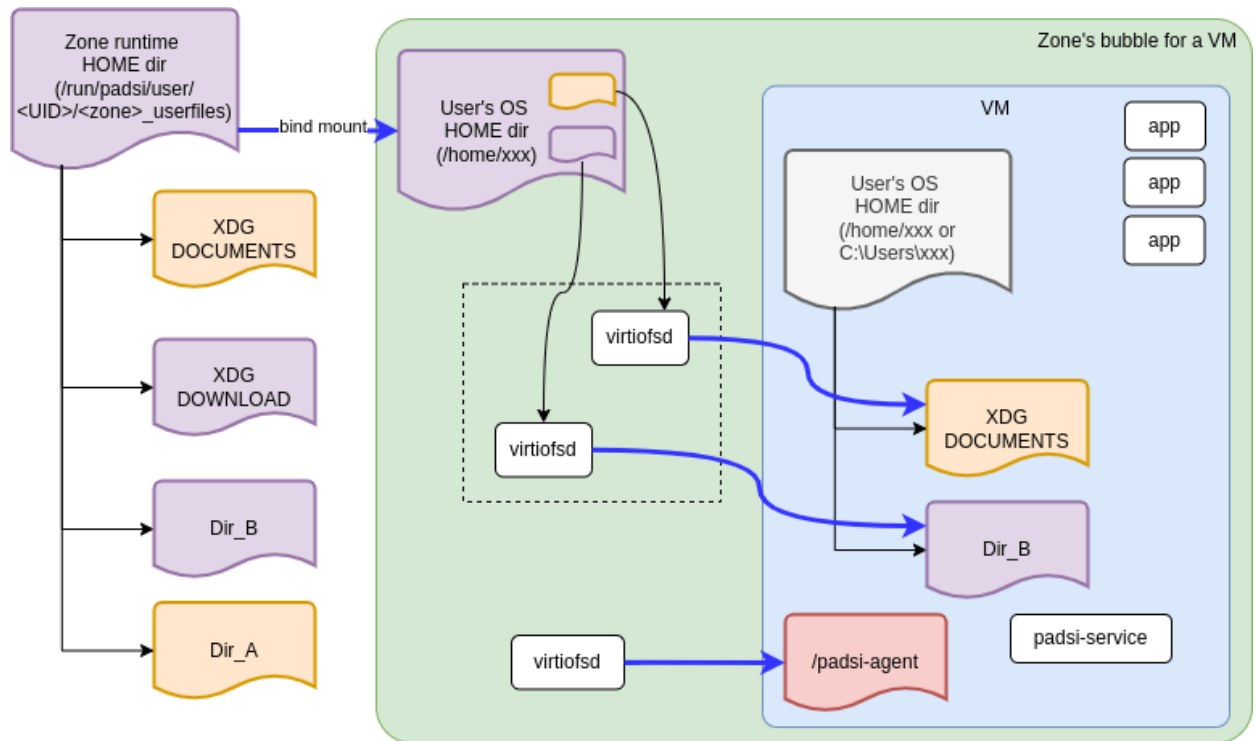
2.2.1 Files isolation applied to zones

The following diagram shows how the zone runtime home directory is then mounted (using a simple bind mount) into the “bubble” where the actual end user's programs will be executed:



2.2.2 Files isolation applied to virtual machines

In the context of virtual machines, VM, (which are actually run by the QEMU program inside their own “bubble”), the architecture is as follows, with an example where the “XDG Documents” and the “Dir_B” directories are shared within the VM (but not the other files and directories of the zone runtime home directory):



Note that:

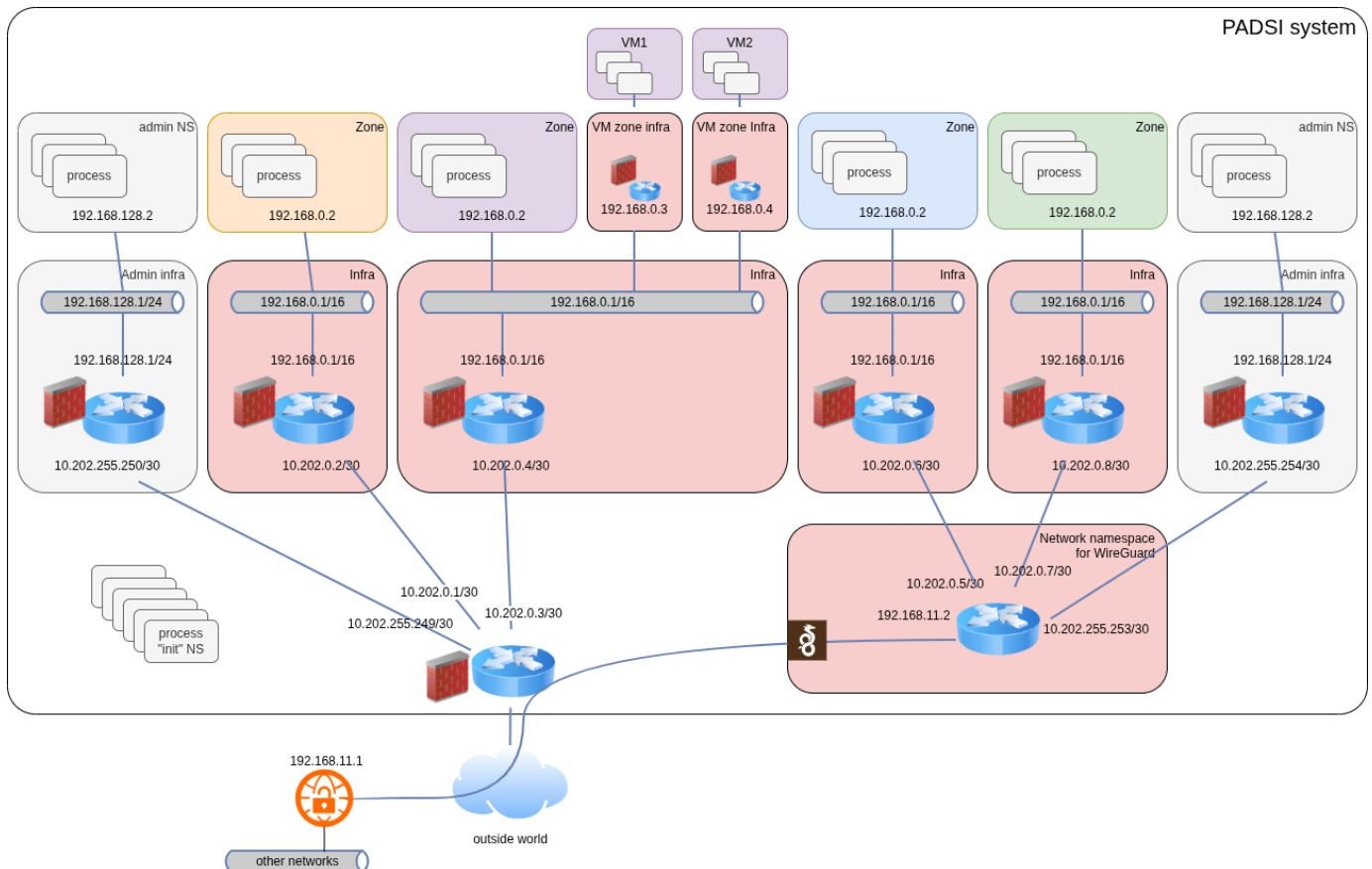
- each shared directory is actually shared by an instance of the `virtiofsd` program;
- each VM version, when running, is executed in its own “bubble”.

2.3 Network isolation

Zones are also defined by which network resources the programs host have access to, each zone thus has a dedicated network infrastructure. Network isolation is provided by the Linux's kernel **net namespace**.

2.3.1 Overview

Before digging into the technical details, here is a global overview of the network setup of a PADSI system in a typical use case where the system has a connection to the Internet, and a WireGuard VPN to securely access resources in an associated organization (bottom right of the diagram).



In this situation, the end user’s workflow is represented by 4 use cases (in orange, purple, blue and green from left to right), with the second use case to the left also relying on the usage of virtual machines (named VM1 and VM2 here) and the two uses cases on the right having all their network traffic routed through a WireGuard VPN. For each set of programs or virtual machines, PADS_I sets up a dedicated network infrastructure, described immediately in the next section, providing traffic routing among other things.

This diagram, for the sake of completeness, this schema also contains (in gray) two administration namespaces (refer to section 2.16). However, it is still a simplified view as some components in the infrastructure dedicated to a zone may also exist such as a Wayland proxy or a Web proxy.

Notes:

- every program and virtual machine is constrained:
 - is forced to use a dedicated network infrastructure which enforces the network isolation defined in PADSI's configuration (refer to section 2.3.3); or
 - for programs executed as-is in the system (systemd services, OS stuff or programs which a user managed to execute), represented in the lower left of the diagram, PADSI configures the local netfilter firewall to remove any non necessary network access (refer to section 2.3.2).

- most of the network address spaces is hard coded to use private IP addresses (192.168.0.1/16 and 10.202.x.y/30 here), using NAT to actually route traffic;
- when a WireGuard VPN is set up (where the VPN configuration imposed 192.168.11.2 IP address here), PADSİ defines a per WireGuard interface specific infrastructure to route the traffic. Note that PADSİ expects a WireGuard configuration file but does not use the `wg-quick` tool to set up the VPN.
- at the moment only IPv4 is supported.

2.3.2 Host network restriction

Many processes will run outside of any zone (all the OS processes, some processes which are configured to be executed as such, like the calculator, and maybe some processes the used managed to execute outside of any zone⁶) even when the PADSİ system is enforced. These processes' network capabilities are restricted based on PADSİ's global configuration (refer to section 5.1.1) using the system's netfilter firewall.

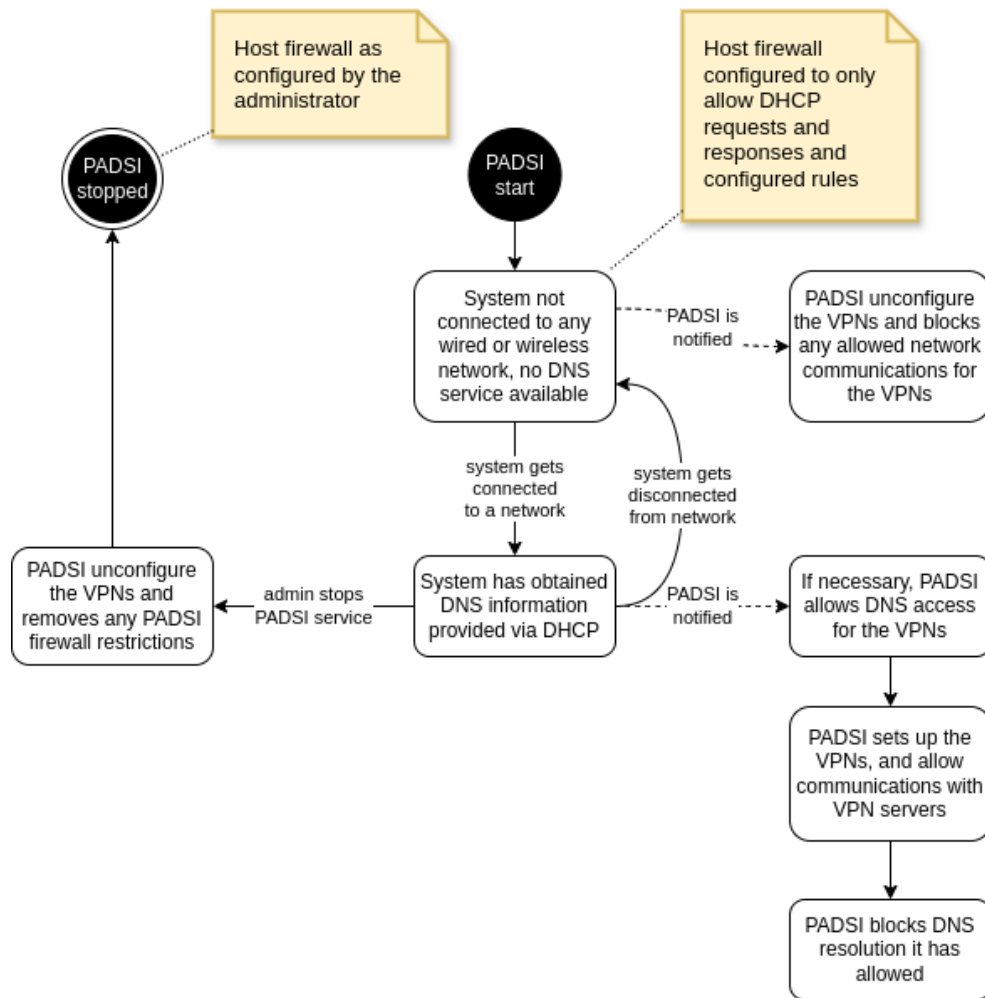
Note that some network access will automatically be allowed:

- DHCP requests and responses so the automatic network configuration of the physical (wired or wireless) network interfaces is still operational;
- DNS requests and responses up to when their are not needed anymore, to enable the usage of VPNs which endpoint is defined as a FQDN and not an IP address;
- access to VPN servers;
- communications from all the zones' network infrastructure (refer to section 2.3.3) and the established VPNs.

The system administrator may also allow for example incoming SSH connections (preferably on a randomly chosen port or with port knocking and using SSH keys) to still be able to remotely log in as a fallback mechanism.

The following diagram shows how PADSİ implements that logic from when the PADSİ service is started up to when it is stopped:

⁶Allowing the user to do so is not a security risk but will merely be a UX degradation.



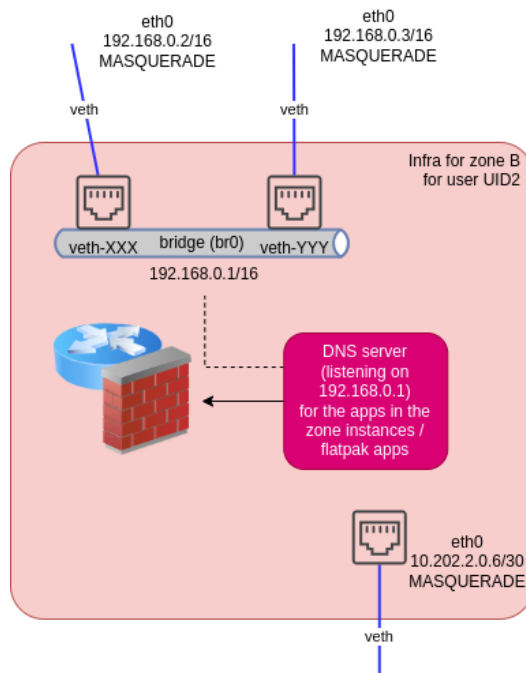
2.3.3 Infrastructure dedicated to each zone

For each zone, if allowed by the zone's configuration, PADSi sets up a dedicated network infrastructure in a "bubble", with:

- a gateway to a "lower" network infrastructure;
- a network bridge which connects the network interfaces of each "bubble" where the user's programs are executed;
- the actual network interfaces of each "bubble";
- the network services such as DNS resolution and firewall.

The following diagrams illustrates a typical "bubble" for a zone's network infrastructure:

- the bridge which always has the 192.168.0.1/16 address;
- two network interfaces `veth-XXX` and `veth-YYY` at the top (for two "bubbles" where user's programs are run) plugged into the bridge;
- a network interface `eth0` at the bottom acting as the gateway to the "outside world", which IP is allocated dynamically in the range of what is configured globally (refer to §5.1.1) through which NAT is performed;
- the DNS server, always listening on the 192.168.0.1 interface, internally relying on the DNS resolvers in the `/etc/resolv.conf` file present in the "bubble".



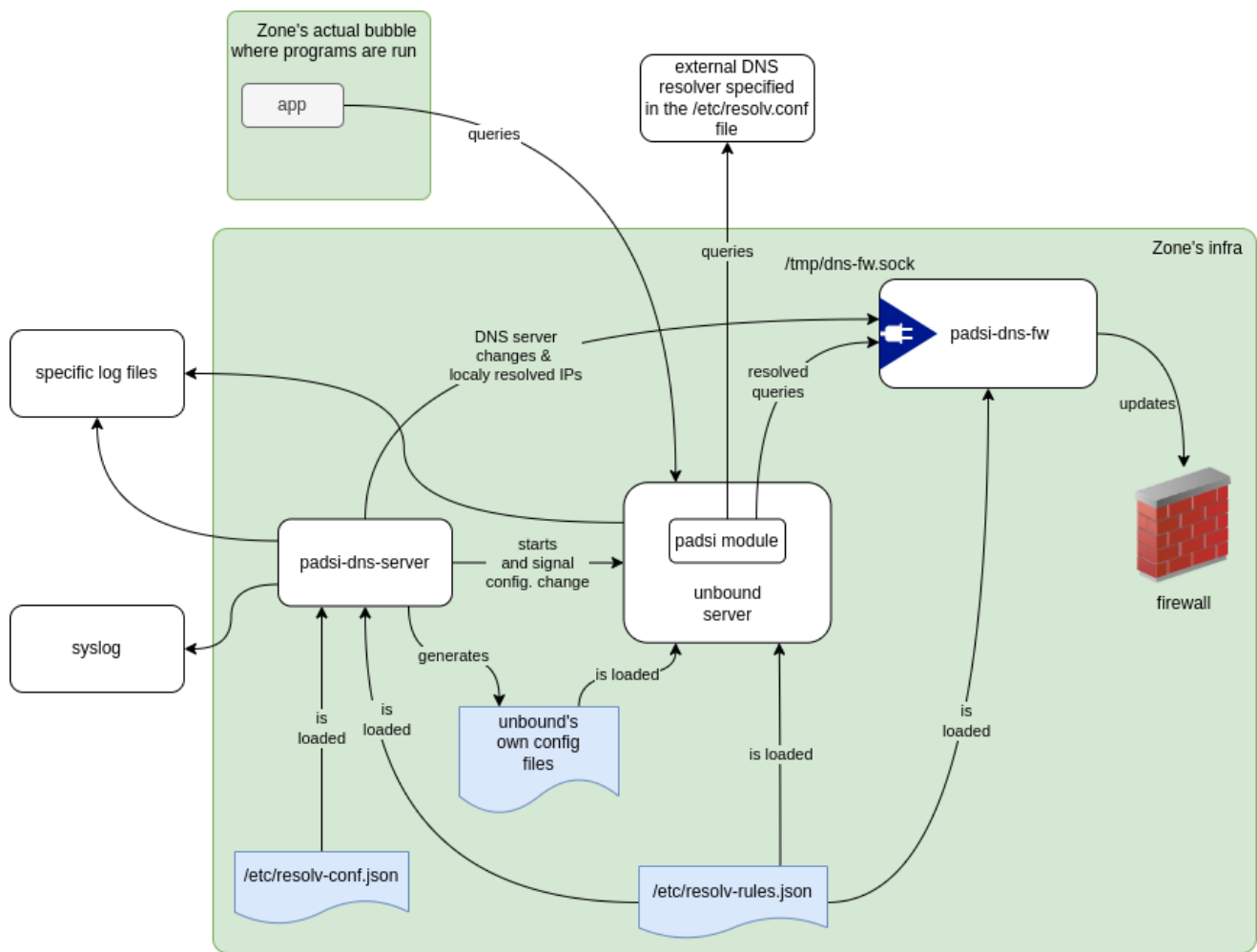
2.3.4 DNS & Firewall services

As stated above, each zone uses the DNS services of the **zone's network infrastructure** actually provided by an instance of the **unbound** DNS server. The DNS server and the firewall in that “bubble” cooperate to enforce the network restrictions defined in the zone's configuration. Namely:

- all the rules based on IP addresses (and optionally protocol type and port numbers) are directly loaded into the firewall – the communications of applications using IP addresses without DNS resolution are thus directly filtered by the firewall;
- any DNS request from an application is handled by the DNS server which may:
 - reject it;
 - resolve it itself; or
 - forward its resolution to a DNS resolver as configured in the `/etc/resolv.conf` file present in the “bubble”
- if the DNS resolution is successful, then the DNS server communicates the actual resolved IPs and the associated TTL to the firewall which will allow them up to when the TTL expires (plus some extra time to allow for long running applications which reuse previously obtained responses without taking care of the TTL);
- the DNS resolution can be forwarded to an external resolver, or be directly handled by the **unbound** DNS server itself.

The following diagram illustrates the DNS component's architecture, with:

- the **unbound** DNS server at the center of the service, which answers queries from applications running in the associated zone, with an embedded PADSISpecific module which allows or denies DNS resolutions to proceed and logs events;
- the **padsi-dns-server** program which task is to (re)configure the unbound DNS server when the DNS server to forward requests to changes (as per the `/etc/resolv.conf` file's contents) or when the filtering rules change (as per the `/etc/resolv-rules.json` file's contents).
- the **padsi-dns-fw** program which manages the configuration of the local firewall based on the configuration and the dynamic resolutions made by the **unbound** DNS server. This program also loads the `/etc/resolv-rules.json` file to extract network flow limitations in terms of protocols and ports.



Notes

- the local⁷ firewall, is implemented using Linux's netfilter and the NFTables;
- depending on the kind of events, the DNS server logs events globally using syslog and locally (refer to §2.15) as the logs directory is mounted in the “bubble” (as the `/var/log` directory);
- PADS supports unsecure (legacy) DNS resolutions and DNS over TLS (DoT).

Resolution in the context of Virtual machines When a virtual machine is started in a zone, all the programs in the same zone can reach it using one of the following names:

- `<nickname>.vm`
- `<nickname>.<VM ID>.vm`
- `<snapshot>.vm`
- `<snapshot>.<VM ID>.vm`

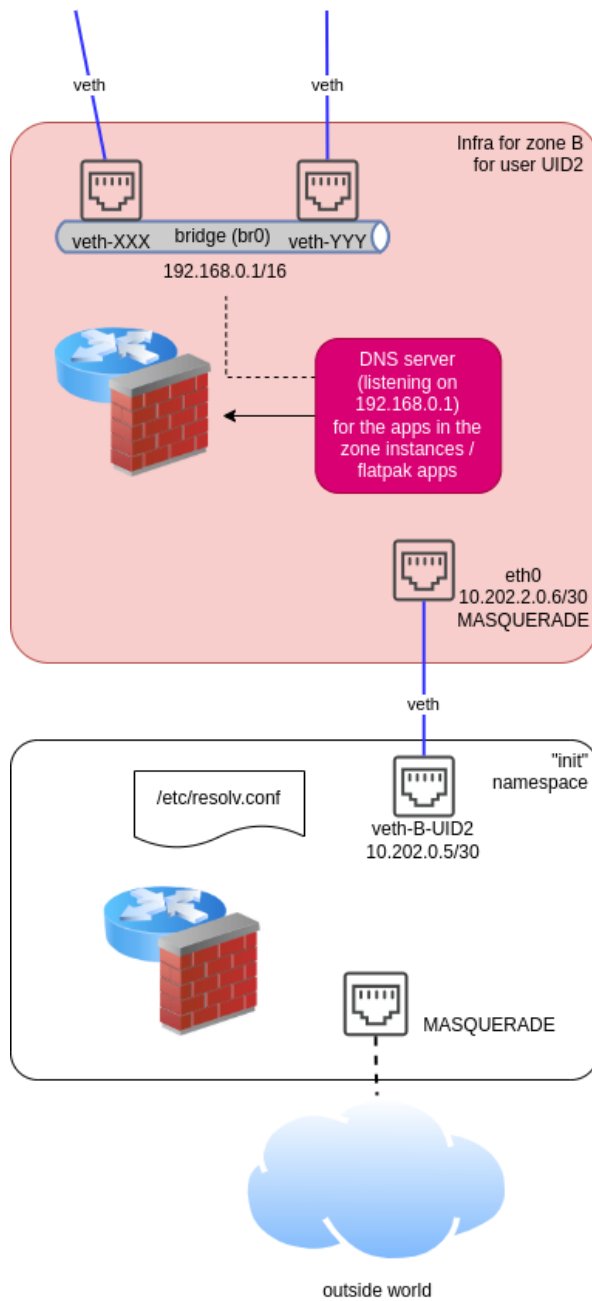
However, because one can start virtual machines from various definitions, to avoid name collisions, some of these names may already be used by previously started virtual machines. PADS prevents this situation by removing duplicates. So for instance, if the user starts a Debian VM named “default” and then starts a Suse VM also named “default” (assuming both Debian and Suse virtual machines have been defined in the configuration), then the Debian VM will be resolved as `default.vm` and `default.debian.vm` and the Suse VM will only be resolved via `default.suse.vm` (not taking into account snapshot here) because of the order in which the virtual machines have been started.

⁷The firewall rules are specific to the “bubble” which contains the zone's network infrastructure

ICMP filtering For now, to prevent potential data exfiltration or access to a command and control, ICMP packets are blocked outside of traffic within a zone. This however prevents low level network management like MTU negotiations. A future evolution might allow some ICMP types of traffic to avoid this drawback.

2.4 Routing traffic to the outside world

Each zone’s network infrastructure is associated to a network interface `veXXX`⁸ (`veth-B-UID2` in the example below) which IP address is configured globally (refer to §5.1.1). The Linux routing of the “init” network namespace then routes the traffic (with NAT) to the outside world as expected, using the network configuration set up by the user (via the NetworkManager for example).



This default situation can be changed on a per zone basis using “**traffic shapers**” (currently implemented only for VPNs), which are described in the following sections.

⁸The actual name of the interface is dynamically determined to avoid overlaps.

2.5 Routing traffic through a traffic shaper

“**Traffic shapers**” allow the system administrator to route one or more zone’s traffic through a dedicated mechanism (VPNs for now): a traffic shaper adds a network namespace which “sits in between” the zone’s infrastructure and the “init” network namespace to route (or perform other actions) on the zone’s network traffic.

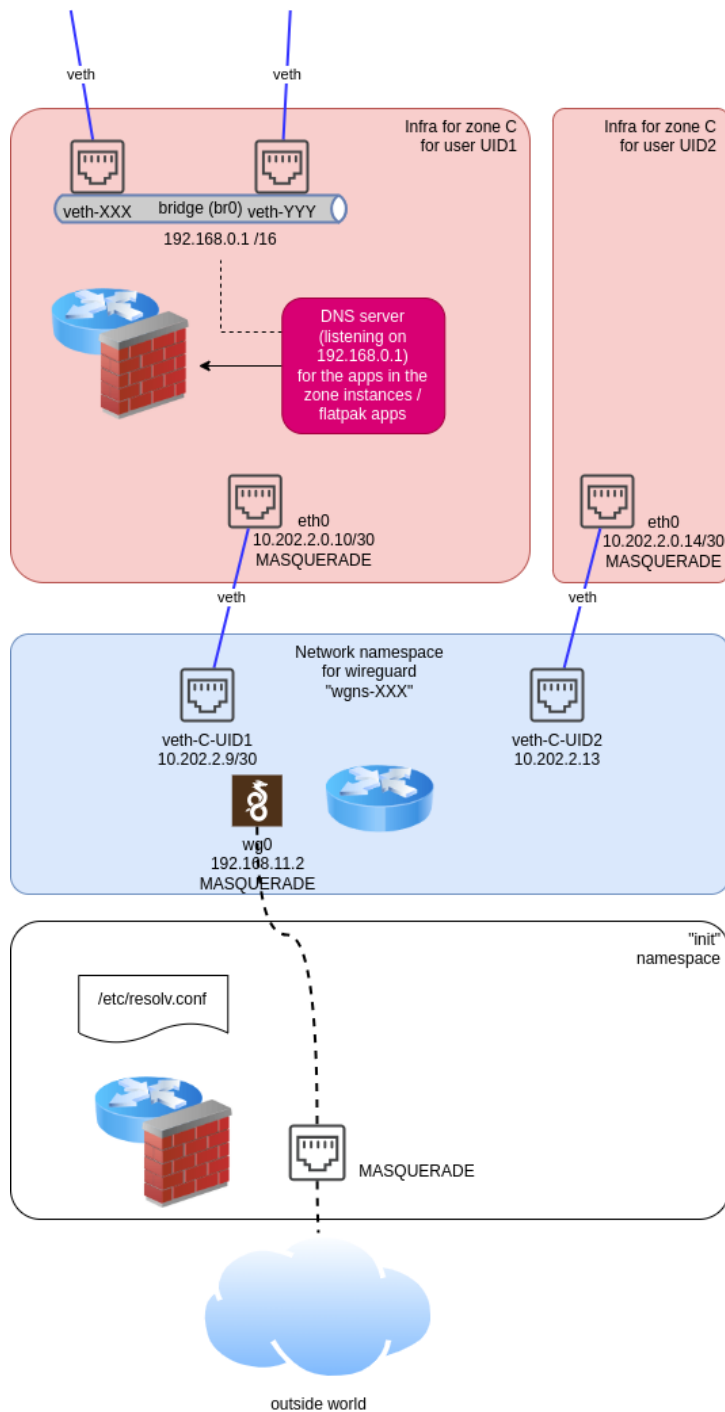
Note: all the traffic shapers monitor and adapt their behaviour based on the DNS resolution’s configuration of the system (which is in turn controlled by the end user who connects to various networks).

2.5.1 WireGuard VPN

To securely give access to resources within an organization, it is possible to configure some zones to route all their traffic through WireGuard network interfaces (multiple zones can route their own network traffic through one or more WireGuard network interfaces).

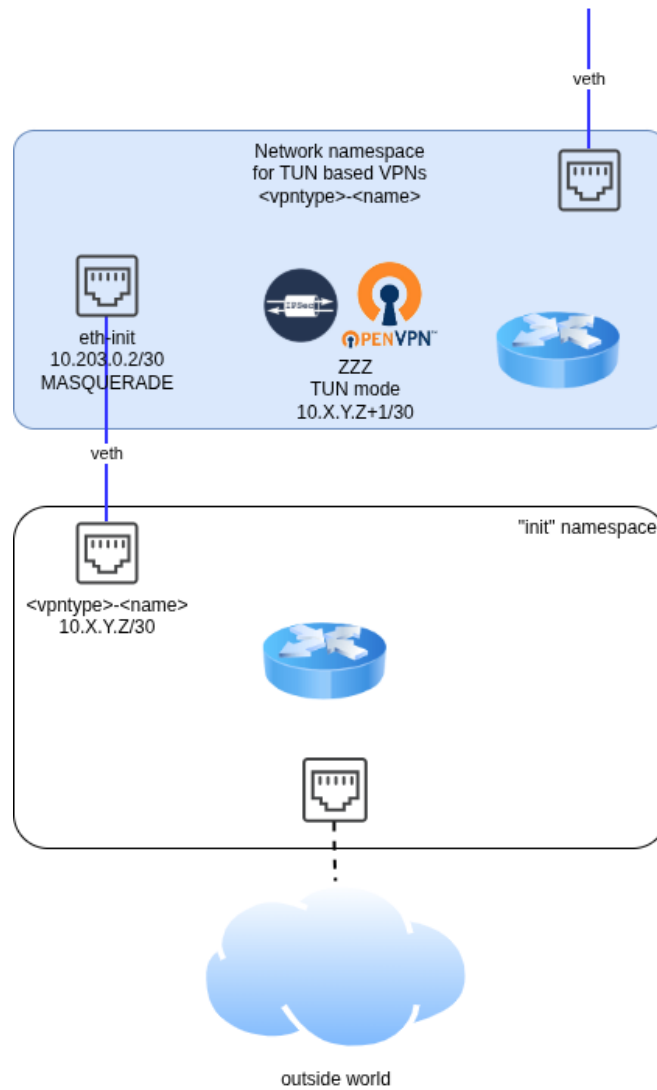
When configured as such, PADSI sets up the WireGuard network interfaces (using the same configuration as expected by the **wg-quick** tool) sets up network namespaces to route the traffic of all the zones’ network infrastructure through the associated WireGuard network interfaces, like in the following situation:

- a WireGuard network interface has been set up with the 192.168.11.2 IP address;
- a user whose UID is UID1 has started some programs in a zone (C), and another user whose UID is UID2 also has started some programs in the same zone, as shown by the two “network bubbles” at the top (refer to section 2.3.3 for more information);
- the bottom part is a network namespace which “imported” the WireGuard network interface (it’s thus not anymore in the “init” network namespace where it probably was initially created) and which acts as a router between the traffic coming from the zones.



2.5.2 Other VPN types

This section describes how routing through VPN which set up TUN interfaces work (at the moment, PADS only supports OpenVPN in "TUN mode"). The architecture is as follows (with the zone infrastructure not represented except for the "veth" interface at the top):



what differs from the WireGuard is the need for a dedicated pair of “veth” network interfaces to connect the VPN’s network namespace with the “init” network namespace.

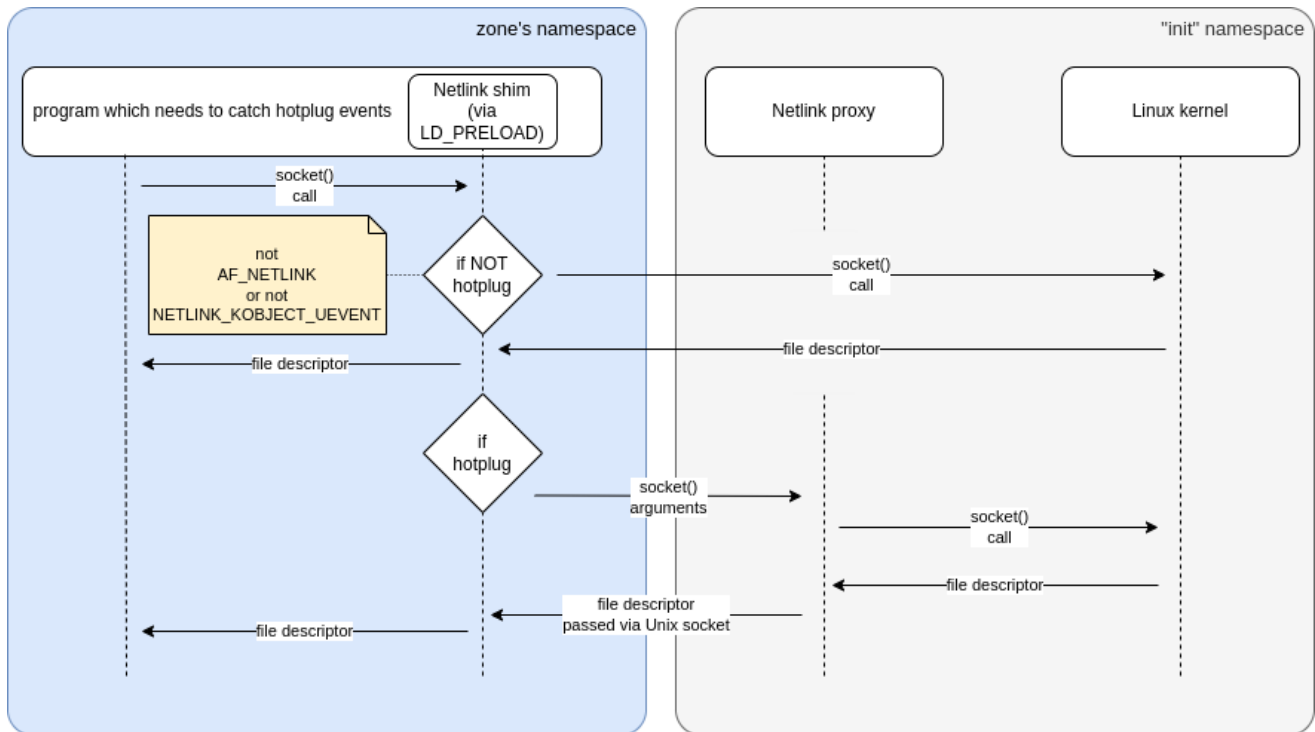
Note: if the VPN is not set up, then no traffic is actually allowed, implementing a *de-facto* VPN kill switch.

2.6 Hotplug events

Many programs need to be notified of hardware hotplug events: Web browsers for FIDO2 authenticators, the VM viewer to let the user “pass” the plugged devices to the VM, etc. Several methods can be used to be notified, the most common for programs is to open a **netlink** socket with the `NETLINK_KOBJECT_UEVENT` protocol.

Netlink sockets are bound to the network namespace, which means that a program running in a non “init” network namespace will not receive any event from the Linux kernel. To circumvent this limitation, when needed, the following setup is instantiated:

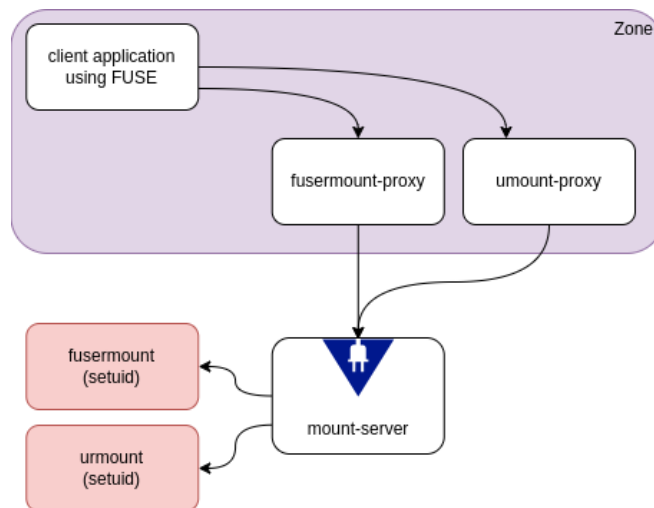
- a shared “shim” library is preloaded to intercept any call to the `socket()` system call. If the call is for a netlink socket with the hotplug events protocol, then it asks a “host helper” process to actually open the socket;
- a “netlink proxy” program (running in the “init” network namespace) serves requests from the “shim” via a Unix socket, opens the netlink socket and pass it back to the “shim” process.



2.7 Mounting filesystems (using FUSE)

If allowed by the zone configuration, normal users are allowed to mount filesystems using the FUSE technology (this is used by XDG portals or applications like Borg backup).

Applications using FUSE rely *in-fine* on the `/usr/bin/fusermount` program which is setuid to obtain the necessary privileges and the `/dev/fuse` device file. For security purposes, the `bwrap` tool drops the capability to ever elevate privileges again, which render the usage of setuid programs useless, which is the reason PADSi sets up a dedicated infrastructure for FUSE mounting (and unmounting) filesystems:



From a technical point of view:

- unmounting FUSE mounted filesystems similarly calls the `umount` program which is also setuid;
- PADSi sets up a “mount-server” (per-zone and run as the user) in charge of executing the privileged `fusermount` and `umount` programs (actual privileges are thus handled by the Linux OS’s configuration) which listens on a Unix socket⁹;

⁹Using a Unix socket is required to pass file descriptors between the FUSE clients and the actual `fusermount` programs.

- the `/usr/bin/fusermount` and `/usr/bin/umount` files are “masked” by PADSIs associated `fusermount` and `umount` proxies to relay calls to the “mount-server” process, via the `/bubble/run/padsi-fuse.sock` Unix socket;
- the unmount operations can be performed by the user, but also by the PADSIs user service when the user logs out.

2.7.1 Limitations

As the actual mount operation is performed by the “mount-server”, the mount point must be visible to it, which limits to `/bubble/run` and its sub-directories or the user’s home directory. Other restrictions may also apply depending on the type of “FUSE engine” used.

2.8 Desktop integration

2.8.1 Desktop file entries

Like for all [Freedesktop.org](https://freedesktop.org) compliant environments, GNOME relies on [desktop entry](#) files (with the `.desktop` extension) to describe and manage user facing programs (the ones which actually display windows in the user’s desktop). The `monitor-desktop-entries` program watches for any changes made to installed desktop entry files and, depending on the configuration, creates some other desktop entry files to:

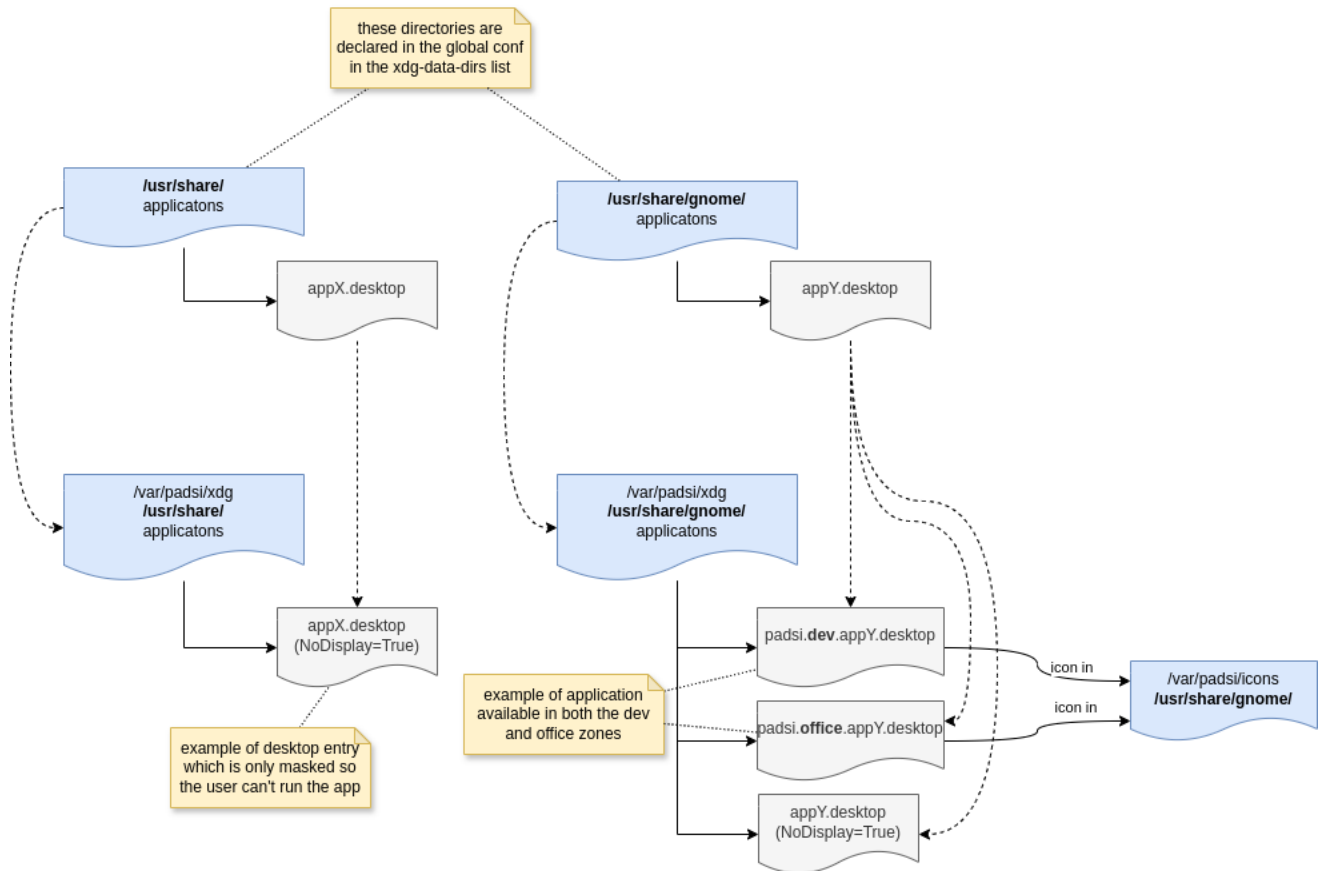
- prevent the user from running a program in the “init” namespaces¹⁰ (for a pure UX standpoint, security is not affected by any program the user may run in the “init” namespaces); or
- allow the user to launch a program in one or several zones (an associated outlined icon may be created in the process).

PADSI takes into account desktop entry files’ changes coming from either:

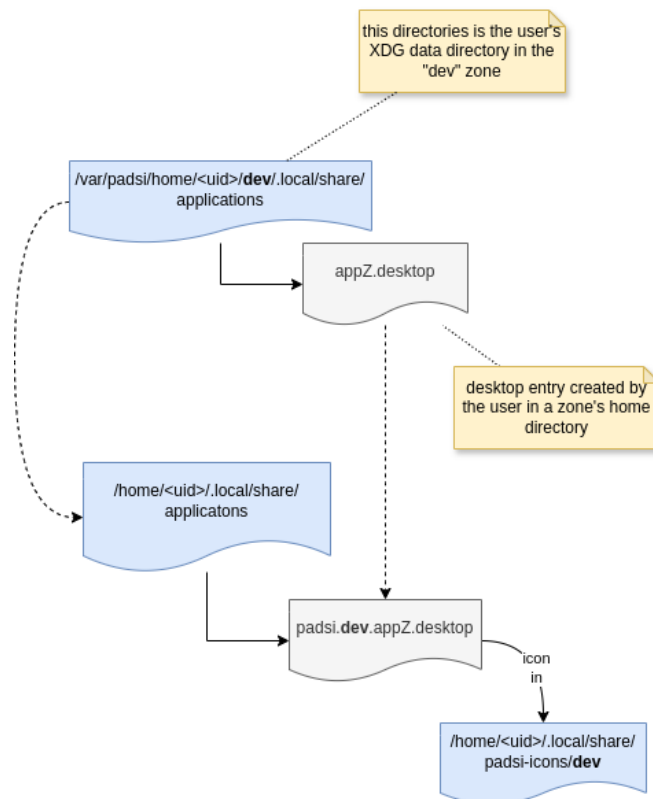
- a package being installed or removed (system wide);
- a user manually creating or removing a desktop entry file in one of the zone’s dedicated directory (`$HOME/.local/share/applications`).

The following diagram illustrates the situation in the system wide case with desktop entry files located in two different XDG data directories (`/usr/share` and `/user/share/gnome` here) containing each one desktop entry file:

¹⁰This is done by setting the `NoDisplay` attribute to `True` in a desktop entry file “masking” the one already existing.



And the per user situation looks like with a single desktop entry file in the “dev” zone:



The system wide monitoring is performed by an instance of the `monitor-desktop-entries` program started

by the PADSId's systemd's service while the per user monitoring is performed by one instance of the same program but running as the user, started by the **user-service** program, refer to §2.1 for more details.

Notes:

- PADSId does not modify any system or user defined desktop entry;
- PADSId declares its own directories (under `/var/padsi/xdg/` by default) before the system ones so the desktop entry files created have a “higher priority” than the ones installed. This is the purpose of the `/etc/profile.d/zzz_padsi.sh` file.

2.8.2 Outlined windows

The colored outline of all windows for programs running in a zone is actually applied by a GNOME shell extension (similar extensions could probably be developed for other desktop environments). Specifically, the extension matches the mount and network namespaces of the zone to the colors defined for that zone and display the outline. The user can disable the extension, but he will then not be able to visually know which window correspond to a program executed in which zone (all the other isolation mechanisms are not impacted).

Specifically, when PADSId starts a new zone to execute a program for the benefit of a user, it employs the following steps:

1. create a “bubble” for the zone, which automatically creates new namespaces;
2. write the actual new namespaces' references to the `/run/padsi/zone-infos/colors.json`¹¹ file for the GNOME Shell extension to (re)load;
3. start programs which will present windows to the end user.

To determine the color of the outline to apply to a given window, the PADSId GNOME shell extension then:

1. gets the PID of the window for which the function is called;
2. gets the mount and network namespaces of that PID;
3. uses the contents of the `/run/padsi/zone-infos/colors.json` file to get the color to apply.

2.8.3 Outlined icons

When an application is started, Mutter (the GNOME Wayland compositor) determines the APP ID of the application¹² which is then used by the GNOME shell to assign a desktop entry which name is `<APP ID>.desktop`.

Pending any modification of Mutter to better integrate PADSId's features, Mutter is actually “patched” while running by hooking the functions which handle APP ID retrieval using `LD_PRELOAD`. When one of the hooked function is called, PADSId:

1. gets the PID of the window for which the function is called;
2. gets the mount and network namespaces of that PID;
3. use the contents of the `/run/padsi/zones-infos/prefixes.txt`¹³ file to determine the zone's prefix (in the `padsi.<zone name>` format);
4. concatenates the real APP ID (as obtained using the real and unhooked Mutter's functions) with that prefix and keeps a cache of it;
5. returns this “adapted” APP ID.

¹¹Generated by the PADSId system service.

¹²Using some quite complex heuristics relying on the APP ID specified by the application, if any, or the APP ID specified by Flatpak (via the `.flatpak-info` file located at the root of the sandbox in which the application is started) or by Snap (via the `/proc/<pid>/attr/current` information).

¹³Also generated by the PADSId system service when it generates the `/run/padsi/zone-infos/colors.json` file.

2.8.4 Clipboard sharing restrictions

Sharing the content of the clipboard between applications in different zones can lead to data exfiltration risks where an application in a zone can request any content copied by the user from another application in another zone (and copy it to disk or exfiltrate it via the network, think for example about passwords copied from a password manager).

In order to address this issue, PADS1 allows to define clipboard sharing restrictions between zones.

When enabled, and as necessary, any program in any zone communicates with the Wayland compositor via a dedicated **per-zone Wayland proxy** which filters copy/paste related messages according to the configuration. Specifically, the wayland proxy:

- sits between all the UI applications in the associated zone, filters all the messages, and deeply analyses the messages related to copy/paste;
- when a copy offer is sent by a Wayland client (in any zone), the wayland proxy adds the “`padsi/zone;z=<zone name>`” mime type to the list of mime types the client sends;
- when a copy offer is received by a wayland proxy, it retrieves the zone in which the client is from the “`padsi/zone;z=<zone name>`” mime type and allows or blocks the copy offer.

IMPORTANT NOTE: the clipboard sharing restrictions don’t apply to contents copied or pasted via the X11 mechanisms, so X11 should be disabled accordingly to avoid any policy bypass.

2.8.5 X11 support

Even though X11 has been superseded by Wayland, in some cases it might be necessary to run X11 applications. so, if a zone configuration requires it, access to the X11 server is provided from within the zone. Wayland offers a much better security model than X11 as clients can’t snoop on other client’s activity (e.g. can’t get its keystrokes) and can’t even enumerate connected clients.

IMPORTANT NOTE: any X11 application in a zone can access to the events of any X11 application in any other zone: this breaks the zone’s isolation features. As such, one should be very careful how X11 access is provided in zones.

2.8.6 Desktop features

Desktop features like screen sharing or notifications are provided via the D-Bus session bus which is not accessible by the zones where programs are executed. When those features are required, the zone instantiates a “D-Bus router” which acts as the D-Bus session bus and actually forwards the messages to and from the applications and the real D-Bus session bus based on the features requested.

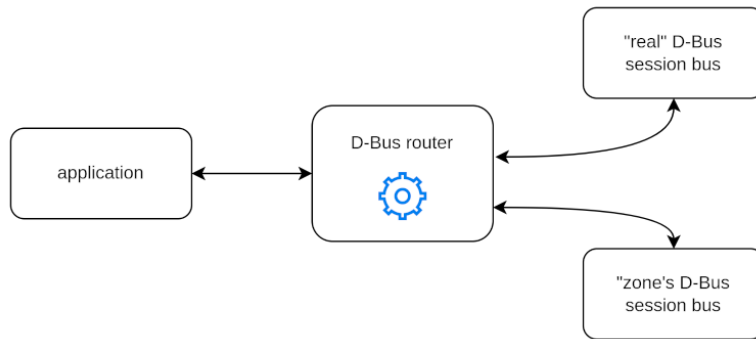
For screen sharing, refer to section [B](#).

2.8.7 D-Bus router

For better isolation, each zone has its own D-Bus bus and no access to the system D-Bus, which means application can’t access features provided by the real D-Bus session bus such as screen sharing or desktop notifications. A zone can nonetheless have some options if those features are required, and in this case, a “D-Bus router” is also instantiated.

The D-Bus router itself can communicate with both the zone’s D-Bus bus and the “real” desktop’s D-Bus session bus and presents itself as the D-Bus session bus. It then

- forwards messages from the application to the correct bus depending on the features required;
- forwards messages from the busses to the application.



To do so, the D-Bus router:

- uses matching rules on messages (based on the service, interface, or objects);
- analyses messages' contents to extract relevant information (such as connection IDs);
- “masquerade” destination addresses;
- guarantees that messages from a remote D-Bus server are always forwarded to the client D-Bus application in the same order in which they arrived;
-

Note:

- the application has no way to bypass the D-Bus router and directly connect to the “real” D-Bus session bus
- if the need arise, another D-Bus router could be spawned to create a proxy to the D-Bus system bus, or the `xdg-dbus-proxy` program could also be used for that.

2.9 Security devices integration

Zones can set to allow users to use smartcards and/or FIDO2 authenticators.

2.9.1 Smartcards

Smartcard access is provided to applications using the PCSC daemon (from the OpenSC project), zones are primarily given access via its Unix socket (`/run/pcscd/pcscd.comm`). Web browsers' policies are adapted to enable direct usage of the smartcard without any further configuration.

However:

- on Debian systems (maybe others), beyond the traditional Unix permissions on the socket file the PCSC daemon running in the system uses PolKit to determine if a process is allowed access, using the D-Bus session bus. Because zones set up their own D-Bus session bus, the PolKit permissions must be adapted as necessary via the creation of a `/etc/polkit-1/rules.d/00-padsi-user- $\langle$ UID $\rangle$ .rules` file (which is taken into account as soon as created and removed when the user logs out):

```

polkit.addRule(function(action, subject) {
    if ((action.id=="org.debian.pcsc-lite.access_pcsc" ||
        action.id=="org.debian.pcsc-lite.access_card") &&
        subject.user=="<USERNAME>") {
        return polkit.Result.YES;
    }
});

```

- direct access to devices in `/dev/bus/XXX` is not granted, as no programs requiring it has been identified.

Note that the full path to the PKCS#11 library file must be provided when this feature is configured (which means that different zones can use different PKCS#11 drivers, which may help solve PKCS#11 drivers incompatibilities).

Applications in zones will be granted access to smartcards if the “PKCS11” option is enabled for the zone (refer to section 5.1.2)

2.9.2 FIDO2 authenticators

FIDO2 authenticators can be used by applications in zones if the “FIDO2” option is enabled for the zone (refer to section 5.1.2).

2.9.3 GPG card

GPG card access is granted via the `scdaemon` process (part of GnuPG) via the PCSD daemon. To allow easy access to YubiKeys and other devices which also offer smartcard features, the `$HOME/.gnupg/scdaemon.conf` file in each zone is initially created with the following contents:

```
disable-ccid
pcsc-shared
```

This contents can then be changed by users if necessary.

Applications in zones will be granted access to GPG cards if the “GPG-CARD” option is enabled for the zone (refer to section 5.1.2)

2.10 Infrastructure for the Web access

This section describes the way PADSi tackles the following Web access needs:

- be able to use external **Web proxies** (popular in enterprises context to filter Web access) and yet also keep Web traffic filtering locally; and/or
- “**Web redirection**” to indicate that a Web page access is denied and propose to open it in the same browser but in a zone which allows the access for better UX.

If applications were allowed to directly ask an external Web proxy to connect to the target Web service, all the local network filtering to achieve network isolation could trivially be bypassed by a malicious actor (including for example to open SSH connections to an owned server).

To avoid that security issue, PADSi:

- prevents direct access to the actual external Web proxies to use;
- adds a local Web proxy¹⁴ to enforce the network access restrictions (applications can use the `http://proxy:3128` proxy setting when needed, e.g. in the `http_proxy` and `https_proxy` environment variables);
- sets up the correct `http_proxy` and the like environment variables and provides a WPAD web service¹⁵ for programs to “auto-configure” them.

These features (Web proxy, WPAD server and “Web redirection server”) are actually implemented using a single “**web-infra**” application, which configuration depends on the zone’s configuration. For the virtual machines, this proxy forwards requests to the associated zone’s proxy.

Notes:

- the WPAD server is enabled only when the Web proxy feature is enabled;
- the “web page blocked” HTML contents and associated resources (like images) are served when appropriate.

2.10.1 Behavior of the Web proxy

When Web redirection is disabled If the Web redirection feature is not enabled for the zone (which is the default), the behavior is as follows:

¹⁴Listening on port 3128.

¹⁵Via the `http://wpad/wpad.dat` URL.

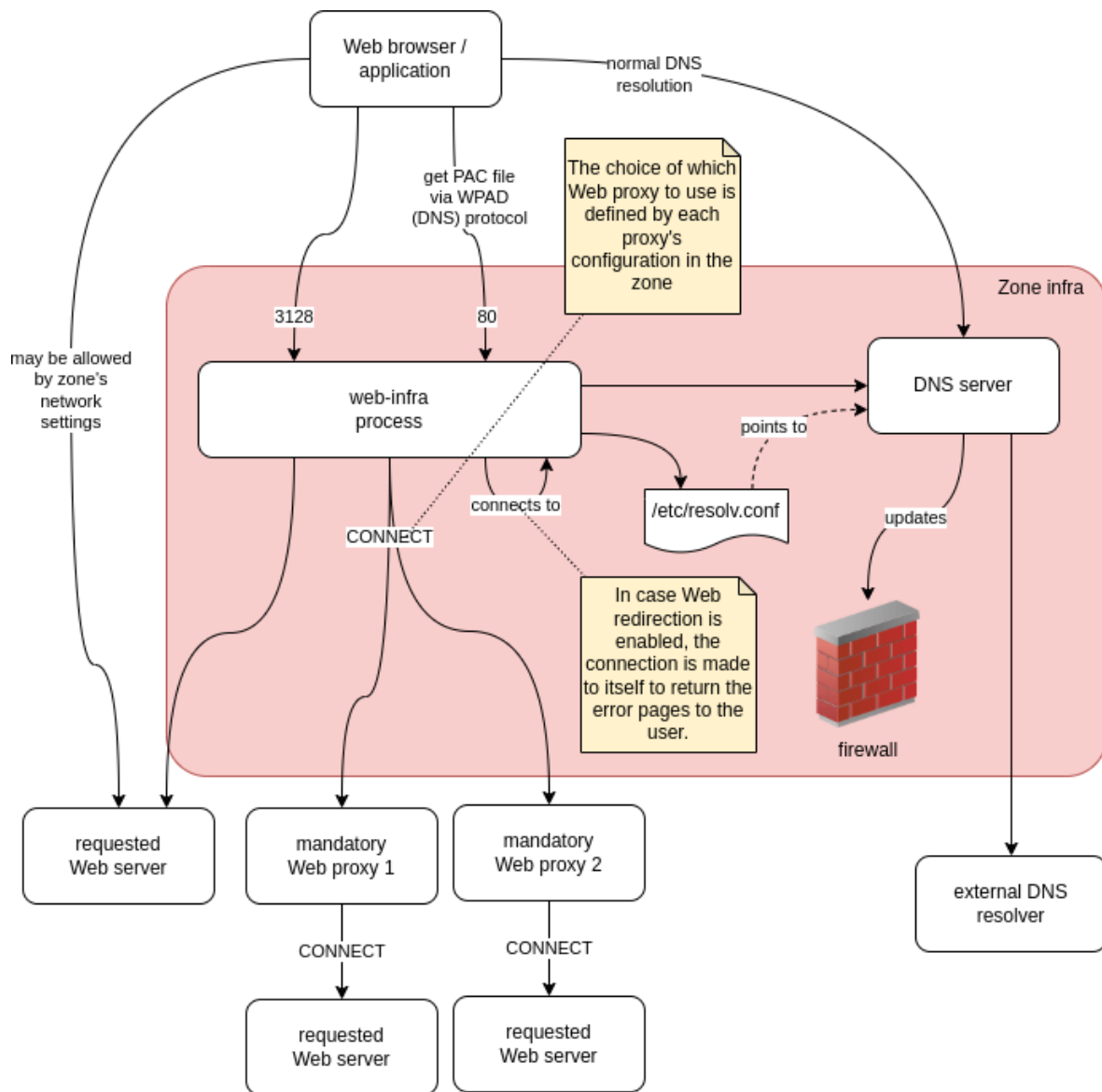
Protocol	Direct access	Proxied access	Request mode	Web page result
HTTP	*	*	direct	GET http://wpad/wpad.dat returns the WPAD file for the proxy's configuration
*	allowed	*	direct	fetches directly (filtered by the zone's infra.)
*	denied	*	direct	DNS resolution error or timeout (if IP address directly used) because of the firewall in the zone's infra.
*	*	allowed	via Web proxy	fetches by the proxy via the external proxy where the request is allowed
*	allowed	denied	via Web proxy	fetches by proxy itself (without relying on an external proxy)
HTTP	denied	denied	via Web proxy	proxy returns "web page blocked" HTML contents
HTTPS	denied	denied	via Web proxy	proxy returns a 403 error for the CONNECT

When Web redirection is enabled If the Web redirection feature is enabled for the zone, the behavior is as follows (grayed sections are unchanged compared to when Web redirection is not enabled):

Protocol	Direct access	Proxied access	Request mode	Web page result
HTTP	*	*	direct	GET http://wpad/wpad.dat returns the WPAD file for the proxy's configuration
*	allowed	*	direct	fetches directly (filtered by the zone's infra.)
HTTP	denied	*	direct	DNS resolution points to the Web redirection service (192.168.0.1) which returns "web page blocked" HTML contents
HTTPS	denied	*	direct	DNS resolution points to the Web redirection service (192.168.0.1) which generates a certificate for the requested site and returns "web page blocked" HTML contents
*	*	allowed	via Web proxy	fetches by the proxy via the external proxy where the request is allowed
*	allowed	denied	via Web proxy	fetches by proxy itself (without relying on an external proxy)
HTTP	denied	denied	via Web proxy	proxy returns "web page blocked" HTML contents
HTTPS	denied	denied	via Web proxy	proxy internally connect to the Web redirection service (192.168.0.1) which generates a certificate for the requested site and returns "web page blocked" HTML contents

2.10.2 Web proxy's architecture

The proxy's architecture is as follows:



Notes:

- the Web proxy also proxies Web traffic which is allowed directly by the zone's configuration to simplify applications' configuration
- the default policy is to block accesses;
- from a technical point of view, if all traffic is routed to Web proxies, DNS resolution can be disabled as it's not used.

2.10.3 Web redirection's architecture

Web redirection is a purely cosmetic feature which has no security value but is a UX bonus when a user tries to connect to a Web site (either directly or having clicked on a link) which is blocked in the zone where the Web browser used is executed.

Without this feature, the DNS resolution in such case will fail and the browser will display an error message. However, when this feature is enabled for that zone:

- the browser displays a "site blocked notice" with some informational description;
- depending on the settings and if the blocked can be opened using a Web browser in another zone, then a notification will be presented in the GNOME desktop along with an action button to actually open the Web

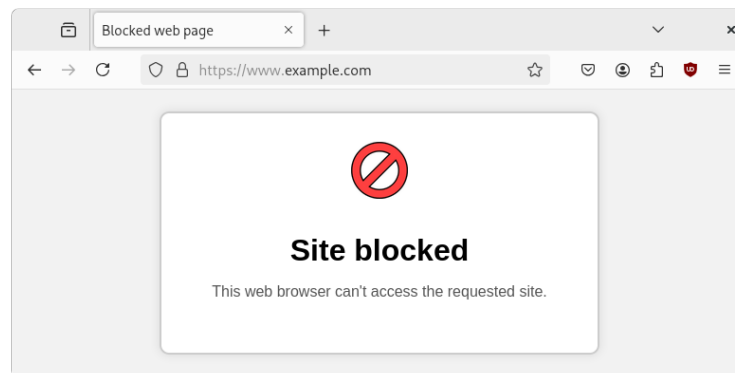
site in the other zone(s). The user then only has to click a button to open the requested Web page in the same browser but in a suitable zone.

Limitations Web redirection has several limitations:

- only HTTP GET requests are taken into account;
- the feature is enabled only if:
 - the path and query of the requested resource is less than 100 bytes (to avoid potential data exfiltration);
 - the connection port is among a common list (80, 443, 8080, 8443);
 - the requested page has a “referrer” header (to filter out non user initiated requests);
 - the requested web site is not among the ADBlocked list.

Architecture Displaying a “site blocked notice” for any Web site, and without any Web browser security warning for those accessed via HTTPS requires:

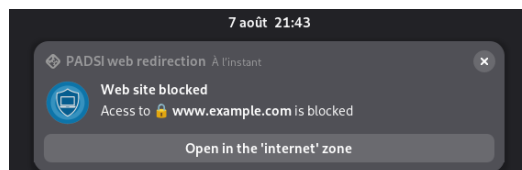
- a “**redirection Web server**” serving the “**site blocked notice**”:



- that the DNS server for the zone resolve any non allowed request to that fallback server;
- that the web redirection server generates a certificate adapted to the requested host using a dedicated certification authority (named “Web access denied CA”);
- that this CA be trusted by the Web browser in the zone.

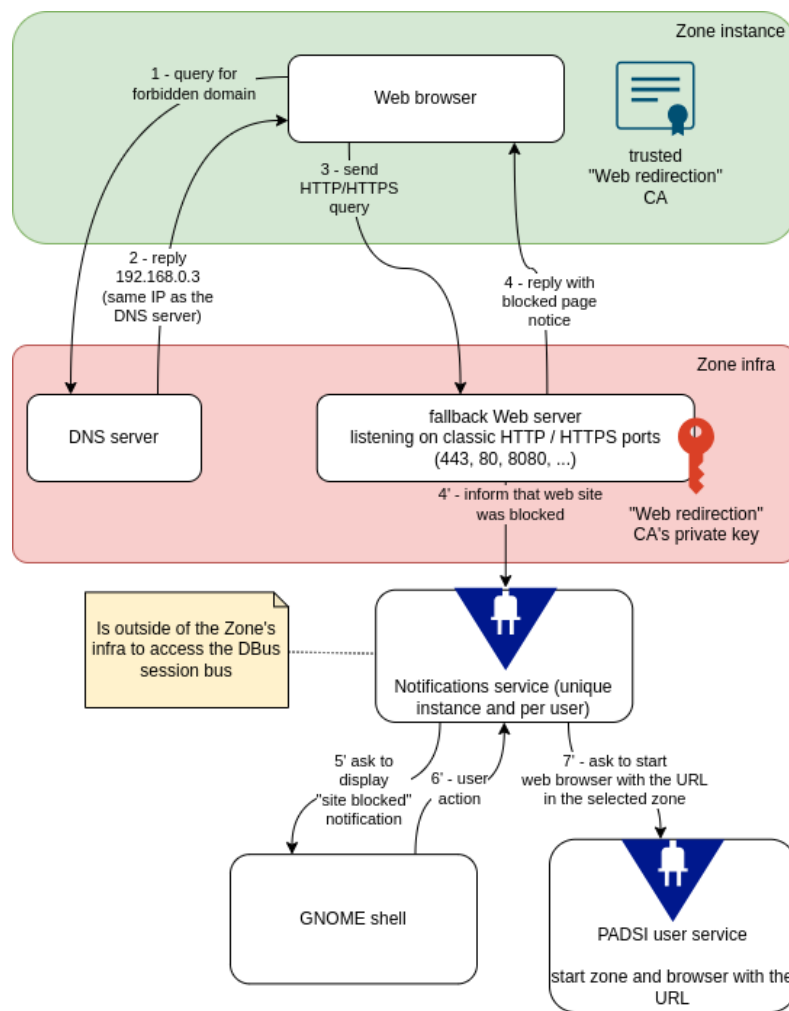
Furthermore, actually notifying that the Web site can be reached using the same Web browser but from another zone requires a **notification service** to:

- ask the GNOME desktop to display a notification (via a DBus message):



- ask the PADSI user service (refer to section 2.1) to actually start the Web browser in the selected zone and open the blocked URL.

The architecture is the following one:



Notes:

- the "Web access denied CA":
 - is short lived and per zone: its private key and self signed root certificate are created when the zone's infrastructure is started and removed when the zone is destroyed;
 - is only declared as trusted in the Web browsers (via specific policies) but not in the zone's "system" files;
 - is generated when the zone is started, stored as a PKCS#12 file (using a random password), passed to the "web-infra" application which uses it. Once this setup phase is done, the password is lost making sure no malicious actor can use that CA for nefarious purposes.
- this is not a "middle box" as the web redirection server will only ever display a notice as a static HTML Web page and not itself query the actual Web server before inspecting and eventually giving it to the Web browser: there is no SSL/TLS "decryption" and inspection;
- this feature is implemented in a way such that a malicious threat actor:
 - can't bypass network restrictions and communicate in any way with his command and control services without any user interaction;
 - can't get any information about PADS's actual configuration.
- the implementation of this feature using Web browser extensions did not work out as expected because Google's manifest V3 which Chrome uses does not allow for that kind of extension to built.

2.11 Flatpak applications

Flatpak applications are executed as any other application from a running zone, creating their own sandbox using the BubbleWrap tool. There is no specific treatment for Flatpak applications.

2.12 Virtual machines

PADSI allows end users to run virtual machines in the context of a zone: each VM is run as a [QEMU](#) process and not via libvirt or other kind of VM management. End users are not however by default allowed to install themselves virtual machines, this role is dedicated to system administrators which:

1. install a VM from traditional boot medias provided by the selected OS's manufacturer;
2. import that newly created VM (via its disk image and UEFI firmware files) to create a “**base**” VM (a “base” VM can also be saved on a PADSI system and loaded into another one);
3. update the VM when deemed necessary (for instance to apply security patches or modify the VM's configuration);
4. make that updated “base” VM available to users.

Virtual machines are defined globally and made available on a per zone basis.

2.12.1 Forcing the usage of virtual machines

To further strengthen security, system administrators may force the usage of virtual machines and disable the possibility to execute applications. Such extra restriction is enabled via the **VM-ONLY** zone option.

When this mode is configured for a zone:

- a default VM has to be specified, which of course must be available in the zone;
- users can't directly run applications in the zone (technically, no zone bubble is created).

This mode allows one to build a “multi environment workstation” as defined by the French ANSSI (refer to the [Sécurisation du poste de travail multi-environnements \(non classifiés\)](#) guide) where each “user domain” can be implemented via a VM (and “support domains” remain isolated via sandboxing mechanisms). Note that PADSI only covers some of the requirements for a secure workstation, specifically OS hardening must be implemented by the administrators.

2.12.2 About VM versions

As introduced in section [1.5.1](#), PADSI imposes a lifecycle over the VM versions associated to a VM definition; this process heavily relies on [QEMU snapshots](#). As a reminder, a VM version is a set of coherent files used to run a specific version of a VM:

- a VM disk image file (with the **.img** extension);
- a VM's UEFI firmware **OVMF** file (with the **.vars** extension); and
- a VM's information file (with the **.infos** extension) which, among other things, keeps track of the life of that VM version.

A given VM version can be in several **usage states**:

- **CREATED**: the associated files have just been created and have never been used;
- **STOPPED**: the VM is currently not running;
- **RUNNING**: the VM is currently running;
- **DISCARDED**: the VM was marked as “to be destroyed” and the associated files will probably soon be removed.

Also, VM versions are associated to **lifecycle types**:

- **BASE**: a VM version which is actually never directly used to run a VM but serves as a reference, making it “read-only”. This is what is deployed by system administrators. A base version is not associated to any zone or any end user.
- **USER**: a VM version which has been customized for a specific user (user name, ID, etc.). A user VM version is always a snapshot of a BASE VM version and also must be considered as “read-only” as, once created, it is never directly used to run a VM. A user version is always associated to a zone (i.e. the customization process will occur once per zone and per base VM version) and an end user.
- **SNAPSHOT**: a VM version which is actually run by the end user. A snapshot VM version is always a snapshot of a user VM version and can be given a nickname as the end user is allowed to run several snapshot VMs at the same time. A snapshot version is also always associated to a zone and an end user.

To make sure end users can run VMs while at the same time an administrator can update the base VM, PADSI introduces the notion of a “**staging area**” which is where installation and updates occur before being “**published**” and declared usable by the end user. The staging area is not associated to any zone, but is associated to an end user.

The following sections detail the relations between the different VM versions during different usage scenarios.

2.12.3 Privileges management

For each virtual machine configuration, the system administrator has to define the following privileges regarding the VM’s usage (which differs with the actual privileges the user may have inside the guest OS of the VM):

- the **privilege to install** the VM allows specified users to boot the VM from an ISO image and actually install the VM creating a VM version in the staged area. To actually be used, that new VM version will have to be published;
- the **privilege to update** the VM allows specified users to create a new VM version in the staged area, and run the VM based on it to make changes to the guest OS such as applying patches, install or remove software, or change its configuration. This privilege also allows the user to publish the VM version to make it available to users who can actually run the VM;
- the **privilege to run** the VM allows specified users to create one or more snapshot VM versions of a published VM version and run them.

2.12.4 QEMU snapshots

To avoid duplicating VM images files (the files which represents the virtual HDD or the VM and can reach several Gb), PADSI uses QEMU’s snapshots feature which allow a VM image file to store only the changes from a “backend” VM image file.

This means VM versions usually have a “dependency” or a cascade of dependencies over other VM versions. As an example, for a Windows VM:

- the BASE (“real”) VM image file size will range from 8Gb to 20Gb;
- the USER VM image file size will be around 200Mb.

And for a Linux VM:

- the BASE VM image file size will range from 3b to 10Gb;
- the USER VM image size will be around 15Mb.

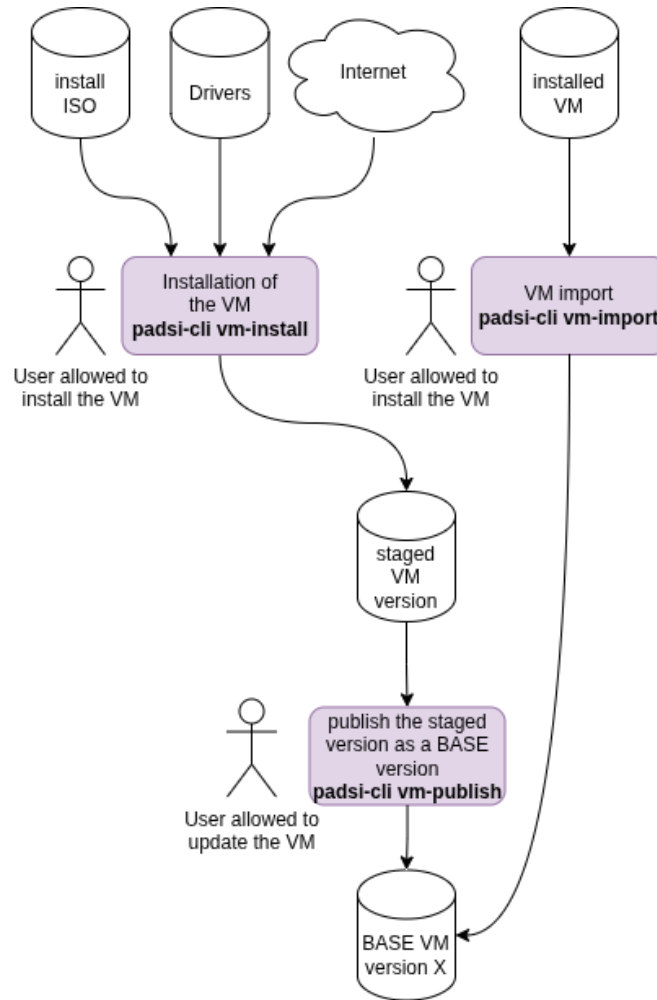
2.12.5 VM version installation / import

VM versions can be installed from an OS’s manufacturer ISO image (like any virtual machine can be installed) or imported from an existing VM disk image (a VM version also be loaded from an archive of an already existing VM version possibly on another PADSI system, refer to section [2.12.8](#)).

The installation workflow is as follows:

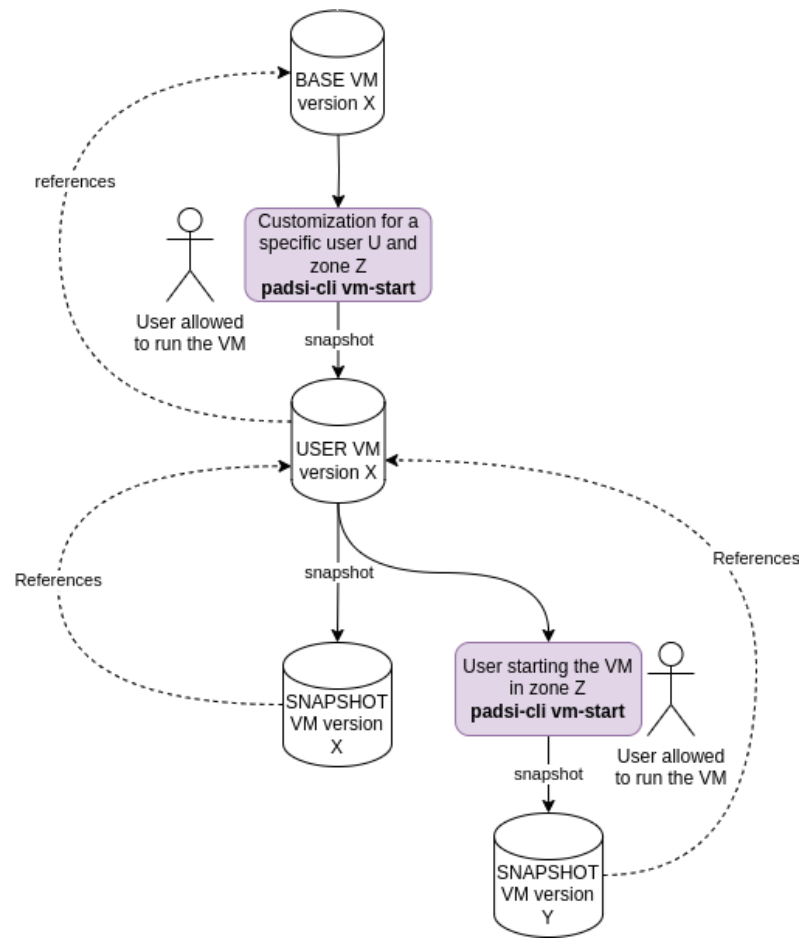
1. The installation process creates a staged VM version (replacing any previous one unless that VM version was not marked as discarded);
2. If deemed Ok by the administrator, the VM version is then published to create a base VM version.

The import workflow is more direct as the import process directly creates a base VM version.



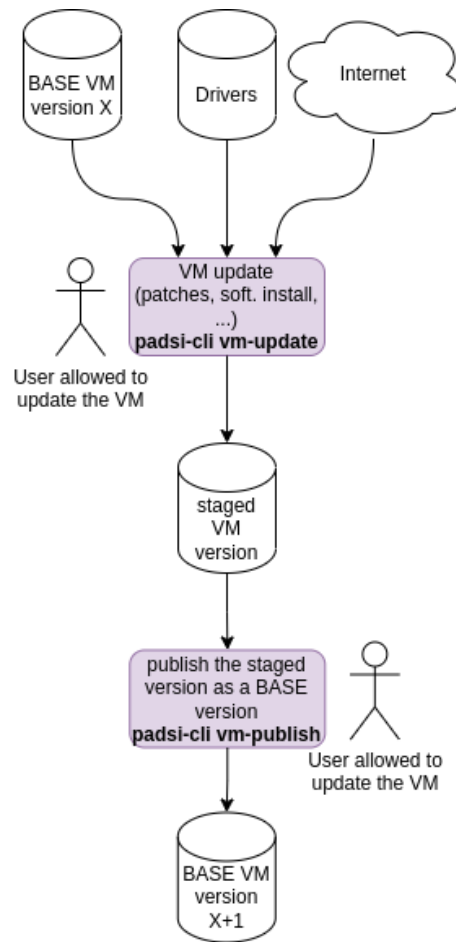
2.12.6 VM version run

When a user wants to run a new VM, PADS will automatically create a user VM version (based on the last base VM version available) if there is not yet one, and only then create a snapshot VM version to actually run the VM on. If the user VM version for that user already exists, then a snapshot VM version will simply be created from it.



2.12.7 VM version update

To update an existing VM version, PADS simply creates a new staged VM version based on the last available base VM version and run the VM with this VM version. Once the administrator is satisfied with the updated VM version, he can publish it as a new base VM version:



Note that the new BASE VM version ends up being a snapshot of the previous base VM version.

2.12.8 VM version save and load

“**Loading**” a VM refers to preparing to use a VM version which has been “**saved**” by an instance of PADS, as opposed to “**importing**” a VM’s disk image which may have been created by any tool. The difference is that when loading a VM version, the VM version’s other files are also taken into account, and that disk images’ dependencies are also handled. Saving a VM version to load it later on a possibly different PADS instance generates a TAR archive.

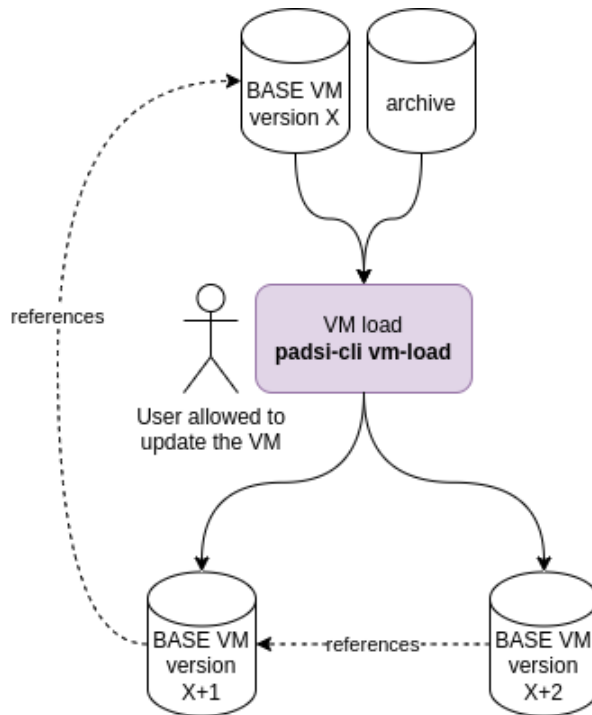
Each archive has the following contents:

- the `manifest.json` manifest file containing various information about the contents of the archive;
- one or more `vm-version-<index>` directories which each contain all the files making a specific VM version (`base.img`, `base.vars` and `base.infos`), where:
 - `index=0` for the VM version being saved
 - `index=1` for the VM version on which the VM version being saved depend (its image file being a snapshot of this VM version);
 - ...
 - `index=X+1` for the VM version on which the VM version of `index=X` depends.

If the PADS instance where the archive will be loaded already has a specific VM version, then the archive may contain only a reference of that VM version (as the SHA256 hash of the VM image file) and not include any subsequent dependencies. In that case, the hash of the VM version is included in the manifest in the `dependency-hash` and `dependency-size` attributes.

For example, in the following diagram:

- the PADSİ instance where the archive is loaded already contains a VM version X;
- the archive will load a VM version X+1 based on the VM version X;
- it will also load the VM version X+2 based on the VM version X+1.

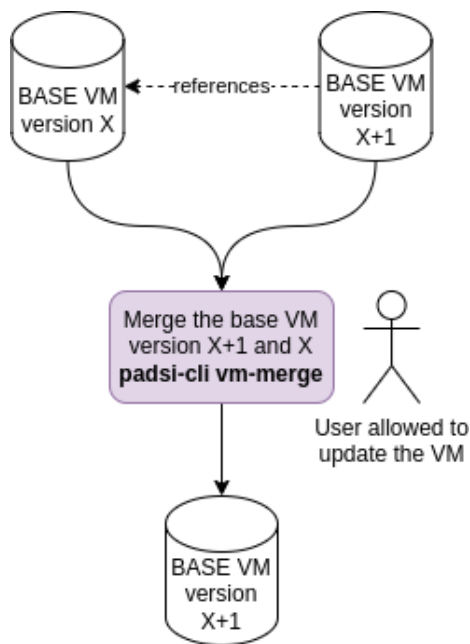


Note that PADSİ will try to “reuse” immutable VM disk images as much as possible to avoid clutter (before each VM disk image is loaded, its file size and possibly hash are compared to what’s already present in the system). This means that actual VM version numbers may differ between the system where the VM versions were saved and the system where they are loaded after an archive has been loaded.

2.12.9 VM versions merge and cleanups

Because of the way PADSİ uses snapshots during the lifetime of VM versions, there can be quite long string of actual QEMU image files linked together associated to base VM versions. At some point though, the snapshot VM versions will all be snapshots of user VM versions based on the latest base VM version, with some user VM versions having become useless. To avoid clutter, PADSİ then proposes to:

- remove discarded snapshot VM versions;
- remove unused (non referenced by any snapshot VM version) user VM versions;
- merge base VM versions with the base VM version of which it is a snapshot. Note that this operation is only possible when no other VM version references the “lower” VM version to merge into.



2.12.10 Relation to the zone' network infrastructure

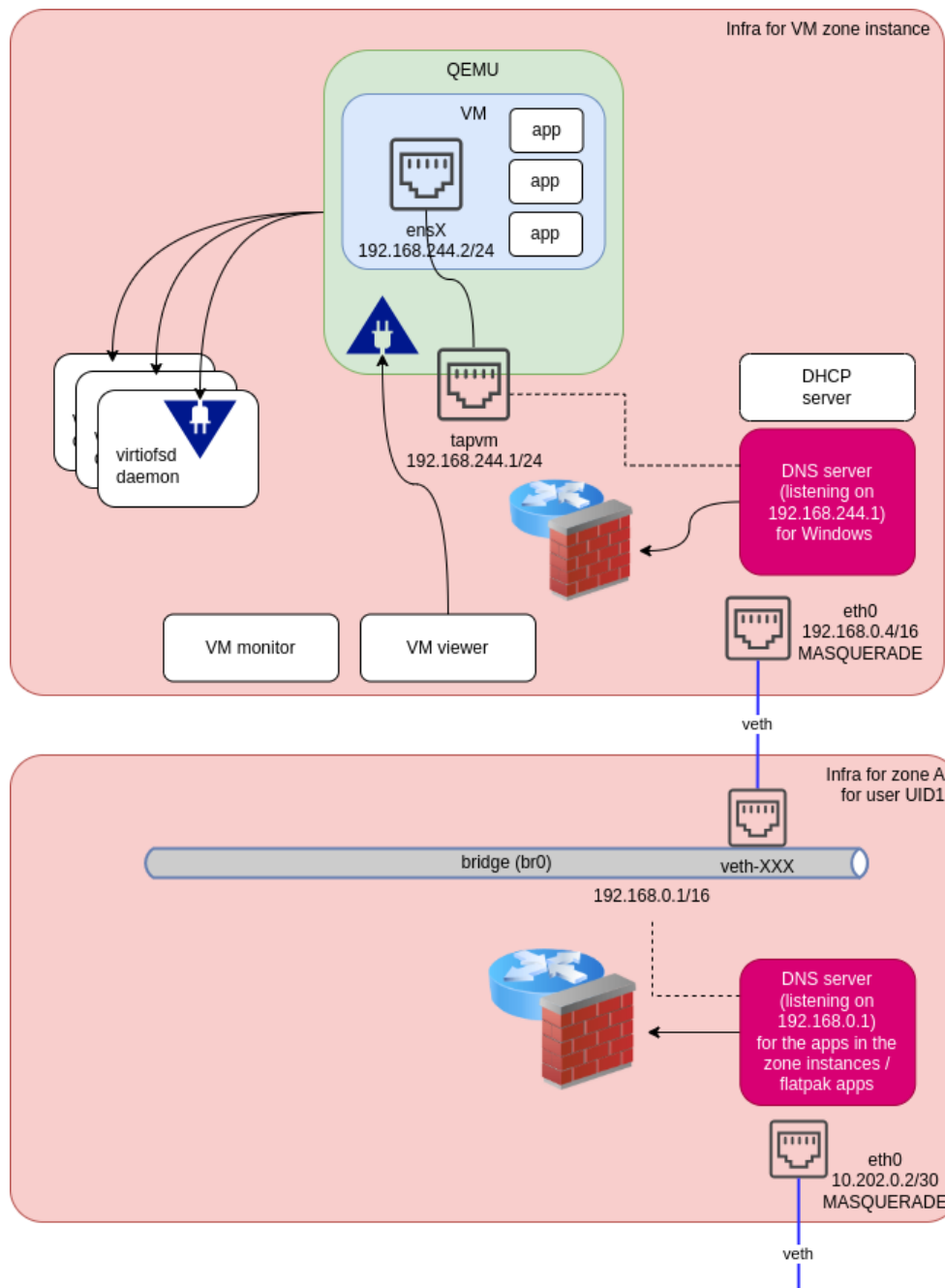
The QEMU process making the VM “run” is not directly executed in the zone’s “bubble” along with other programs: it runs in its own “bubble” along with the ancillary services to support the VM such as:

- a DNS service and the associated firewall which functions as for the zone’s infrastructure (refer to §2.3.4) which allows the administrators to restrict the communications capabilities of the VM even further than for the associated zone;
- a DHCP service to configure the IP address, gateway and DNS resolver of the VM’s;
- one or more `virtiofsd` services which provide each the capability to access files from the zone (more section 2.12.16);
- a VM viewer which is the actual UI through which the end user can use the VM;
- a VM monitor which is in charge of detecting when the QEMU or VM viewer processes terminate to end the VM’s “session” properly.

From a network standpoint, the VM is run with a single Ethernet network interface via which it can communicate with the programs and other VM in the same zone.

The following diagram illustrates the situation when a VM is running, note that:

- the VM’s IP address always being `192.168.244.2/24` while the corresponding IP address of the associated `tapvm` network interface in the “bubble” is `192.168.244.1/24`;
- the “bubble”’s own interface to communicate with the zone’s network infrastructure is in the same sub network as the other “bubbles” in the same zone (always in the `192.168.0.0/16` address range);
- the VM viewer uses a Unix network socket to communicate via the SPICE protocol with the VM (no other program can “grab” the VM’s display because the Unix socket file is visible only within the “bubble”).



If the zone uses Web proxies (refer to section 2.10), then PADSII also runs a dedicated Web proxy for the VM, which

- blocks requests to Web services which are denied;
- forwards all other requests to the web proxy running the zone's network infrastructure.

Notes:

- the WPAD feature is available;
- the Web redirection feature, if enabled in the zone is not enabled for the VM as it makes little sense to do so.

2.12.11 Relation to zones

The PADSIs administrator defines which VM can be run in the context of each zone, and by with end user. This is fine but installing or updating a VM requires different network accesses, specifically resources available only via Internet and these resources may not be available from the zone(s) where the VM can be run.

To cope with this situation, VM installation and update are not done in the context of a predefined zone but (unless otherwise specified), PADSIs will determine which zone is best suited to be used and run the VM in that zone but without any zone's files being accessible (in other words: benefit from the zone's network infrastructure, but don't let the VM use the zone's files).

2.12.12 VM viewer

While a virtual machine is running, the associated VM viewer process is kept running, even if it does not display any window. When the VM terminates (either properly shut down or if the QEMU process is killed one way or another), PADSIs marks the VM as discarded and remove any associated resources. However, the VM viewer is not shown by default if VM has its "show-ui" attribute set to `False`, but it can still be shown afterwards (sending `USR1` or using the PADSIs tools). Specifically:

- When the VM is being installed or updated, the VM viewer is always show in console mode when the VM is being installed or updated.
- When the VM is customized for the user: the VM viewer, if shown, will only display the startup window while the VM is being customized for the user.
- When the VM is actually run: when the VM boots, the VM viewer will display a startup window to hide the VM's actual console to the user while still allowing him to show the console or discard the VM. If the user has not opted to show the console, it will be shown when the user session in the VM has been opened and the VM is thus completely usable.

2.12.13 SSH connections

Because SSH is such a powerful tool, when a zone in which VMs can be running is started, a new pair of fresh SSH keys is created with the aim to be used to SSH into all the VMs in that zone. Then, when a VM is actually started:

- the SSH private key is copied to the zone's `~/.ssh/padsi-vm-key` file;
- via the PADSIs agent (refer to section [2.12.15](#))
 - the zone's `~/.ssh/config` file is modified to create an entry for the VM;
 - the SSH's public key is copied to the `etc/padsi-vm-key.pub` file and ultimately appended to the `~/.ssh/authorized_keys` file in the user's HOME directory in the VM;
- the VM's SSH's public key is scanned and:
 - the public SSH key added to the zone's `~/.ssh/known_hosts` file (one entry per domain name for the VM in the zone, refer to section [2.3.4](#));
 - a Host entry is created in the zone's `~/.ssh/config` file

so that the user can simply run `ssh` to get a shell in the VM (of course if the VM offers an SSH service). Note that any information in those files which may have been added by the user is kept AS-IS.

2.12.14 USB redirection

USB devices redirection in the context of virtual machines is performed by the VM viewer using the Spice technology which mainly requires:

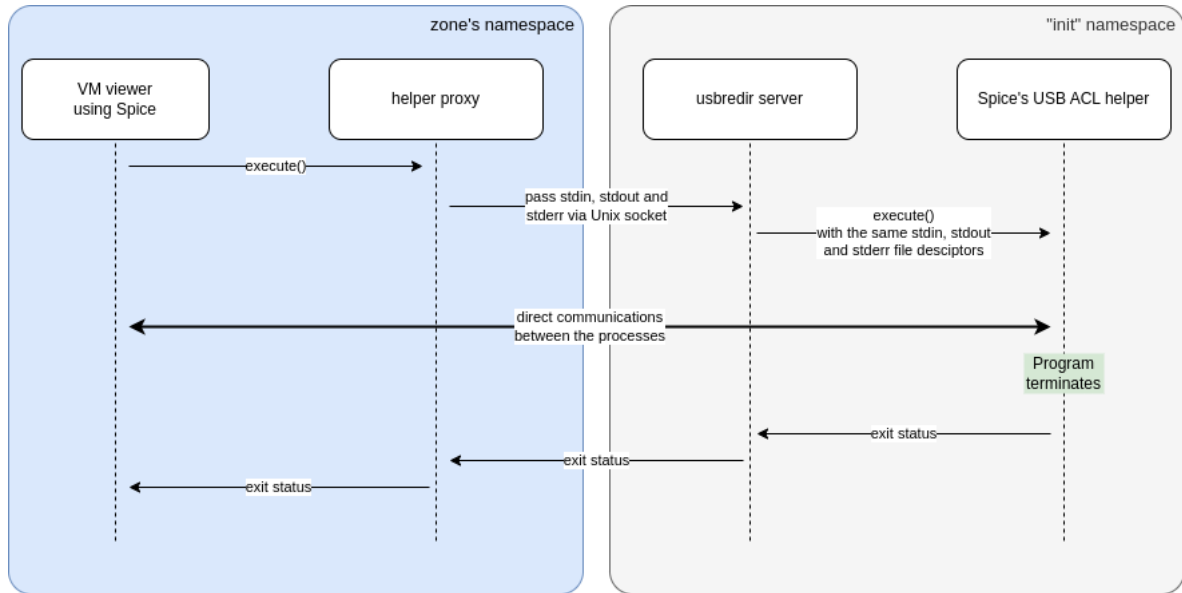
- to be informed of devices hotplug events and be able to access devices (refer to section [2.6](#)); and
- to be able to execute the `spice-client-glib-usb-acl-helper` helper program to change the ACL of devices files. This helper program is setuid to set the correct ACL, but the setuid possibility is lost when running within a BubbleWrap context.

Thus, this second point is performed via a dedicated component made of two parts:

- an “helper proxy”, which is executed by the Spice component of VM viewer thinking it actually is the `spice-client-glib-usb-acl-helper` program, and which behaves exactly like it from its point of view;
- the usb redirection server which is running in the “init” namespaces which is in charge of executing the actual `spice-client-glib-usb-acl-helper` program (which can elevate its privileges by being `setuid`).

All the while, once this setup is done, the Spice component of the VM viewer can directly communicate with the actual USB ACL helper as this process uses the same stdin, stdout and stderr file descriptors as the ones defined by the VM viewer.

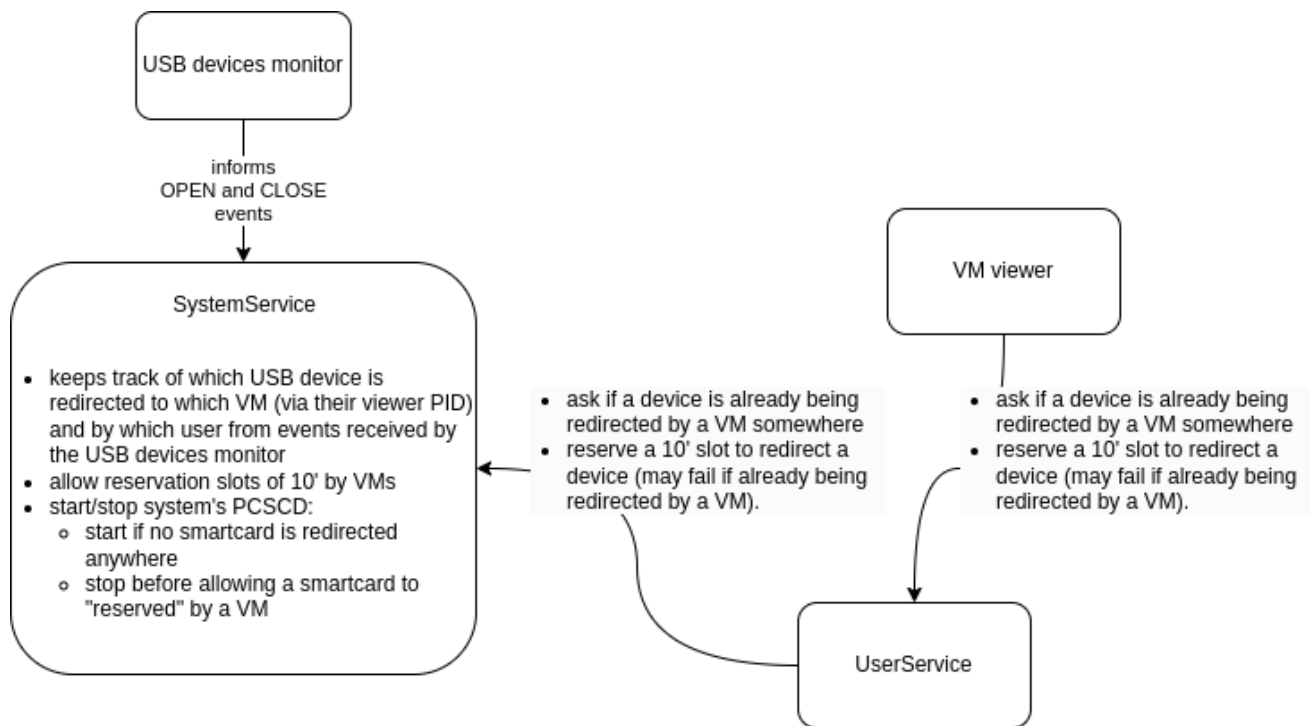
The following schema summarizes this setup:



Safe resources usage When a USB device is being used by a virtual machine, the host system and other virtual machines lose their ability to use that device. This is even more salient in the case of smartcards where the host’s PCSC daemon (refer to section 2.9.1) will prevent any virtual machine from acquiring the usage of the smartcard.

In order to tackle this problem, the PADS system service maintains a list of which USB device is used by which virtual machine (or by the host in case of smartcards) from information received by the **USB devices monitor**¹⁶, and allows the viewer of each virtual machine to query this status and “ask” before redirecting a USB device.

¹⁶Which technically uses an eBPF program as traditional inotify and fanotify mechanisms don’t work for devtmpfs.



Note: the PCSCD service is automatically started and stopped by PADS I when a VM wants to use a smartcard.

2.12.15 PADS I VM agent

The **PADS I VM agent** is a small component via which PADS I interacts with running virtual machines mainly to customize them for a specific end user, mount the shared directories as specified in the configuration, and run specific programs on boot. It is implemented as a service in both Linux and Windows guests systems.

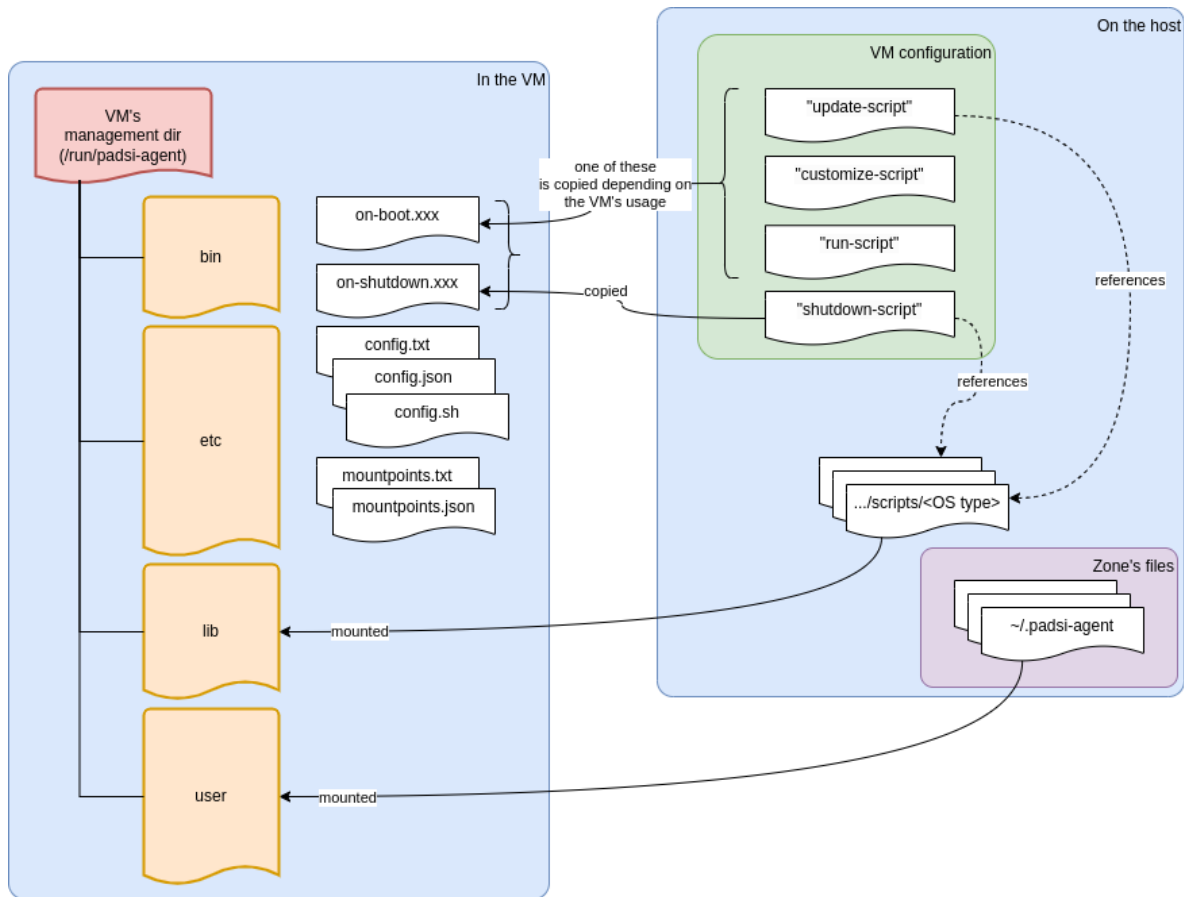
However, the PADS I agent also implements a REST API to:

- get the status of the system;
- execute programs;
- shutdown the system (used by the VM viewer).

Technically, the PADS I agent is given access to resources shared by the host (mounted in `/run/padsi-agent/` for a Linux guest OS and in the `Z:\` directory for a Windows guest OS):

- information about the context in which the VM is run (in the `etc/config.*` files);
- the list of files shared with the host and the associated mount point in the user's home directory or profile, (in the `etc/mountpoints.*` files);
- the program to execute at boot time (the `bin/on-boot.<ext>` file) determined from the VM's configuration and usage context (update, user customization or simply run by the end user);
- some support scripts (in the `lib/` directory);
- if the user has set up a `~/.padsi-agent` in the zone where the VM is run, then that directory is also mounted in the `user/` directory along with `etc/`, `bin/` and `lib/` directories.

The following diagram illustrates this:



The PADS agent is provided as a package which must be installed in the VM (in the installation or in a update phase) and is implemented as an OS level service.

Other SSH keys If the user already has some SSH public keys as `~/.padsi-agent/.ssh/*.pub` files in the zone where the VM is run, then those keys are also appended to the the `~/.ssh/authorized_keys` file in the user's HOME directory in the VM.

2.12.16 Miscellaneous

User configuration As each VM is customized for a specific user before being run and because that VM can only be used by the user for which it was customized, PADS configures the created user in a very "loose" way from traditional user configuration:

- the user has no password;
- the session is automatically opened for the user (graphical or textual).

Files sharing Files sharing is actually "directory" sharing as a VM configuration cannot reference sharing of a single file. Additionally, all files are shared as the UID of the user running the VM.

As stated above, all the directories shared by configuration between the host and the VM are mounted relative to the user's home directory (or Windows profile); no user controlled file can be mounted outside.

However, if it exists, the `~/.padsi-agent` directory exists in the host's home directory of the considered zone, it is mounted read-only in the `user/` directory relative to the resources directory where PADS sets up resources for the VM, namely `/run/padsi-agent/user/` for a Linux guest OS and `Z:\user` for a windows guest OS).

From a technical point of view:

- for a Linux guest OS, the shared files are simply mounted as the expected target directory after having created its parent directory if necessary;

- for a Windows guest OS:
 - the shared directories are mounted as drive letters starting at Z: and going backwards for each shared directory;
 - the target directory is removed if it existed, and its parent directory created if necessary;
 - the target directory is a link to the drive letter.
 - all the drive letters mounted as such are hidden to improve UX.

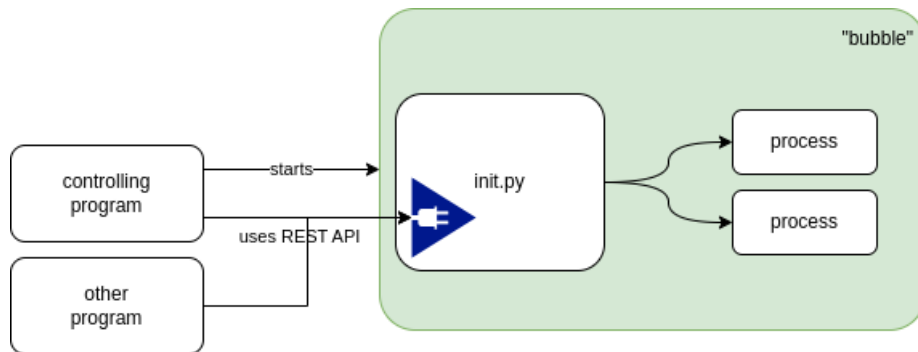
2.13 “Bubbles” implementation

Many components in PADSİ are isolated from one another using what is referred to as “bubbles”. This section describe what a “bubble” is and its architecture.

A “bubble” is a set of processes isolated from the rest of the system using kernel namespaces. At its core a bubble is set up and governed by the [Bubblewrap](#) tool (hence the name). Every “bubble” has a dedicated “init” process (the `init` program) which:

- is the first process started;
- acts as an “init” process in the way it inherits parenthood for all the orphaned processes in the bubble;
- offers a REST API available via a Unix socket (thus benefiting from the Linux’s files permissions to control its access) to manage processes running within the bubble from outside the bubble;
- can restart processes marked as “restart” if they terminate (up to a pre-defined rate to avoid overload due to a buggy process);
- can kill all the processes running in the bubble if some process marked as “required” ever terminates.

The following diagram illustrates this with a bubble having started two processes:



2.14 Command line tools

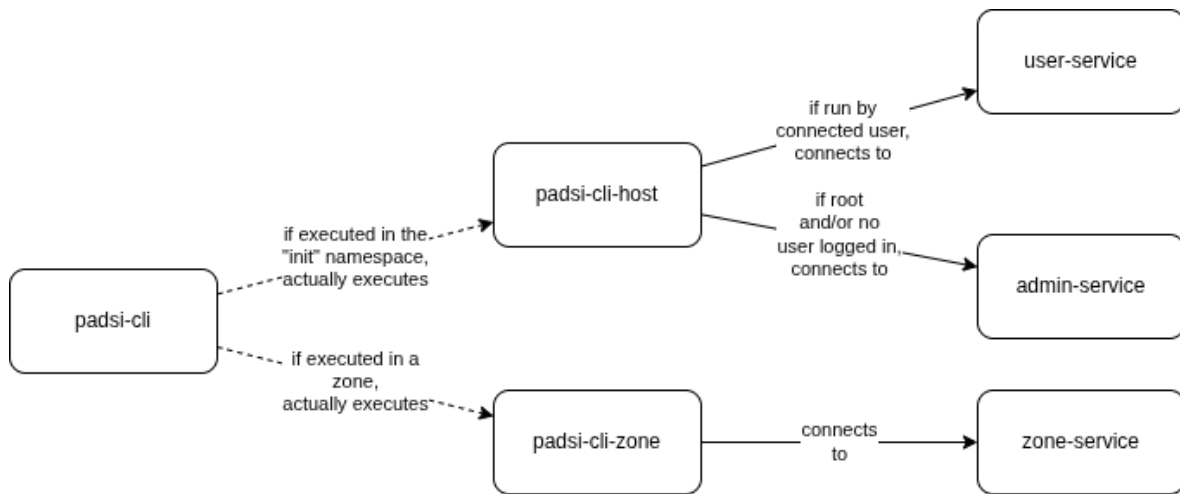
2.14.1 User interactions

Interacting with PADSİ can be done using the command line tool `padsı-cli` which comes as two different flavors:

- one to be used from the “init” (host) namespaces to perform actions like getting the status, or starting programs or virtual machines. This tool is not meant to be used directly by the end user but rather by other tools (mainly the GNOME shell to start programs in various zones when the user clicks on icons);
- one to be used from within a zone to perform actions related to managing virtual machines from the command line.

Both versions are installed in the `/usr/bin/padsı-cli`, the actual version depends on the execution environment.

The following diagram shows which program is run depending on the context, and to which actual service it connects to:



2.14.2 For diagnostics

PADSI administrators or developers can use the `pdsi-sys` tool to obtain or interact with PADSI's running structure, like:

- get the network configuration;
- get information about zones and enter specific namespaces.

This tool requires root privileges for most of its operations.

2.14.3 For administrators

System administration (be it local or via remote access) requires to execute programs in a specific context (refer to section 2.16 for more details). The `pdsi-do` tool can be used to execute programs in that specific context (from local users running `sudo` to services like SSH for remote access).

2.15 Logging

Being a security product, PADSI logs many events which can be later used either to understand bugs or to detect security issues:

- warning and errors events are sent to syslog, and presumably handled by the `journalctl` component depending on the Linux distribution;
- debug and informational events are written to dedicated log files;

The dedicated log files are located in the `log/` directory below the location of all non volatile PADSI related data (as defined in the configuration and which defaults to `/var/pdsi/`). Specifically:

- system wide log files are stored in the `log/` directory;
- per user log files are stored in:
 - `log/<UID>/` for non zone specific events;
 - `log/<UID>/<zone name>/` for zone specific events but not related to any VM;
 - `log/<UID>/<zone name>/VM-<VM ID>/<VM nickname>/` for VM related events in the context of a specific zone.
- administration namespaces (refer to section 2.16) log files are stored in the `log/pdsi/<admin.NS>/` directory.

Specifically:

- DNS resolution events located in `resolv.log` files;
Note: DNS resolution handled locally (for configuration defined and hard coded resolution and for virtual machines' resolution) is not logged;
- events for network traffic blocked by the firewalls are in `firewall.<year>-<month>-<day>.json` files;
Notes: non blocked traffic is not logged.
- events generated by the web infrastructure (refer to section 2.10) are in `web-infra.<year>-<month>-<day>.json` files;
- events generated when using the padsi-do tool (refer to section 2.16.3) are in `padsi-do.<year>-<month>-<day>.json` files.

Note: events are all timestamped in UTC.

2.15.1 Netfilter logging

If the `firewall-logs-group` property in PADSI's global configuration (refer to section 5.1.1) specifies a group number, then all of PADSI's firewall blocked traffic attempts are logged via NFLOG using the specified group number.

Capturing those events to create the associated JSON files mentioned above is done by the “fw-logger” component, which acts similarly to the “ulogd” daemon¹⁷ but is using a hard coded configuration and is spawned in each of the network namespace where netfilter is used¹⁸.

2.16 System administration

This section details the features and corresponding architecture to allow a PADSI system to be administered with regards to the network restrictions which are applied, namely remote access and local administration.

2.16.1 Remote access

Remote access is often used by enterprise administrators or orchestration tools to remotely work on systems or apply policies to them. Remote access is usually performed via a network service like SSH. In this *would-be* situation, the service is exposed either:

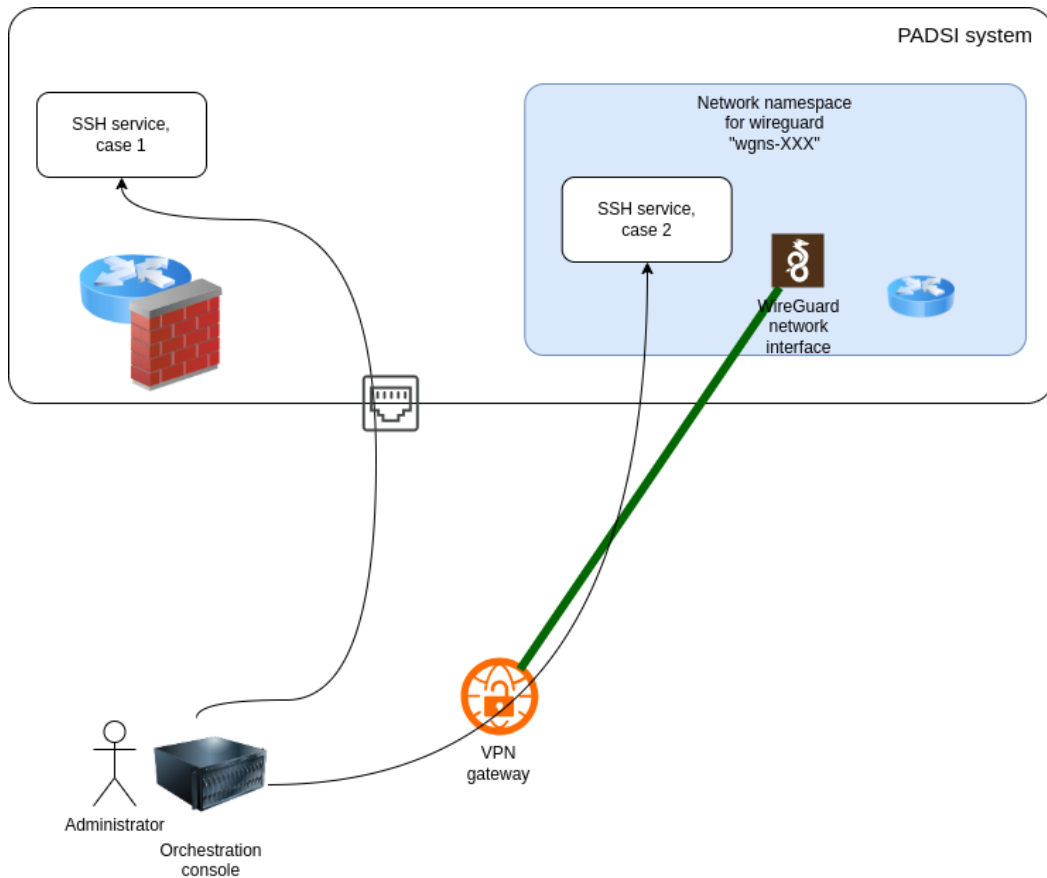
1. directly via the system's network configuration, but is only readily usable when the system is connected to the enterprise's internal network and not when connected to external unknown networks like home or public Wi-Fi networks (because they all perform some sort of network address translation); or
2. via a VPN set up in a way such that all PADSI systems are always connected to a VPN gateway (refer to section 2.5) as long as the PADSI system is actually connected to a network through which the VPN gateway can be reached.

The “natural” approach would be that in the 1st case, the network service is running in the “init” network namespace and a global PADSI configuration which allows the incoming network connection (via the “`host-access`” key, refer to section 5.1.1). In the 2nd case, the network service is running in the network namespace where the VPN network interface is.

The following schema illustrates this *would-be* (not the actual, and with a WireGuard VPN) setup:

¹⁷Refer to <https://www.netfilter.org/projects/ulogd/index.html>.

¹⁸In its default system-bundled configuration, ulogd only gets events generated in the “init” namespace by netfilter and there is no easy way to have it get similar events from other network namespaces.



In both cases, the DNS configuration is the one defined by the network the PADS system is actually connected to¹⁹ (home, public Wi-Fi, corporate, ...) and often points to a non public resolver (provided by the ISP or the enterprise).

This *would-be* setup however has some limitations because the mount namespace is still the “init” one and access to network resources will be partially or totally blocked because:

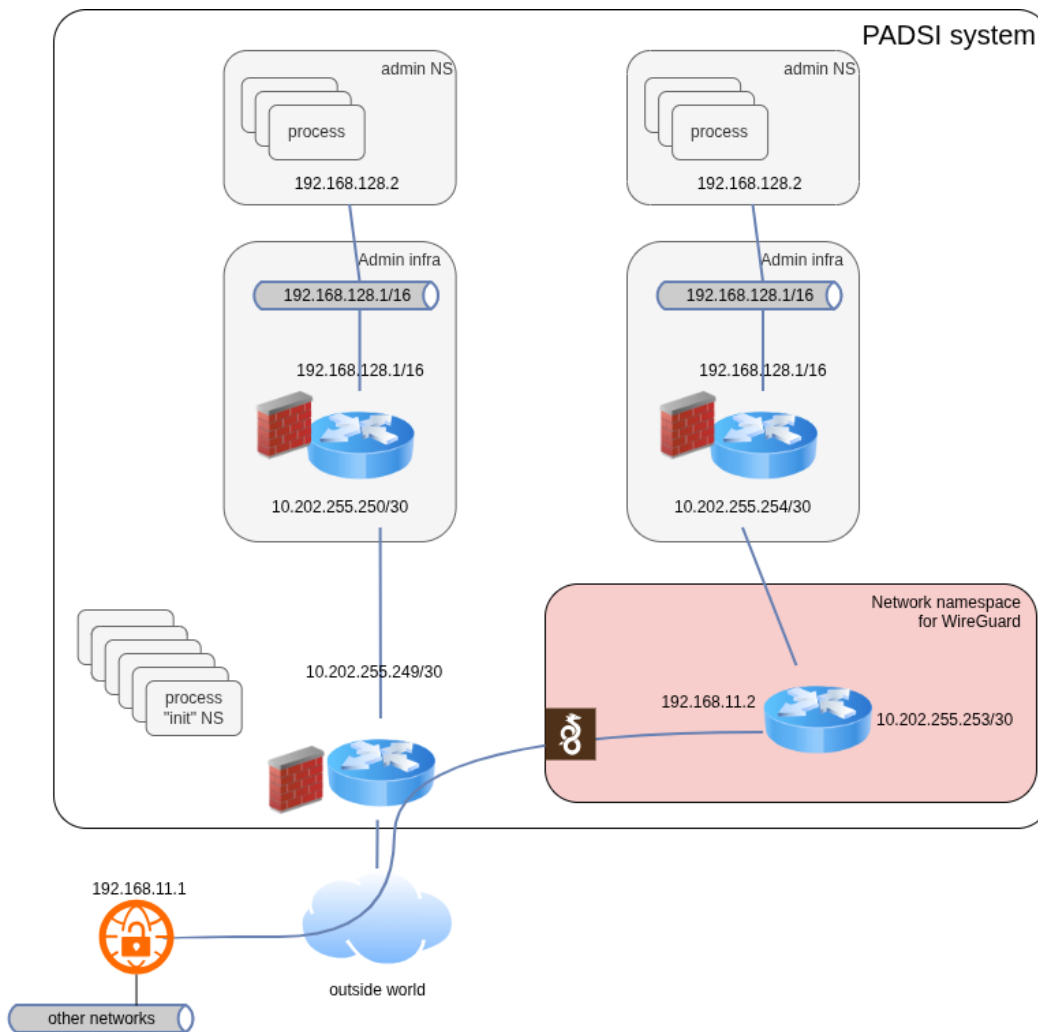
- in the 1st case: the firewall of the “init” zone blocks both the DNS resolution requests and the outbound connections;
- in the 2nd case: the DNS server(s) cannot be generally reached because the traffic is routed through the external VPN gateway and not directly via the physical network interfaces.

So, in both cases, remote administration would be dysfunctional and the actual setup needs to be adjusted as follows:

- the remote access network services is executed in an “**administration namespace**” (**admin NS**) which is a combination of a mount and network namespaces:
 - the mount namespace is derived from the “init” namespace but the `/etc/resolv.conf` file points to a “**admin network infrastructure**”’s DNS service;
 - the network namespace which routes all the traffic trough the admin network infrastructure.
- the “**admin network infrastructure**” is set up like a zone’s infrastructure (refer to section 2.3.3);
- services are exposed via DNAT;
- “moving” services to the correct administration namespace is handled via the dedicated `padsi-do` tool.

The following figure illustrates this setup:

¹⁹PC endpoints’ network connectivity is usually configured automatically via DHCP which also contains the DNS configuration to use.



Compared to the architecture presented in section 2.3.1:

- the network ranges of IP addresses is different (in the 10.202.0.0/16 range but starting from the end, or 192.168.128.0/24);
- the administration namespace is set up when a process is started. Specifically:
 - the network namespace always exist;
 - a mount namespace is actually started when a process is started (via the `padsi-do` tool).
- the admin infra. is configured similarly as for end users usage.

Notes:

- configuring the actual services in an administration NS is not done by PADS but must be performed by a system administrator (refer to section 5.3.1);
- the administration namespaces remain unchanged when the `padsi systemd` system service restarts. However, the underlying infrastructure is restarted which means that any opened incoming connections may experience a little delay during which they will appear as non responsive while the infrastructure is set up.

2.16.2 Local administration

Local administration allows users to perform actions (command line or scripts) with elevated privileges controlled via the `sudo` program. Running `sudo` from a zone will not work as expected because zones loose the ability to elevate privileges via “setuid” and because `sudo`’s configuration is not available (as a security feature). Escaping the zone’s network and mount namespaces is not possible (as it would be a huge Linux kernel vulnerability).

Local administration thus needs to be performed by executing `sudo` from a program running in the “init” namespaces (refer to section 1.4.1).

However, even once this is done, the `sudo` program will actually be in the same situation as the remote access (1st case) with the same limitations, and so the setup also needs to be adjusted. The same solution is applied here as for the remote access, and `padsi-do` tool is also used for that purpose.

2.16.3 padsi-do tool

This tool runs a command in a combination of namespaces:

- the network namespace of the selected administration namespace (if none specified, tries to find it if PADSi only created one);
- a new mount namespace based on the “init” one where a dedicated `/etc/resolv.conf` file points to the correct DNS resolver;
- relies on `sudo` to filter which commands can actually be executed by the user;
- if running from a non root user, restarts itself using `sudo`.

2.17 Misc.

2.17.1 Controlled access to zone’s associated files

The `data-access-guard` program (an eBPF LSM program type) prevents programs running in the “init namespace” (and not as root) from:

- reading or writing any zone’s associated files (to avoid zone’s policies bypass, refer to section 2.2);
- modifying the log files generated by components in the zones.

This program is an LSM type program which intercepts files accesses and only allows access for the root user. It logs all denied accesses to `syslog`.

2.17.2 Other

- Printing
- external mass storage handling

2.17.3 Side benefits

The following problems happen to be solved by PADSi as side benefits:

- enterprises often struggle to prevent end users to multi-attach their endpoint to several networks (as the endpoint is the a *de facto* router between the routers to which it is attached by network interfaces). In a PADSi system, being simultaneously attached to several networks is not a security risk as the endpoint’s firewall blocks any strictly required network communications.
- end users and administrators are given a near complete visibility²⁰ into all the network traffic, in the “init” namespace, per zone and per administration namespace. Such information allows to adapt the network rules, lets users understand how programs behave almost in real time and security administrators be more efficient with regards to malicious communications.
- security services can be deployed and executed in an “area” where the end user (or any malicious software running with his privileges) has no reach, thus making it difficult or impossible for any malware to “kill” them (like EDR killers do).

3 Using PADSi

This section highlights the various tools available to interact with PADSi.

²⁰Except for legitimate network traffic which is not logged.

3.1 padsi-cli

The `padsi-cli` tool is the central command line tool to perform actions on a running PADSIs instance (it does not allow one to create or modify the PADSIs configuration); it is for example used to create customized desktop entry files for programs (refer to section 2.8.1). This tool can be also executed from a zone even though it provides for much less (non required) features.

This tool can be used from a program outside of any zone (refer to section 1.4.1) or from within a zone where it is limited to manage virtual machines in the zone²¹.

The tool's global options are:

- `show help`:
 - `-h`: globally;
 - `<command> -h`: for a specific command;
- `--config-dir`: specify a directory where the configuration files are;
- `-j`: return JSON data instead of a textual format;
- `-U`: force to be used as a specific user (when used as root, otherwise will be denied);
- `-v`: show verbose information when applicable.

3.1.1 Getting information from the system

The following commands are supported:

- `status`: to get a quick status of the PADSIs service;
- `zones`: to get the list of all the zones declared in the configuration;
- `zone-info`: to get information about a zone;
- `net-rules`: to get information about which network endpoint can be reached from which zone;
- `net-usage`: to get information about network traffic allowed and/or blocked;
- `admin-ns`: to get information about administration NS (refer to section 2.16).

3.1.2 Running programs

There is only a single command to run a program in a specific zone: `run`.

3.1.3 Managing virtual machines

The following commands are supported:

- `vms`: to list all the declared virtual machines, and in which zone they can be used;
- `vm-save` and `vm-load`: to respectively create an archive of a VM and load a saved VM in a system;
- `vm-discard`: to delete a specific version of a VM;
- `vm-clean`: to remove unused and obsolete VM versions;
- `vm-import`: to import the virtual hard disk of a VM from another (non PADSIs) system;
- `vm-install`: to create a new VM using an OS installer provided as an ISO file;
- `vm-update`: to update an existing VM;
- `vm-publish`: to make an installed or updated VM available to users;
- `vm-merge`: to merge a snapshot and its “parent” VM version;

²¹Actually even though the program is still named `padsi-cli`, the actual programs are not the same.

- **vm-status**: to display the status of a VM;
- **vm-start**: to start a VM, or show it if it's already running;
- **vm-display**: to display a running VM without starting it if it's not running;
- **vm-create-launcher** and **vm-delete-launcher**: to respectively create or remove a desktop entry file representing a VM.

vm-import This command imports a VM disk image and its associated VARS file in a PADS system. The command returns immediately as it has created a (potentially long running) task.

Ex. before import where there is already an existing VM version:

```
$ padsi-cli vm-status thevm
VM Versions:
  base.0: referenced by user.0, STOPPED
  user.0: references base.0, referenced by snap.0, STOPPED
```

To start the VM import:

```
$ padsi-cli vm-import thevm /path/to/thevm.img /path/to/thevm.img.VARS
VM version: base
```

To check the task:

```
$ padsi-cli status
Started zones:
[...]
Running tasks:
- Importing VM 'win11' from '/path/to/thevm.img' and '/path/to/thevm.img.VARS' (msg: 'None')
```

And when VM is finally imported, notice there is a STAGED VM version in the STOPPED status:

```
$ padsi-cli vm-status thevm
VM Versions:
  staged (STOPPED)
  base.0: referenced by user.0, STOPPED
  user.0: references base.0, referenced by snap.0, STOPPED
```

The publish the new VM so it can be used:

```
$ padsi-cli vm-publish -m "New VM" thevm
{
  "task": "Publishing VM 'thevm' for user XXX"
}
```

The new VM can now be used (identified as **base.1**):

```
$ padsi-cli vm-status thevm
VM Versions:
  base.0: referenced by user.0, STOPPED
  base.1: STOPPED
  user.0: references base.0, referenced by snap.0, STOPPED
```

3.2 VM viewer

3.3 User defined desktop entry (launcher) files

Beyond the applications which are listed in a zone and for which PADS will create a desktop entry file with the associated icon (outlined as necessary), each user who want a specific launcher and associated icon can create their own desktop entry file in the `$HOME/.local/share/applications` directory²² of the zone.

For example, if the user wants to use the KeepassXC password safe program in a specific zone, all is needed is to copy the system's one (provided KeepassXC is installed of course):

```
$ cd ~/.local/share/applications
$ cp /usr/share/applications/org.keepassxc.KeePassXC.desktop .
```

²²This is the directory where users are expected to create desktop entry files in almost any Linux system.

3.4 padsi-do

The `padsi-do` tool allows one to execute a command in an administration namespace (refer to section 2.16).

It must be used via `sudo` (if not, it will call `sudo` itself) and can run any command which is also allowed by the `sudo` configuration, without asking for any authentication as this should already have been done when the tool is executed by `sudo`; **system administrators need to take this into account when designing their policies**.

This tool will execute any command and its arguments it is passed, and accepts the following command line options (which must be specified first for obvious reasons):

- `-l` or `--list`: to list all the available administration namespace ;
- `-a` or `--adminns <admin. NS>`: to specify an administration namespace if more than one are defined;
- `-d` or `--delay <delay>`: to specify a delay during which the tool tries to use the administration namespace before failing if it does not exist.

3.5 padsi-sys

The `padsi-sys` tool can be used to show extra information about a running instance of PADSİ (for a specific user when the tool connects to the user specific PADSİ service). It accepts the following commands:

- `adm-list`: to list all administration namespaces and their properties.
- `config-test`: to test the validity of the whole configuration and report errors. Does not print anything if there is no error.
- `dump`: prints a JSON formatted status of the whole system;
- `network`: prints the network configuration of the system, the administration namespaces and the traffic shapers (refer to section 2.4). For example:

```
Network interfaces in the 'init' namespace:
  wlp0s20f3 => 192.168.1.49/24
  enx8a362049378 => 192.168.1.56/24
Administration namespace: admin
  net namespace: admns-admin-p
    - LISTEN tcp:22 (PID: 1051379)
    - LISTEN tcp6:22 (PID: 1051379)
  Via traffic shaper: ts0
Traffic shaper: ts0
  net namespace: wgns-ts0
  ts0 VPN is fonctionnal
  network interfaces (in namespace):
    - ve1v-997: 10.202.255.253/30
    - wg: 192.168.34.3/32
Traffic shaper: ts1
  net namespace: wgns-ts1
  ts1 VPN is fonctionnal
  network interfaces (in namespace):
    - ve1de-1000: 10.202.0.1/30
    - veeen-1000: 10.202.0.5/30
    - ve3o-1000: 10.202.0.9/30
    - wg: 192.168.228.3/32
```

shows:

- the two physical interfaces of the host (in the “init” network namespace): `wlp0s20f3` and `enxc8a362049378` and their current IP addresses;
 - the two traffic shapers (`ts0` and `ts1`) currently set up, their state and the network interfaces present in each associated network namespace and their IP addresses.
- `pstree`: lists all the processes’ tree which the PADSİ service has launched (no need to be root);
 - `zone-exec`: executes a program within a zone’s part;
 - `zone-network`: shows the network set up for any given zone, e.g.:

```
$ sudo padsi-sys zone-network myzone
Traffic shaper: ts1
  net namespace: wgns-ts1
  ts1 VPN is fonctionnal
  network interfaces (in namespace):
    - veide-1000: 10.202.0.1/30
    - veeen-1000: 10.202.0.5/30
    - ve3o-1000: 10.202.0.9/30
    - wg: 192.168.228.3/32
INFRA: init PID: 901157
  - br0: 192.168.0.1/16
  - eth0@if3: 10.202.0.2/30
APPS: init PID 901482
  - br0: 192.168.0.1/16
  - eth0@if3: 10.202.0.2/30
```

- **zone-parts**: lists all the “features” instantiated for any given zone, e.g.:

```
$ sudo padsi-sys zone-parts myzone
INFRA: init PID 901157
USERFILES
APPS: init PID 901482
```

shows that the **myzone** zone is set up with:

- a bubble (refer to section 1.4.1) where the user’s applications are run (the “APPS” part), where the “init process” has a PID of 901482 (as seen in the “init” namespaces);
- a bubble where the zone’s infrastructure processes (refer to section 2.3.3) are running (the “INFRA” part) with an “init” process with PID 901157;
- a mount point assembling all the files and directories mounted for the zone (the “USERFILES” part, refer to section 2.2), which has no associated process and thus is not a bubble and has not “init” process.

When root privileges are required, the tool can be used via **sudo** or logged in as root (in which case, the **-U** flag should be used to specify which user to consider when applicable).

4 Installation

4.1 Requirements

The installation requirements are:

- a Debian 13 (trixie) distribution, not yet tested on any other OS;
- the system has to be configured to use nftables (and the associated **nft** tool) and not the obsolete iptables;
- an access to the Debian software repository or a mirror;
- the GNOME desktop environment (initial support would be very easy but integration of PADSI’s UI hints may require significant more work) with Wayland (not X11).

4.2 Installation

To install PADSI,

1. install the **padsi_XXX_all.deb** package using apt to also install all the dependencies;
2. create the configuration files in **/etc/padsi** (refer to §5.1). The default configuration in **/usr/share/doc/padsi/example** should provide a “Quick start” for anyone wishing to quickly test PADSI, and then refine the settings:
 - (a) create a **/etc/padsi** directory (should be readable by all users but not writable);
 - (b) copy all the files in **/usr/share/doc/padsi/conf-example** to **/etc/padsi/**.
3. test the configuration (an invalid one will prevent the PADSI service from starting):

```
# padsi-cli config-test /etc/padsi
```

4. start the PADSI service:

```
# systemctl start padsi
```

5. once everything is working as expected, enable the service to be started at each reboot:

```
# systemctl enable padsi
```

4.3 VM load quick start

In case a VM archive has been saved from another PADSI instance, it can be directly loaded as follows:

1. define an environment variable containing the ID of that new VM definition (adapt depending on the shell used):

```
$ export VMID=test
```

2. ensure the VM configuration has been done, check using:

```
$ padsi-cli vm-status $VMID
No base VM version defined yet, use 'padsi-cli vm-install' to create one
No VM version can be merged
No VM version should be discarded
```

3. load the VM's resources, assuming for example that the resources are in the `vm-exported.tar` file:

```
$ padsi-cli vm-load vm-exported.tar
```

4. ensure the VM configuration has been done, check using:

```
$ padsi-cli vm-status $VMID
VM Versions:
  base.1: STOPPED
No VM version can be merged
No VM version should be discarded
```

The VM is now ready for its first execution.

4.4 VM installation quick start

This section is a quick start to set up a running VM, the steps are:

1. if not yet done, download the OS installation media from the OS provider and check its integrity and authenticity;
2. create a VM definition file for example as the “`test.vm`” and place it in PADSI's global configuration directory (`/etc/padsi/` by default), making sure the current user is given the privileges to install the VM, and define an environment variable containing the ID of that new VM definition (adapt depending on the shell used):

```
$ export VMID=test
```

3. verify the VM's definition is valid (it is if no error is reported):

```
$ padsi-cli config-test
```

4. restart the PADSI service as root:

```
# systemctl restart padsi
```

5. check the new VM definition has been taken into account:

```
$ padsi-cli vm-status $VMID
No base VM version defined yet, use 'padsi-cli vm-install' to create one
No VM version can be merged
No VM version should be discarded
```

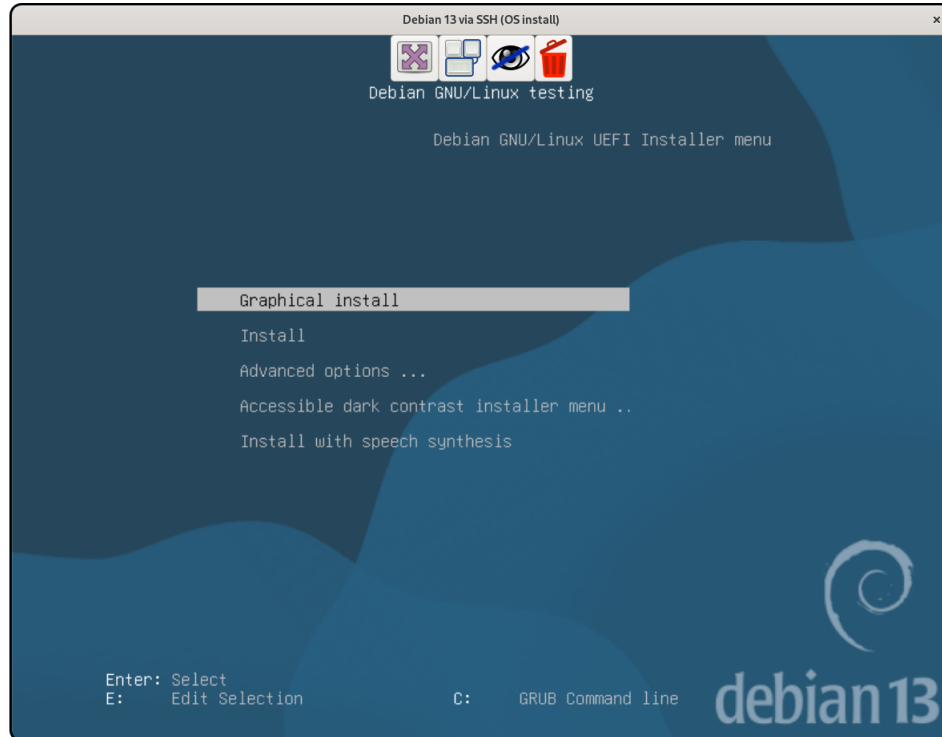
6. install the VM specifying both the ISO image of the OS to install²³ (a Debian testing in this example) and

²³For Windows VM, refer to annex A.

the PADSİ agent package as a resource to be able to install it after the OS's installation is done:

```
$ pdsi-cli vm-install \
--resource packages/pdsi-agent_0.7.25_all.deb pdsi-agent_0.7.25_all.deb \
$VMID debian-testing-amd64-netinst.iso
VM version: staged
```

The command line tool exits stating the “staged” VM version is created and will be used while the VM is being installed. The VM viewer will then show the UI to install the OS, for example with a Debian 13 installation:



7. finish the OS installation and install the PADSİ agent (refer to section [2.12.15](#)) in the VM from the package in the pseudo CD-ROM²⁴ created on the fly (without it, the automatic VM customization for the user won't work and operations will have to be done manually);

8. shut down the system properly;

9. check the status of the VM:

```
$ pdsi-cli vm-status $VMID
VM Versions:
  staged (STOPPED)
No base VM version defined yet, use 'pdsi-cli vm-publish' to publish the staged version
No VM version can be merged
No VM version should be discarded
```

10. publish the VM to be used with a comment:

```
$ pdsi-cli vm-publish -m "Initial version" $VMID
{
  "task": "Publishing VM 'test' for user 1000"
}
```

11. check the publish operation has succeeded:

²⁴The `/dev/cdrom` device will probably correspond to the OS's ISO image, so use the `/dev/sr1` device instead.

```
$ padsi-cli vm-status $VMID
VM Versions:
  base.0: STOPPED
No VM version can be merged
No VM version should be discarded
```

At this stage, the VM is ready for its first use. Note however that this new VM must be declared as an allowed VM in the zone(s) where it will be used.

5 Configuration and administration

5.1 Configuration

In order to benefit from PADSI's security, one should respect the following rules:

- define business activities which correspond to a set of network services and of files, and assign a zone to each of these activities;
- **never set up a zone which at the same time allows unrestricted network access and access to sensitive information.** This ensures an attacker which is able to execute his own malicious code in any zone will either be able to communicate with his malicious code OR have access to internal networks and/or sensitive information, but not both.

PADSI's configuration is composed of multiple files in the `/etc/padsi` directory (which can be changed in the `padsi.service` file if necessary):

- the `padsi.conf` file is the “entry point” of all the configuration, it contains the global settings.
- the `*.zone` files each describe a “zone”;
- the `*.netres` files each describe a set of network resources;
- the `*.vm` files each describe a virtual machine;
- the `*.tsp` files each describe a traffic shaper configuration.

5.1.1 Global configuration

The global configuration file `padsi.conf` is a JSON file containing the following keys:

- `"var-dir"`: points to the directory which will contain all non volatile PADSI related data such as user's files in the various zones, virtual machines resource, etc. It default to `/var/padsi`. (which is created the first time the PADSI systemd service is started)
- `"xdg-data-dirs"`: a list of XDG directories of the system (refer to the `XDG_CONFIG_DIRS` variable as per [XDG Base Directory Specification](#));
- `"networking"`: specifies the global networking settings of PADSI, via the sub keys:
 - `"host-network"`: network used by PADSI to create all the virtual network structure in the host's “init” namespaces. Defaults to `10.202.0.0/16` and should be left unchanged unless this network collides with some other enterprise network settings.
 - `"host-access"`: list of network flows allowed for applications in in “init” namespaces when PADSI is running (there should only be the minimum necessary in and out network flows). This section is composed of network rules (refer to section 5.1.4) as lists in the `"out-rules"` (for outbound connections) and `"in-rules"` (for inbound connections) sections immediately below `"host-access"`.
- `"clipboard"`: a list of copy/paste rules to prevent data being copied from some zones to be pasted by possibly malicious application executed in less secure zones. Each rule is made of the following attributes:
 - `"descr"`: a description of the rule;
 - `"action"`: the action to be done, among “deny” and “allow”;

- “rule”: the actual rule, in the “<copy zone>[, <copy zone>, ...] > <paste zone>[, <paste zone>, ...]” format and where “*” can be used to reference all defined zones.

WARNING: if not blocked by configuration, copy/paste operations are allowed by default.

- “xdg-default-zone”: the name of the zone in which the user is expected to execute most programs. If not null, the windows of programs in that zone will not have any outline;
- “firewall-logs-group”: null to log firewall blocked attempts to syslog, or a group number (integer) to log through a netlink socket to the specified group (requires an additional system configuration to handle those logs, probably using the ulogd daemon);
- “nozone-apps”: list of applications, identified by their [Desktop File ID](#), which will be by default executed outside of any zone (they will be executed in the “init” namespaces). It should contain applications which don’t allow to author files or require network resources like the calculator, the disk mounter, the GNOME shell’s extensions manager or the help system. If null, then all applications will be considered to run outside of any zone.

5.1.2 Zones configuration

Each zone is defined by a single file named <zone name>.zone (the zone name must start with a lower case letter and contain only lower case letters and numbers) which is a JSON file containing the following keys:

- “friendly-name”: the user friendly name of the zone;
- “start-mode”: how PADS starts the zone, among:
 - “always”: the zone’s infrastructure and the applications’ part is always started even if no user requested program is running in it (for faster startup, but consumes a bit of resources all the time);
 - “always-infra”: the zone’s infrastructure is always started but the and the applications’ part is not (it will be started when the user needs to run a program in it);
 - “on-demand”: the zone infrastructure and the applications’ part are started only when the user starts a program in that zone.
- “color”: the color of the outline applied to windows of programs running in the zone, the “#rrggbb” color format, or null if no outline must be shown;
- “options”: specific options for the zone, among:
 - “DESKTOP-NOTIFICATIONS”: if true, programs in the zone will be able to display notifications in the user’s desktop.
 - “DNS-BLOCKLIST”: if set, must point to a file containing blocked domains, imported for example from <https://github.com/StevenBlack/hosts>, and in the following format (which can be generated using the helpers/generate-blocklist.sh script):

```
server:
  local-zone: "bad.domain.com" refuse
  local-zone: "bad.domain2.com" refuse
  ...
```

Note: this option is only useful if the zone actually performs any DNS resolution.

- “DRM”: if true, allow access to the Direct Rendering Manager (/dev/dri);
- “FIDO2”: if true, permits usage of FIDO2 authenticators (e.g. for WebAuthn);
- “GPG-CARD”: if true, permits usage of GPG cards (note that the GPG keys still need to be “manually” imported in the zone’s \$HOME/.gnupg directory);
- “FUSE”: if true, programs will be able to mount filesystems using FUSE;
- “MEDIAS”: if true, the contents of the /media directory where mass storage medias are plugged will be accessible;
- “MOUNT-POINTS”: if not empty, identify directories or files which will be mounted in the zone’s context. Mount points are listed as a pair of:

- * a file path (directory or file) in the “init” mount namespace, a non full path (i.e. not starting at the root) will be interpreted relative to the configuration directory (`/etc/padsi` by default);
- * a file path where it will be mounted (read-only) in the zone’s context, a non full path will be mounted in the user’s home directory;

Note: when mounting directories, the mounted directory may also contain other files or sub-directories. This is typically the case for the `/etc` directory which may contain other files or directories required by the applications which will run in the zone.

- “NET_LOG_ONLY”: if true, network restrictions are only logged but not actually enforced;
- “PKCS11”: allows one to specify a PKCS#11 driver (shared library) to use smartcards, the option must be specified as a dictionary with:
 - * “driver-name”: a user friendly name for the driver;
 - * “driver-file”: the path to the actual shared library file.
- “PKI”: allows to declare certification authorities’ certificates as trust anchors. Must be specified as a dictionary where keys are the user friendly name for a CA and associated values are paths to the associated PEM encoded certificate;
- “SCREEN-SHARE”: true if programs in the zone will be able to share (i.e. stream, like in conference calls) the whole Linux session’s screen or any window of any zone;
- “VM-ONLY”: name of a VM which users have to use (they will not be allowed to execute applications in the zone, refer to section 2.12.1);
- “WEB_REDIRECTION”: allows to enable the Web redirection feature (refer to section 2.10.3). The configuration is a dictionary with:
 - * “zones”: a list of zone names where redirections can be proposed to the users.
- “X11”: if true, programs in the zone will be able to access the system-wide X11 system to display applications.

WARNING: this poses a security risk as such applications will then be able to interfere with all the X11 programs of all the zones. It should thus only be enabled when absolutely required.

- “network”: specifies the network settings of the zone (set to `null` to disable networking in the zone), as the following keys:

- “traffic-shaper”: name of the traffic shaper to use, if any.
- “dns-resolvers”: list of the DNS resolvers to use:
 - * like `["8.8.8.8", "1.1.1.1"]` to use specific DNS resolvers, or
 - * `null` to avoid querying any external DNS resolver (local static DNS resolution can still provided),
 - * or `[]` (empty list) to use the system’s configured DNS resolvers (NOT YET IMPLEMENTED).

DNS over TLS (DoT) is also supported, for example `["86.54.11.213@853", "86.54.11.13@853"]` to use the EU’s DNS’ privacy and security focused resolvers²⁵.

The following syntaxes are supported to specify a single resolver: `<IP address>[@<port>][@<protocol>]` where the protocol can be “DoT” for DNS over TLS.

- “out-rules”: list of applicable outbound filtering rules for the zone, refer to section 5.1.4 for the syntax of each rule. The filtering decision is the action specified in the first matching rule, and a deny if no rule matched.
- “web-proxy”: can be `null` or can specify if a forwarding Web proxy need to be used (a non forwarding Web proxy has no use since all the filtering is already done by PADSI). This may be the case if the Web requests have to go through an already existing Web proxy. Specify the Web proxy using:
 - “proxy”: the actual proxy to use in the format `<ip address>:<port number>;`
 - “out-rules”: list of rules applied locally by the proxy before forwarding the requests to the Web proxy down the chain, refer to section 5.1.4 for the syntax of each rule.

²⁵DNS4EU , refer to <https://www.join4eu.eu/>

- **"mounts"**: specifies the mount points in the zone: directories in the user's home directory in the zone which point to directories either in the real user's home directory or in other zones. Each mount point in the zone (i.e. the directory in the zone, relative to the home directory) is a key to which the following attributes are associated:

- **"source"**: where the directory actually is;
- **"mode"**: read-write (**"rw"**) or read-only (by default if not specified)

The directories used as keys (relative to the home directory) can contain variables in the form `{<XDG dir>}` where the XDG dir can be `"DESKTOP"`, `"DOWNLOAD"`, `"TEMPLATES"`, `"PUBLICSHARE"`, `"DOCUMENTS"`, `"MUSIC"`, `"PICTURES"`, `"VIDEOS"`. For example, `{DOWNLOAD}/Internet` corresponds to the `Download/Internet` directory in a English locale or `Téléchargements/Internet` in a French locale²⁶.

The directories used as sources (also relative to the home directory) can also contain variables in the form `{[<zone name>_]<XDG dir>}` with the same XDG constants. If the zone name is not specified, then the directory in the actual user's home directory will be used.

For example then, the following in a zone specifies that the `$HOME/Download/Internet` (in an English locale) directory will actually be the `$HOME/Download` directory of the zone "internet" and its contents will be read-only:

```
"{DOWNLOAD}/Internet": {
    "source": "{internet_DOWNLOAD}",
    "mode": "ro"
}
```

And the following in a zone specifies that the `$HOME/Documents` (in an English locale) directory will actually be the real `$HOME/Documents` directory of the user:

```
"{DOCUMENTS}": {
    "source": "{DOCUMENTS}"
}
```

- **"apps"**: a list of applications which are expected to be used in the zone, as identified by their App ID (like "org.gnome.Terminal"). Other applications can still be used but there will be no launcher icon and the GNOME shell will not be able to associate the correct icon's outline.
- **"virtual-machines"**: contain the list of virtual machines which may be used (run by a user²⁷) in that zone, along with some more possible restrictions. Thus each entry will be the VM's ID (defined as a `<vm ID>.vm` file, refer to section 5.1.6), and the restrictions are specified via the **"specs"** and **"network"** sections as described in section 5.1.6.

5.1.3 Traffic shaper

Each traffic shaper (refer to section 2.4) is defined by a single file named `<traffic shaper name>.tsp` (the traffic shaper name must start with a lower case letter and contain only lower case letters and numbers) which is a JSON file containing the following keys

A traffic shaper's configuration is a JSON object with the following keys:

- **"type"**: the actual traffic shaper type to use ("wireguard" being the only supported for now);
- **"config"**: the configuration of the selected traffic shaper, as a JSON object.

WireGuard traffic shaper The WireGuard traffic shaper accepts a the following configuration keys:

- **"file"**: points to the actual WireGuard configuration file (an INI style textual file) which:
 - must be protected to prevent any read or write by end users;
 - can be in any directory in the system if the specified path is absolute;

²⁶Linux behaves differently than Windows and MacOS with regards to the "standard" top level directories present in each user's home (or profile) directory: the name presented to the user via any UI is actually the real directory name where both Windows and MacOS present a translated name in the UI while keeping the real name of the directory in English, which can be confusing.

²⁷The installation and update of virtual machines is not taken into account here.

- is searched below PADSI's configuration directory (`/etc/padsi` by default) if its path is relative.
- **"ping-test"**: optional IP address to ping to actually determine if the VPN link is actually working as expected (testing will not be possible if absent or null);
- **"interface"**: optional name the WireGuard interface will have (defaults to "wg" if unspecified or null).

5.1.4 Network rules

There are two types of network rules: the simple one and the ones referencing a network resources set (refer to section 5.1.5). In both cases, there is a mandatory **"action"**, among **"allow"** and **"deny"**.

Note: **wherever network rules can be specified, the default rule for network flows which have not matched any rule will always be a deny.**

Simple rules then also need:

- an optional description, used only to help in the rule's maintenance;
- a definition of the "other" part of a connection, as either:
 - a **"endpoint"**: IP addresses or CIDR, or domain names and possibly the transport means (TCP and/or UDP) and port numbers, like `"*.pool.ntp.org. ^ udp ^ 123"` to represent any UDP connection on port 123 to any server ending in `.pool.ntp.org`.
 - a **"netres"** a reference to a "network resources set", refer to section 5.1.5.
- an optional **"resolve"** section: a list of IP addresses the domain names will be resolved to, in the form `"A/<response-validity>/<response as IPv4>"` where:
 - `"<response-validity>"` is the TTL of the response (in seconds);
 - `"<response as IPv4>"` is the IPv4 address.

Below is an example of a list of network rules:

```
[
  {
    "action": "allow",
    "descr": "Incoming SSH connection (not on the default port)",
    "endpoint": "* ^ tcp ^ 12345"
  },
  {
    "action": "allow",
    "netres": "testing"
  },
  {
    "descr": "Test server",
    "action": "allow",
    "endpoint": "myserver.local. ^ tcp ^ 80,443",
    "resolve": [ "A/1200/192.168.1.4" ]
  }
]
```

5.1.5 Network resources sets

A network resources set is a named collection of endpoints grouped into a single entity for easier maintenance. Network resources sets are defined in **.netres** files which are in the JSON format where a key correspond to each network resources set which then is defined by:

- **"descr"**: the network resource set's description (for easy maintenance);
- **"endpoints"**: a list of endpoints part of the network resources set, each an IP address or CIDR, or a domain name and possibly the transport means (TCP and/or UDP) and port numbers.

The network resources set to use MS365 can easily be built from Microsoft's own JSON definition using the **padsi-ms365-importer** tool as follows:

1. generate a UUID for the system (keep it in between downloads);

2. download the JSON file from <https://endpoints.office.com/endpoints/worldwide?clientrequestid=<uuid>>²⁸, e.g as the `worldwide.json` file;

3. generate the `.netres` file (e.g. as the `ms635.netres` file):

```
padsi-ms365-importer create worldwide.json > ms365.netres
```

Note: the generated file contains several sections as provided by Microsoft like `ms365-common` (Microsoft 365 Common and Office Online) or `ms365-exchange` (Exchange Online) like in the example below.

4. integrate this file in padsi's configuration (the actual name of the file does not matter but its extension must be `.netres`);
5. reference the network resources sets in the zone's configuration files,

This will for example produce the following (only a few lines shown):

```
{
  "ms365-exchange": {
    "descr": "Exchange Online",
    "endpoints": [
      "13.107.6.152/31~tcp^80,443",
      "13.107.18.10/31~tcp^80,443",
      "13.107.128.0/22~tcp^80,443",
    ]
  }
}
```

5.1.6 Virtual machines configuration

A virtual machine is defined by a `.vm` configuration file, the name of the file being the name of the virtual machine. It is a JSON file with the following top sections:

- **"specs"**, which contain the specifications of the virtual machine:
 - **"descr"**: a human readable description of the VM,
 - **"variant"**: the OS in the `<OS type>/<OS version>` where the OS type is "windows" or "linux", and the OS version in the form `<OS distribution>[/<number>]`, for example: "windows/10", "windows/11", "linux/debian", "linux/debian/12".
 - **"disk-size-mb"**: the virtual dist size in Mb;
 - **"mem-mb"**: the memory size, in Mb;
 - **"nb-cpu"**: the number of CPUs allocated to the VM;
 - **"secure-boot"**: true if Secure Boot is enabled.
- **"admin"**, which contains references to scripts used when booting or stopping the VM, depending on the context in which the VM is running. There can be a script name associated with each `<context>-script` key, the associated scripts will be searched in PADSI's `vm-management/scripts/<OS type>` directory. The contexts are:
 - **"update"**: when the VM is being updated;
 - **"customize"**: the transient state when the VM is being customized for a user;
 - **"run"**: when the VM is being used by a user
 - **"shutdown"**: when the VM is being shut down.
- **"install", "update", and "run"**: contain the constraints on the VM when is it respectively installed, updated and customized or run by the user. The contents of these sections is:
 - **"allowed-users"**: users (the root user is always allowed) which are allowed to use the VM in the associated context, as:
 - * a list of user names or group names (which must be prefixed by the "%" character);
 - * null to allow all users.

²⁸Refer to <http://aka.ms/ipurlws>.

- **"with-net"**: true if the VM has network access;
- **"show-ui"**: true if the VM's UI is shown when it starts;
- **"network"**: a network configuration, using the same syntax as for the zones' configuration (refer to section 5.1.2) but with only the rules taken into account (refer to section 5.1.4)
- **"mounts"**: allowed only when usage is **"run"**, list of directories mounted in the virtual machine, in the same way as the ones for the zones (refer to section 5.1.2) with however the following difference: mounted directories can be outside of the user's home directory (or profile) if the path is a full path.

5.1.7 Administration namespaces' configuration

Administration namespaces allow remote access and local administration of a PADS system, refer to section 2.16 for more details. Each administration namespace is defined by a single file named `<name>.adms` (the name must start with a lower case letter and contain only lower case letters and numbers) which is a JSON file containing the following keys:

- **"friendly-name"**: the user friendly name of the zone;
- **"traffic-shaper"**: name of the traffic shaper to use, if any.
- **"network"**: specifies the network settings of the zone (set to **null** to disable networking in the zone), as the following keys:
 - **"dns-resolvers"**: list of the DNS resolvers to use, like `["8.8.8.8", "1.1.1.1"]` for example, or **null** to avoid querying any DNS resolver (local static DNS resolution can still be provided).
DNS over TLS (DoT) is also supported, for example `["86.54.11.213@853", "86.54.11.13@853"]` to use the EU's DNS' privacy and security focused resolvers²⁹.
The following syntaxes are supported to specify a single resolver: `<IP address>[@<port>][@<protocol>]` where the protocol can be "DoT" for DNS over TLS.
 - **"out-rules"**: list of applicable outbound filtering rules for the zone, refer to section 5.1.4 for the syntax of each rule. The filtering decision is the action specified in the first matching rule, and a deny if no rule matched;
 - **"in-rules"**: list of applicable inbound filtering rules for the zone, where only IP based filtering is allowed, refer to section 5.1.4 for the syntax of each rule. The filtering decision is the action specified in the first matching rule, and a deny if no rule matched.
- **"web-proxy"**: can be **null** or can specify if a forwarding Web proxy, refer to section 5.1.2 for the syntax.

5.2 Patching / new versions of PADS

update the package normally and restart the service.

ATTN: any user run program will be closed

5.3 System administration

5.3.1 Remote access

Remote access to a PADS system is often required for administration purposes (to apply policies, perform verifications, install packages, ...); refer to section 2.16 for more explanations about the specificities of a PADS system. This section details how to set up a remote access to a PADS system.

The first thing is to determine the network(s) on which PADS systems will be connected and where remote access will be performed, for example:

- corporate networks: in this case, a remote access to a PADS system will only be possible when the system is physically present within the premises of the enterprise;
- the open Internet: in this case, one needs to find a way to get the IP address of a PADS system "wherever" it is (at a home location, an Internet cafe, within the premises of the enterprise, ...) maybe using some sort of dynamic DNS;

²⁹DNS4EU , refer to <https://www.joindns4.eu/>

- via VPN: this is the most convenient way for remote access as remote access will be possible in every situation provided that the PADSIs has access to the Internet and/or the VPN server, but requires to set up a VPN server.

From the perspective of PADSIs configuration, remote access is configured by creating one or more administration namespace configuration (refer to section 5.1.7 for the details) for each of the remote access situations above (or others), which are not exclusive.

PADSI will configure the networking stack to allow incoming traffic, but will not set up any listening service, this action must be done manually or via automatic configuration tools like Ansible. The remaining of this section illustrates how to set up an SSH service within an administration namespace named “remote-access” here (to be adapted for other services).

The steps are:

- presumably (except if the SSH service must still be run in the “init” namespaces and PADSI’s global configuration allows it) stop and disable the SSH service:

```
# systemctl stop ssh.service
# systemctl disable ssh.service
```

- create an ssh service template dedicated to PADSI (the template will be instantiated per administration namespace used), for example “padsi-ssh@”:

```
# cp /lib/systemd/system/ssh.service /etc/systemd/system/ssh-padsi@.service
```

- adapt the template using the %i placeholder (which will contain the value of the template when used), specifically to:

- run the ssh daemon via the padsi-do tool to which the administration namespace is passed as an argument (the --adminns argument);
- if necessary adapt the environment variables in a dedicated /etc/default/ssh-padsi-%i file where the listening port number may be specified like SSHD_OPTS=-p 2222;

for example the modifications might look like:

```
EnvironmentFile=-/etc/default/ssh-padsi-%i
ExecStart=/usr/bin/padsi-do --adminns %i /usr/sbin/sshd -D $SSHD_OPTS
```

- ask systemd to take into account the new configuration:

```
# systemctl daemon-reload
```

- start, test, and then enable the SSH service for each administration namespace:

```
# systemctl start padsi-ssh@remote-access
# systemctl status padsi-ssh@remote-access
ssh-padsi@remote-access.service - OpenBSD Secure Shell server
   Loaded: loaded (/etc/systemd/system/ssh-padsi@.service; disabled; preset: enabled)
   Active: active (running) since Thu 2025-12-04 10:04:22 CET; 18min ago
  Invocation: cfc7c5bf61964bd1a20f54c85680edf1
     Docs: man:sshd(8)
           man:sshd_config(5)
  Process: 668397 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
 Main PID: 668399 (sshd)
[...]

# systemctl enable padsi-ssh@remote-access
```

5.3.2 Local administration

The padsi-do tool relies on sudo’s configuration to operate, and as such, each user allowed to perform some local administration tasks must:

- be allowed to run padsi-do (possibly limiting the name of the admin namespace to be used);

- be explicitly allowed to run commands which will be executed by `padsi-do`, same as for `sudo`.

For example, to give full admin privileges to the “john” user, one could create a `/etc/sudoers.d/padsi-sys-john` file with the following contents:

```
john ALL=(ALL:ALL) ALL
john ALL=(ALL) NOPASSWD: /usr/bin/padsi-do
```

Notes using this setup (which is useful for a system user without password):

- no password is asked to run `padsi-do`, which is convenient but can be security problem;
- a password is asked for all other commands, which `padsi-do` will ignore as the user already has authenticated to run `padsi-do`.

5.3.3 Unattended upgrades (Debian specific)

Debian’s automatic updates deployment, the “unattended upgrades” feature, relies on a whole pipeline starting at some pre-defined intervals via the `apt-daily-upgrade` systemd timer. For this mechanism to work on a PADSİ system, the process run in the associated `apt-daily-upgrade` service must be started in the admin namespace which provides adequate network access, like for example (where `theadminns` is the admin. namespace to use):

```
ExecStart=/usr/bin/padsi-do --adminns theadminns /usr/lib/apt/apt.systemd.daily install
```

6 Virtual machines administration

This section describes the operations to manage virtual machines, beyond their declaration (refer to section 5.1.6).

6.1 Creating a VM disk image

There are two possibilities to create a VM disk image (and its associated NVRAM file):

- use the usual toolset, install the VM (manually or automatically depending on the toolset), export the VM disk image file and then import it in a PADSİ system; or
- perform a VM installation on a PADSİ system, and use the VM disk image.

In both cases, the PADSİ VM agent, which is required to perform per user VM customization and proper setup (like shared directories) must be installed in the VM. The agent’s installation package should be present in the `packages/` directory if the installation (usually `/usr/share/padsi/packages`).

6.1.1 VM import from an externally created VM

Using non PADSİ tools to install a VM or export a VM is possible, provided that:

- the file format of the VM’s disk image is [QCow](#)
- the VM’s associated NVRAM file is also present.

Import To actually import the VM, use the `padsi-cli vm-import` command (which may take some time depending on the disk image file size).

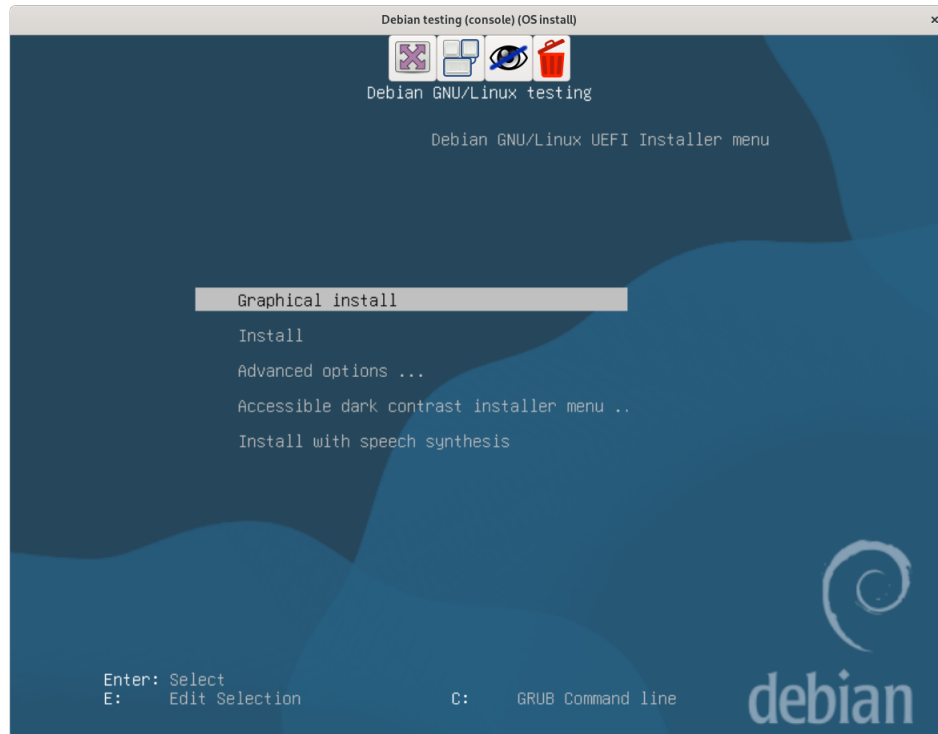
Post installation Use the `padsi-cli vm-publish` command to promote this new installation as a base VM image so users can actually use it.

6.1.2 VM installation using PADSi's tools

Installing a VM via PADSi's tools required that the VM be first defined in the configuration, and specifically that any required network accesses during the installation be identified (at least to download any security updates to make sure the installed VM is up to date).

If any, PADSi will ensure these network accesses are available before the VM boots on its installation media: it will try to find a zone's network compatible with the requirements³⁰ (the user can also specify a zone's network to use, in which case PADSi will ensure its compatibility with the VM's requirements).

Installation To actually start the VM installation, one needs the OS's installation media (ISO file, to be downloaded from the OS's manufacturer or project), and use the `padsi-cli vm-install` command which opens the VM viewer and allows the actual VM installation, like:



Notes:

- if the zone's network requires the usage of a Web proxy, the administrator will have to specify this proxy during the installation as `http://proxy:3128` (for HTTP and HTTPS connections);
- for proper operations, PADSi's VM agent must also be installed (refer to section 2.12.15) and enabled. The associated packages are automatically made available during installation via a virtual CD-ROM.

Once the installation is complete, the system should be properly shut down (if the installation is not Ok, then the VM can simply be discarded), leaving a VM disk image (and its associated files) in the **staged** state.

Post installation Use the `padsi-cli vm-publish` command to promote this new installation as a base VM image so users can actually use it.

³⁰Note that only the zone's network is actually used as a communication means, and restricted to the declared VM's network requirements; the VM will also not be able to access any files accessible to the user in the zone.

6.2 Updating a VM version

6.3 Updating the padsi-agent program

6.3.1 In a Linux VM

To update the padsi-agent, install the associated package.

6.3.2 In a Windows VM

To update the padsi-agent of a virtual machine, the steps are:

1. open a PowerShell console
2. extract the `padsi-agent_<version>_windows.zip` archive
3. ensure PowerShell policy allows script execution:

```
> Set-ExecutionPolicy -ExecutionPolicy Unrestricted -Scope CurrentUser -Force
```
4. enter the extracted directory and run

```
> .\install.ps1
```
5. reset PowerShell policy if necessary:

```
> Set-ExecutionPolicy -ExecutionPolicy Restricted -Scope CurrentUser -Force
```

7 Development

PADSI is mainly written in Python for its universality, its easy of use, its memory safeties and its powerful features, also considering that all PADSI's components don't need to have top performances. Also, the asynchronous features of Python are used when concurrency is required, to make it easier to debug than traditional multi threading.

Before starting, the required tools are:

- Python3 and all the dependencies for PADSI (refer to the `DEBIAN/control-main` file for details)
- Make
- Git

-
- analysing the current situation, debug techniques
 - entering in a zone using `nsenter` or `padsi-sys`
 - packaging

7.1 Technologies used

PADSI's source code uses the following technologies:

- it is mainly written in Python, mainly with the standard modules, all other modules are packages by Debian, so using `pip` or similar to download “unknown” packages should not be required, and the development does not require using virtual environments;
- Linux's namespaces;
- Linux's netfilter firewall (via the `NFTables` tool);
- Unix sockets used as the primary way to connect “providers” and “consumers” within PADSI;
- Bubblewrap to run “bubbles”;
- QEMU for the virtualization.

7.2 Source code layout

This section describes PADSIs source code organization, sorted by top directories:

- **crates**: PADSIs Rust code;
- **DEBIAN**: packaging for the Debian Linux distribution;
- **doc**: PADSIs documentation;
- **etc**: example configuration files;
- **firewall**: simple (very limited) library to “manage” Linux’s netfilter firewall;
- **gnome-shell-extension**: source code of the GNOME Shell extension which outlines application’s windows depending in which zone they are being executed;
- **graphics**: PADSIs icons and such;
- **helpers**: misc. helpers tools;
- **nsbubble**: implementation of “bubbles”, including to run virtual machines;
- **packages**: location where built packages are generated;
- **padsi**: main PADSIs source code, organized as:
 - **cli**: command line tools;
 - **config**: objects used represent a loaded PADSIs configuration;
 - **run**: objects used when PADSIs is creating zones.
- **systemd**: PADSIs systemd programs;
- **testing**: test suites;
- **vm-management**: virtual machines management objects, organized as:
 - **padsi-agent**: the PDSI agent designed to be integrated in all the VMs;
 - **scripts**: ancillary scripts for each supported OS;
 - **viewer**: the VM viewer.

7.3 Locations of files used by a running instance

The PADSIs service relies on both temporary files (mainly the zone’s runtime files) and persistent files (user’s files in the different zones, virtual machines’ disk images, desktop entry and icon files, log files, ...). The runtime files are in the `/run/padsi` directory (which is usually a temporary filesystem) whereas the persistent files are stored by default in the `/var/padsi` directory.

7.3.1 Runtime files

Specifically, the `/run/padsi/` directory contains the **user/** directory for all per-user related data and the **zone-infos/** directory for the information used by the GNOME Shell extension and other components to know which process is running in the context of which zone, and on which window it needs to add an outline and with which color.

The `/run/padsi/user/` contains a sub directory per user’s UID with an opened session, with the following sub directories:

- **userfiles/<zone>/**: the user’s home directory for each zone (assembled via bind mount points);
- **vm-management-files/<zone>/<vm ID>/<UUID>**: containing misc. files related to running virtual machines;
- **<object-type>-<UUID>/**: containing all the run time files needed by the various components of the objects contributing to having a functional zone like Unix socket files and other run time files depending on the context. Object types are:

- INFRA: infrastructure in support for a zone;
- APPS: where a user’s applications are actually running;
- VM: infrastructure in support to run a virtual machine.

7.3.2 Persistent files

All the persistent files (the ones kept between reboots and to be backed up) are in a directory defined by the global configuration, by default `/var/padsi/` (which will be used here). The files are organized as follows:

- the `home/` directory contains all the home directories of all the users and zones, in `home/<UID>/<zone>/` directories (the user has no direct access permissions to these directories as they are not intended to be used from outside any zone);
- the `icons/` directory contains contains all the outlined icons automatically generated for each desktop entry file, refer to 2.8.1. That single directory contains the reproduced tree structure of each XDG data dir variable so the generated desktop entry files are in the “same” directory as the one of the source desktop entry from which they were generated;
- the `log/` directory contains all the per-user and per-component logs outside of syslog generated events, refer to §2.15.
- the `VM/` directory contains all the virtual machines’ disk images;
- the `xdg/` directory contains the various desktop entry files automatically generated, refer to 2.8.1. That single directory contains the reproduced tree structure of each XDG data dir variable so the generated desktop entry files are in the “same” directory as the one of the source desktop entry from which they were generated.

7.4 High level objects

- nsbubble & VM
- firewall

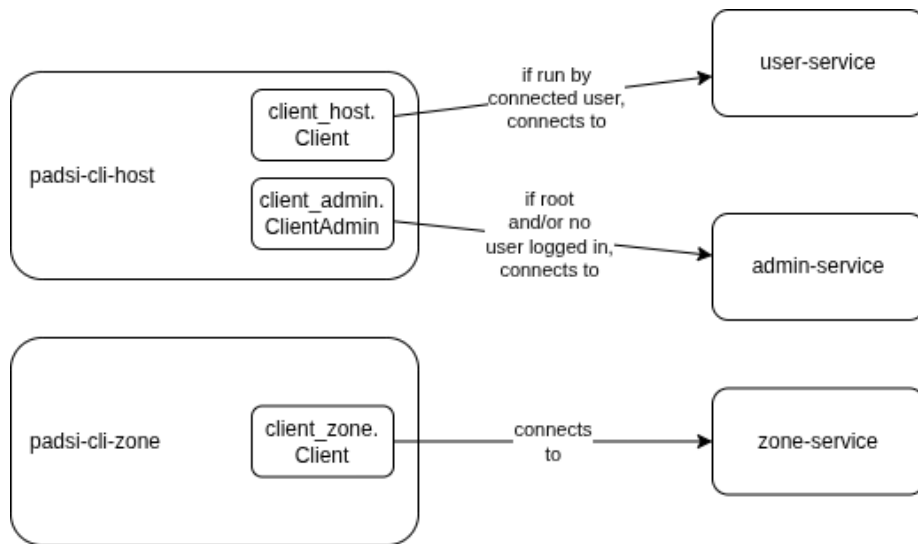
7.5 Run components

- DNS & FW
- DHCP
- wayland proxy
- X11 tricks
- virtiofs
- VM monitor

7.6 Command line tools

7.6.1 padsi-cli

Refer to 2.14 for the variants of the `padsi-cli` tool depending on the environment in which it is executed. All the variants use the same structure as they all use a “Client” object to actually connect to and use the API of the various services (user service, admin service or zone service):



7.7 GNOME shell extension

The GNOME shell extension requires versions 45 and more (Debian 13 and later). PADSIs GNOME shell extension is based on the “[Rounded Window Corners Reborn](#)” extension, as a patch applied to that extension.

Notes

- use the `gnome-extensions` program to manage extensions
- to start a nested GNOME shell:

```
$ MUTTER_DEBUG_DUMMY_MODE_SPECS=1200x1000 dbus-run-session -- gnome-shell --nested --wayland
```
- system wide extensions are installed in `/usr/share/gnome-shell/extensions`

References

- [Getting Started](#) with extensions creation
- [API documentation for writing GNOME applications in JavaScript](#)
- [GJS is a JavaScript runtime](#)

Building This version of the GNOME shell extension uses TypeScript and the associated source code must be “compiled” into JavaScript before it can be packaged.

To build this version of the extension, one can follow the compilation steps from the [rounded-window-corners GitHub page](#), or create a Docker or Podman container to perform just that and avoid installing many dependencies on the system. In that situation, build the extension, just run

```
$ make dist
```

in the extension’s source code directory, the output being the `padsi@dgac.shell-extension.zip` file.

Docker/Podman container setup To build a Docker/Podman image, use the `gnome-shell-extension/DockerFile` file, and, if using Podman, simply run

```
$ make podman-image
```

which will generate the `padsi-typescript` Docker/Podman image.

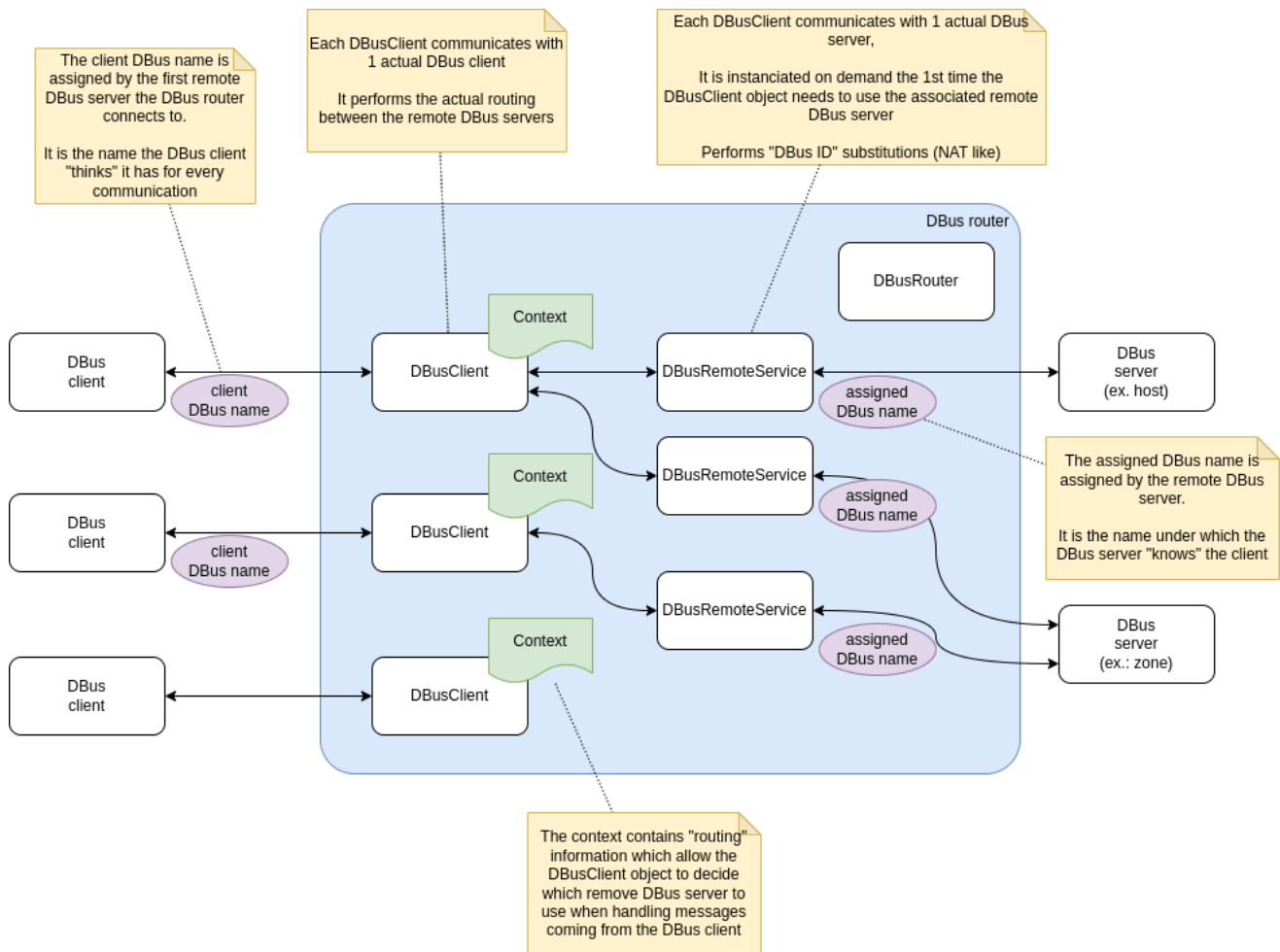
7.8 D-Bus router

Because a D-Bus client's "unique bus connection name" is assigned by the D-Bus server right after the connection is opened, the client receives its D-Bus address (it knows only one) from the first remote D-Bus server the D-Bus router connects to³¹. The D-Bus router thus has to keep track of which bus connection name each client has, of which connection name each remote D-Bus server has assigned to each client, and perform some "translations" of addresses while messages are forwarded between the client and each remote D-Bus server.

The structure of the D-Bus is as follows:

- a global `DBusRouter` object listens for incoming connections on a Unix socket, authenticates each connecting client and creates `DBusClient` objects as necessary;
- each connected D-Bus client is handled by a `DBusClient` object which:
 - receives messages from the D-Bus client and "routes" them to the appropriate remote D-Bus server based in the predefined rules and the "context" (while opening connections to each remote D-Bus server, and authenticating to them), while falling back to a default D-Bus service;
 - analyses messages and extract relevant information into its "context" later used to route future messages (in the form of callbacks assigned to messages and called when the reply associated to the message is received);
- as required, each `DBusClient` object opens a connection to a remote D-Bus server, and creates a `DBusRemoteService` object to handle it;

Here is a summary diagram:



³¹In the case of screen sharing (but possibly for other use cases), the objects which are manipulated across D-Bus messages are derived from that connection name on the session bus, so the first remote D-Bus server the D-Bus router connects to must be the "real" session bus.

The following diagram help understand the situation where a D-Bus client (identified as “CLIENT-0”) connects to the D-Bus router and calls the `GetId()` method on the `org.freedesktop.DBus` interface of the `/org/freedesktop/DBus` object of the `org.freedesktop.DBus` service (the `/bubble/run/dbus-IH...090` is the name of a second D-Bus’s Unix socket):

1. the client connects to the D-Bus router, calls the `Hello()` method, and get its connection name (“:1.434” here), the D-Bus router simply forwards the messages between the client and the “real” host D-Bus session;
2. the client calls the `Introspect()` method which is “routed” to the other D-Bus service by the D-Bus router (based on its rules). Before forwarding that message to the selected D-Bus server, the D-Bus router opens a connection to that bus, calls the `Hello()` method, and catches the connection name assigned (“:1.0” here); this is done in the “say `Hello()`” frame;
3. the D-Bus router then sends the `Introspect()` call message to the D-Bus server, and, when reply come, translates the destination address before relaying the reply to the client;
4. the client then calls the `GetId()` method, and the D-Bus router simply “routes” messages back and forth while applying addresses translation.

bus and receive method calls via D-Bus;

- compared to `xdg-dbus-proxy`:
 - `xdg-dbus-proxy`'s role is to filter D-Bus calls to a single remote D-Bus service, it can't route between several remote D-Bus servers.
 - `xdg-dbus-proxy` allows unlimited access to the session's bus service `org.freedesktop.DBus`, and more as specified by filters, the D-Bus router allows unlimited access but the application can't choose which remote D-Bus server to use.

Debugging tips Refer to the following log files:

- `/tmp/dbus-router.log`: contains all the inner events while handling messages, including the BASE64 encoded messages which can be analyzed using the [python-dbus-next](#) module via some private objects, e.g.:

```
>>> from dbus_next._private.unmarshaller import *
>>> fd=open("msg.bin", "rb")
>>> u=Unmarshaller(fd)
>>> msg=u.unmarshall()
>>> # analyse the msg object
```

- the log file can be visualized using the `dbus-logs-visu.py` tool which will generate³² UML sequence diagrams in PDF files (the output being like the one above). For example:

```
$ dbus-logs-visu.py --out report.pdf /tmp/dbus-router.log report
```

will generate a `report.pdf` file, while

```
$ dbus-logs-visu.py /tmp/dbus-router.log list
20250519-09:17:07
20250519-09:17:19
```

will list all the timestamps where some D-Bus events have been logged. Use this information to limit the size of the generated files via the “`--since`” or “`--until`” command line options.

References

- [D-Bus Specification](#)
- [Portal Documentation](#) from Flatpak
- [XDG Desktop Portal](#) Web page from Flatpak
- [xdg-desktop-portal-gnome](#)
- ArchLinux's [XDG Desktop Portal](#) wiki

7.9 Development environment

Here is a list of packages to set up a development environment for PADS on Debian:

```
# apt -y install dpkg-dev wireguard lyx make rsync borgbackup curl python3-colorama tcpdump wireshark
```

Also install Draw.io, which can easily be done:

```
# apt -y install flatpak
# flatpak remote-add --if-not-exists flathub https://dl.flathub.org/repo/flathub.flatpakrepo
# flatpak install com.jgraph.drawio.desktop
```

³²This relies on the [PlantUML tool](#).

7.10 Virtual Machines

7.10.1 Scripts

The following environment variables (along with examples) are set up when the PADSI agent runs scripts:

```
PADSI_ETC_DIR=/run/padsi-agent/etc
PADSI_GROUP_ID=1000
PADSI_GROUP_NAME=groupname
PADSI_LANG=fr_FR.UTF-8
PADSI_LIB_DIR=/run/padsi-agent/lib
PADSI_USER_DIR=/run/padsi-agent/user
PADSI_USER_FULLNAME=JohnDoe
PADSI_USER_ID=1000
PADSI_USER_NAME=jonhdoe
PADSI_USER_SHELL=/bin/bash
PADSI_VM_CONFIG=debian-noui-13
PADSI_VM_NAME=snap.1
PADSI_VM_NICKNAME=demo
PADSI_VM_USAGE=RUN
```

7.11 Misc.

7.11.1 Processes tree

The process tree while running PADSI looks like the following:

```
padsi-system-se+-monitor-desktop
| -monitor-usb-dev
| -user-service--+2*[bwrap---init.py+-padsi-dns-fw]
| | | -padsi-dns-serve---unbound]
| | | '---{init.py}}
| | -2*[bwrap---init.py+-dbus-daemon]
| | | '---{init.py}}
| | -bwrap---init.py+-dbus-router
| | | '---{init.py}}
| | -bwrap---init.py+-fallback-server
| | | -padsi-dns-fw
| | | | -padsi-dns-serve---unbound
| | | | '---{init.py}}
| | -bwrap---init.py+-at-spi-bus-laun+-dbus-daemon
| | | | -4*[{at-spi-bus-laun}]
| | | | -at-spi2-registr---3*[{at-spi2-registr}]
| | | | -dbus-daemon
| | | | -dconf-service---3*[{dconf-service}]
| | | | -gnome-terminal--+bash---nautilus---41*[{nautilus}]
| | | | | -5*[{gnome-terminal-}]
| | | | -goa-daemon---6*[{goa-daemon}]
| | | | -goa-identity-se---5*[{goa-identity-se}]
| | | | -gvfs-afc-volume---5*[{gvfs-afc-volume}]
| | | | -gvfs-goa-volume---4*[{gvfs-goa-volume}]
| | | | -gvfs-gphoto2-vo---4*[{gvfs-gphoto2-vo}]
| | | | -gvfs-mtp-volume---4*[{gvfs-mtp-volume}]
| | | | -gvfs-udisks2-vo---4*[{gvfs-udisks2-vo}]
| | | | -gvfsd+-gvfsd-network---5*[{gvfsd-network}]
| | | | | -gvfsd-recent---5*[{gvfsd-recent}]
| | | | | -gvfsd-trash---8*[{gvfsd-trash}]
| | | | | -4*[{gvfsd}]
| | | | -gvfsd-fuse---6*[{gvfsd-fuse}]
| | | | -localsearch-3---9*[{localsearch-3}]
| | | | -org.gnome.Nauti---40*[{org.gnome.Nauti}]
| | | | -xdg-desktop-por---6*[{xdg-desktop-por}]
| | | | -xdg-desktop-por---4*[{xdg-desktop-por}]
| | | | -xdg-document-po+-python3
| | | | | -7*[{xdg-document-po}]
| | | | -xdg-permission---3*[{xdg-permission-}]
| | | | '---{init.py}}
| | -monitor-desktop
| | -mount-server.py
| | -mount-server.py---fusermount3
| | -netlink-proxy
| | -notif-server.py
| | -3*[padsi-zone-serv]
| | -4*[{user-service}]
| | '---{padsi-system-se}
```

where the situation is:

- one single user is logged in;

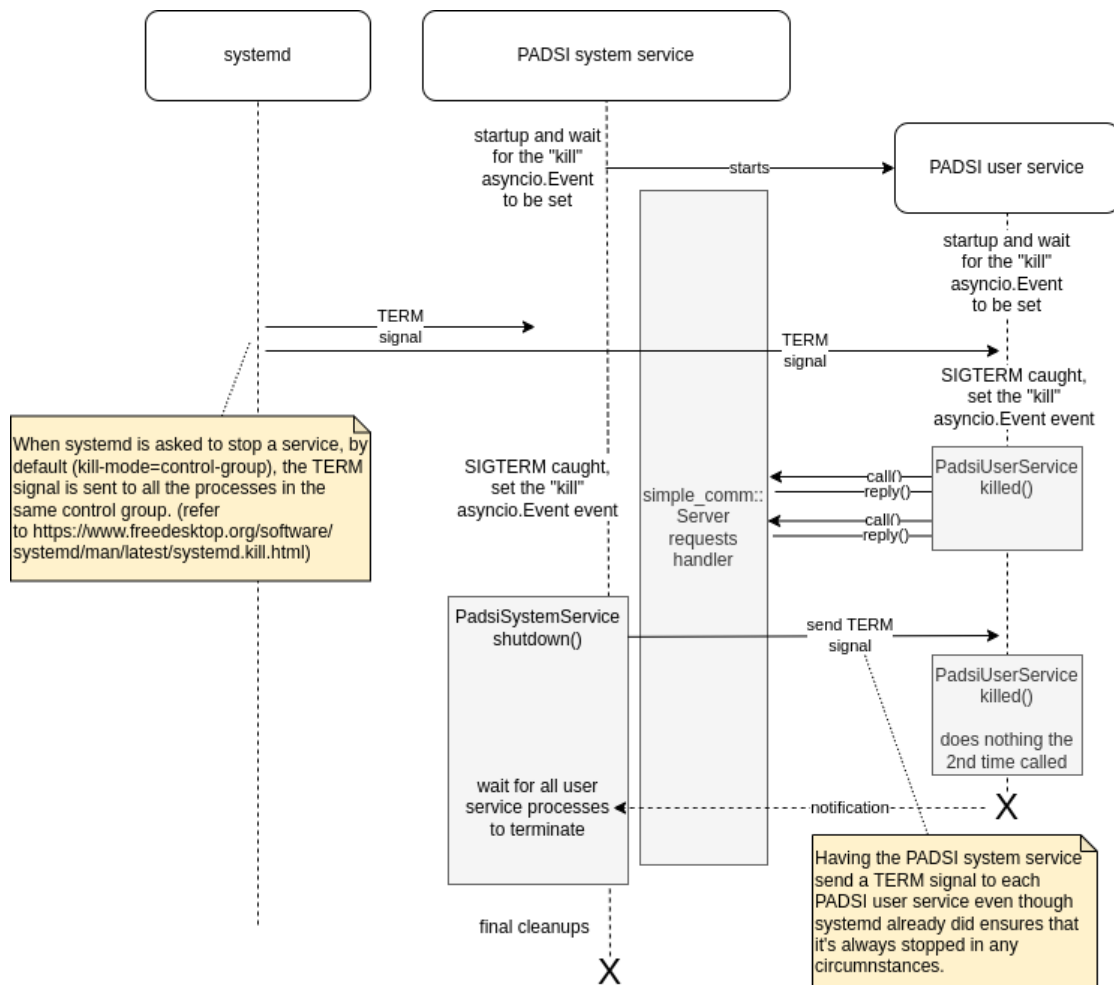
- 3 zones are started, but the user is actually running processes (a GNOME terminal and the Nautilus file manager started manually in the terminal) in a single zone.

Notes:

- the “NS changes” denotes a namespaces isolation;
- the `monitor-desktop-entries` processes implement the desktop entries monitor (refer to section 2.8.1) both system wide and per user;
- the `zone-service` processes implement the zone service for each zone;
- the `mount-server.py` processes implement “mount-server” of the FUSE features (refer to section 2.7).

7.11.2 PADSİ system and user services lifecycle

The PADSİ system service is launched and terminated by systemd, and it itself spawns as many user service processes as there are users logged at any time. The following diagram illustrates this lifecycle (with only one user logged in for clarity):



Notes:

- both the system and user services rely heavily on pytjon asnycio;
- this diagram does not show all the processes run by the user service.

A Windows VM installation

PADSI uses the [virtIO](#) system to para-virtualize hardware seen by guest operating systems in virtual machines. Linux natively includes the associated drivers but Windows does not. The drivers must thus be loaded before installing Windows in the VM as otherwise the virtual hard disk will not be “seen” by the Windows installer.

The drivers can be downloaded from:

- <https://fedorapeople.org/groups/virt/virtio-win/direct-downloads/> (multiple formats available);
- <https://github.com/virtio-win/virtio-win-pkg-scripts/blob/master/README.md>

Also, access to actual files in the host from the VM is implemented using [Virtiofs](#). Again, Windows is not natively aware of this filesystem and specific <https://winfsp.dev> drivers must be installed:

- download from <https://github.com/winfsp/winfsp/releases> or <https://winfsp.dev/rel/>;
- silent installation can be done, adapt to exact version:

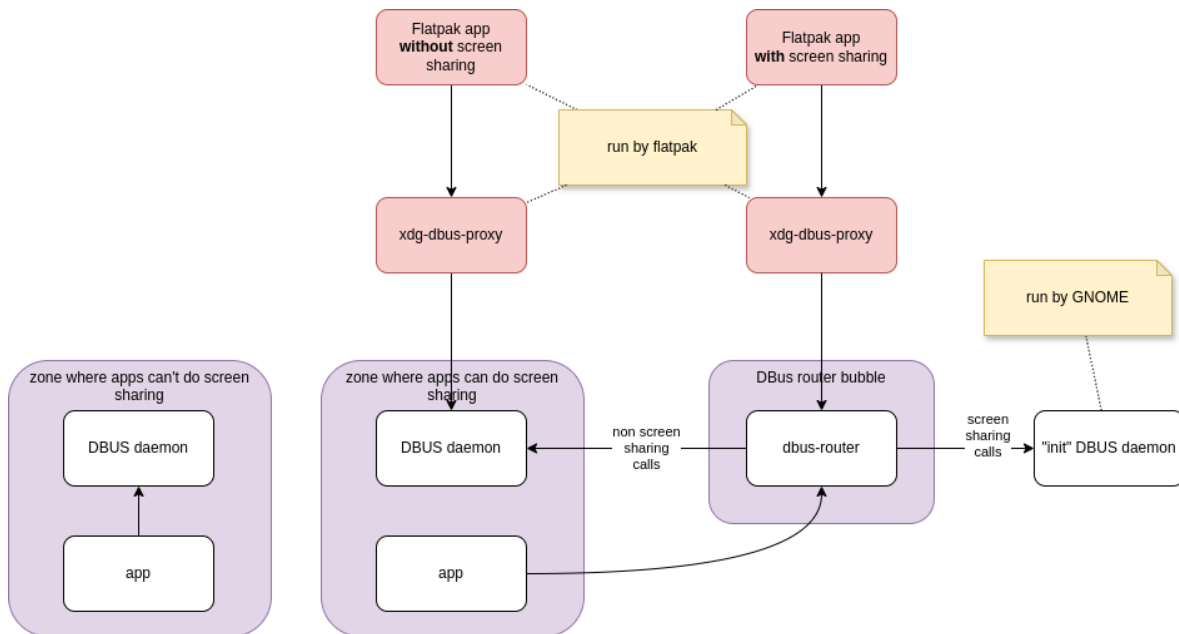
```
msiexec /i winfsp-2.0.23075.msi /qn INSTALLLEVEL=1000
```

Once the virtual machine has been installed, either manually or via some automatic mechanism³³, PADSI’s VM agent (refer to section 2.12.15) must be installed:

1. locate the `padsi-vm-agent_XXX_windows.zip` file³⁴ and extract it;
2. start a powershell console and run the `install.ps1` script which takes care of removing any previous version of the VM agent and install the new one.

B Screen sharing

To have a program in a zone share the whole screen, it needs access to some objects on the “real” D-Bus session bus. PADSI achieves this using the D-Bus router (refer to section 2.8.7):



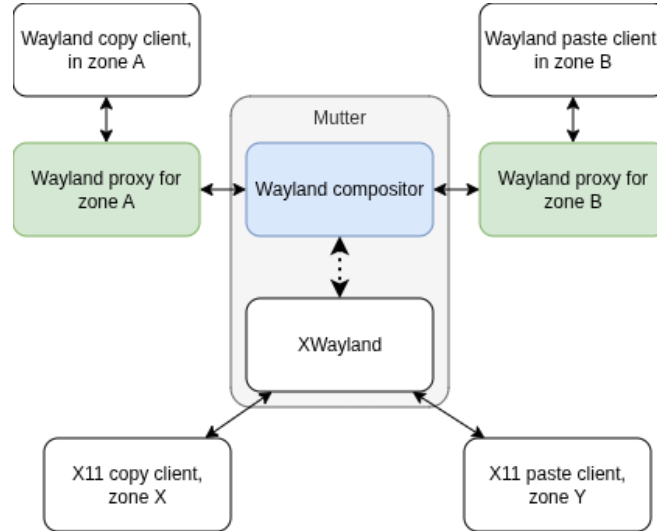
³³For example using an `autoattend.xml` answer file which contains all the settings. Creating this file is not trivial, and some sites on the Internet like <https://schneegans.de/windows/unattend-generator/> can help there.

³⁴If the VM is running via the `padsi-cli vm-update` command, this file should be present in a virtual CD-ROM attached to the VM, probably as `D:`

C Wayland proxy

This annex details the implementation of the Wayland proxy, used in the context of clipboard sharing restriction (refer to section 2.8.4).

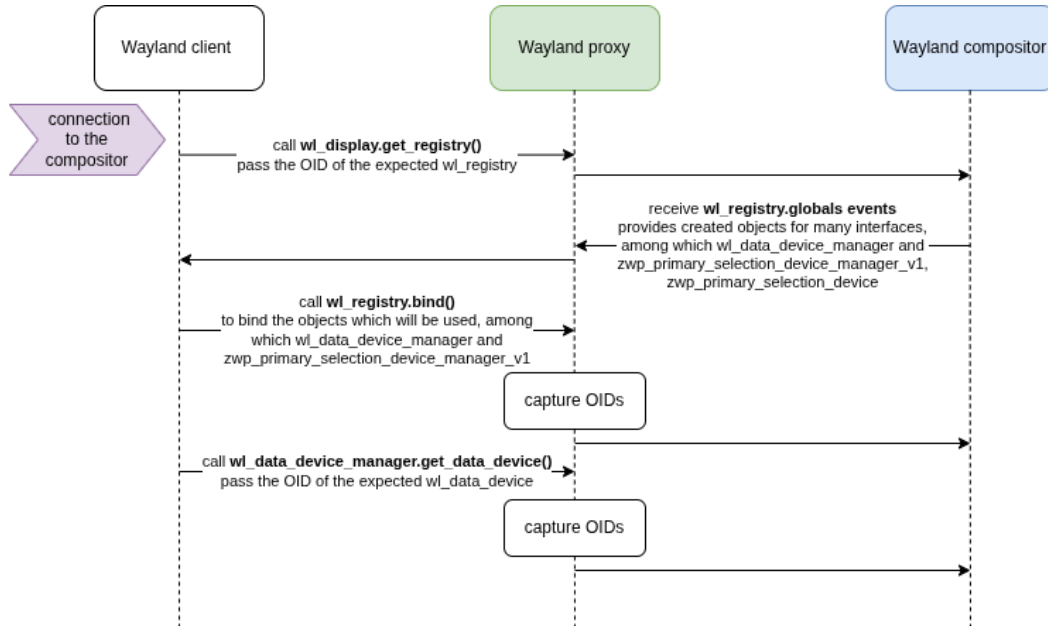
The following diagram illustrates how the Wayland proxy is deployed, note that the X11 clients' communications are not intercepted by any Wayland proxy and thus copy/paste policies cannot be implemented when they are involved³⁵.



C.1 Connection to a Wayland compositor's cinematic

In order to understand how the Wayland proxy works, having the connection, and copy/paste cinematic is important.

When a Wayland client connects to the Wayland server (the **compositor**), the following happens with regards to the copy/paste feature:

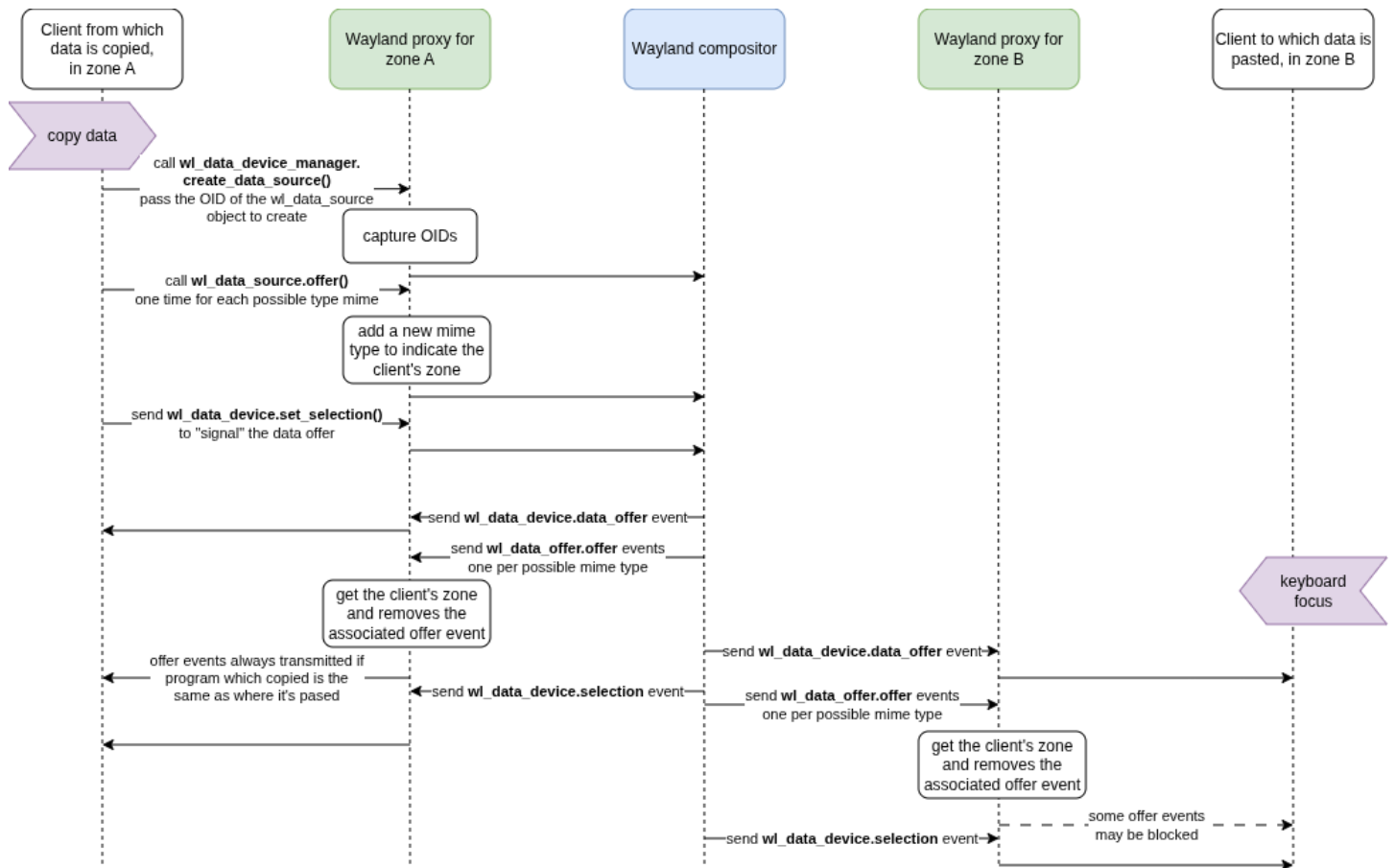


The Wayland proxy analyses the messages and determine the object IDS (OIDs) of the Wayland objects which will later be used while implementing the restrictions.

³⁵Creating an X11 proxy could be done but, considering how unsecure X11 is and its deprecation phase, the effort does not seem worth the result.

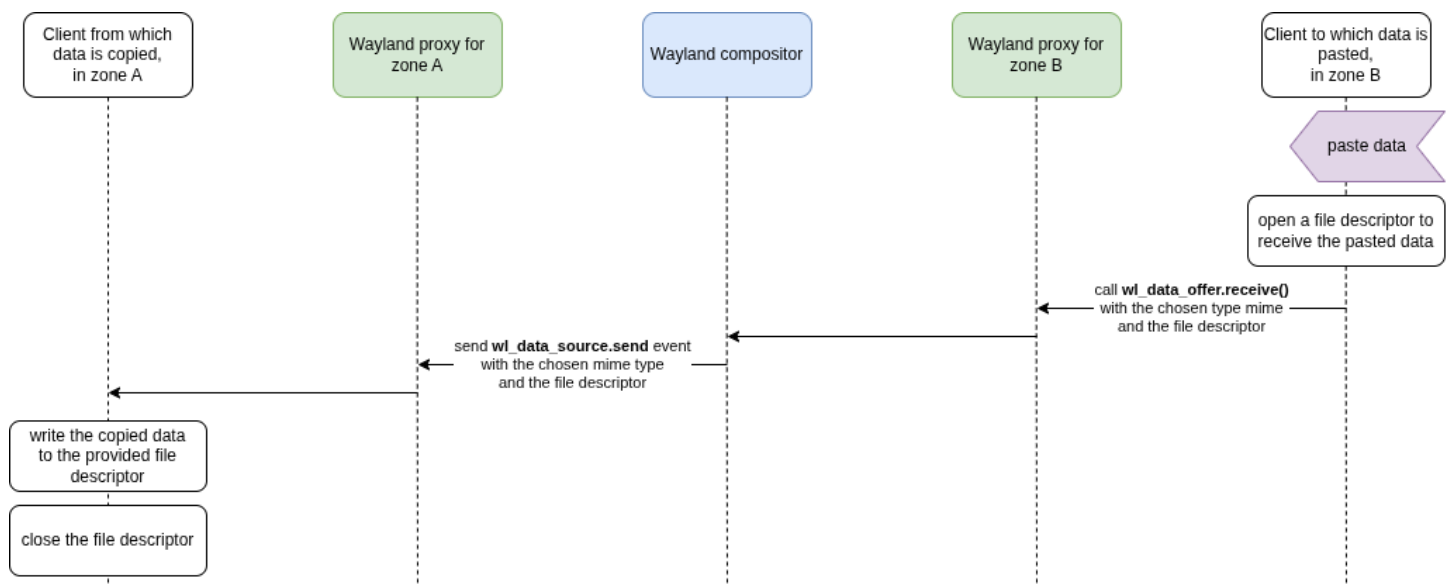
C.2 Copy/paste cinematic

When data is copied by a Wayland client application, the following happens:



Note: the application from which data was copied always receives the notification (via **selection** events) from the compositor because it has the focus.

The following schema shows what happens when the Wayland client wants to paste the copied data, the Wayland proxy does not perform any filtering at this stage.



C.3 Implementation details

C.3.1 X11 filtering

The X11 windowing system provides a rudimentary, text only, copy/paste mechanism. The Wayland compositor (as it usually also implements the X11 protocol via the XWayland program) can also handle this mechanism via the [Primary selection protocol](#) (the native mechanism being part of the [Wayland core protocol](#)).